

Arduino Sensors and Actuators

Outline

- ▶ Introduction to Arduino Sensors
- ▶ Types of Arduino Sensors
- ▶ DHT11 Digital Humidity and Temperature Sensor
- ▶ Light Dependent Resistor (LDR) Sensor
- ▶ HC-SR04 Ultrasonic Sensor
- ▶ HC-SR501 PIR Sensor
- ▶ Introduction to Arduino Actuators
- ▶ Types of Arduino Actuators
- ▶ SG90/MG996R Servo Motor
- ▶ Electromagnetic Relay
- ▶ 16x2 LCD Display

Introduction to Arduino Sensors

- A sensor is a device that detects and measures physical properties from the environment (like temperature, humidity, or motion) and converts them into data that can be processed by Embedded Systems or Internet of Things (IoT) devices.
- In Embedded Systems and IoT, sensors play a key role by providing real-time data, enabling devices to monitor and respond to their surroundings autonomously.
- This data can be used locally or transmitted over networks, making sensors essential for automation, monitoring, and smart applications.

Types of Arduino Sensors

- ▶ Temperature Sensors
 - ▶ Examples: LM35, DS18B20, DHT11, DHT22.
 - ▶ Purpose: Measure ambient temperature and convert it into an electrical signal. Often used in weather stations, HVAC systems, and smart home automation.
- ▶ Humidity Sensors
 - ▶ Examples: DHT11, DHT22.
 - ▶ Purpose: Measure relative humidity, often paired with temperature sensors. Commonly used in agriculture, HVAC systems, and environment monitoring applications.

Types of Arduino Sensors

- ▶ Light Sensors
 - ▶ Examples: Photoresistor (LDR), BH1750, TSL2561.
 - ▶ Purpose: Detect light levels to adjust lighting systems, control displays, or trigger actions in IoT devices based on ambient light.
- ▶ Motion Sensors
 - ▶ Examples: Passive Infrared (PIR) sensor, Accelerometer (ADXL345), Gyroscope (MPU6050).
 - ▶ Purpose: Detect movement and orientation changes. Used in security systems, fitness trackers, and gaming controllers.

Types of Arduino Sensors

- ▶ Proximity and Distance Sensors
 - ▶ Examples: Ultrasonic sensor (HC-SR04), IR proximity sensor, LiDAR.
 - ▶ Purpose: Measure the distance to an object or detect its proximity. Useful in robotics, automated doors, and parking assistance.
- ▶ Gas and Air Quality Sensors
 - ▶ Examples: MQ-series sensors (MQ-2 for smoke, MQ-135 for air quality).
 - ▶ Purpose: Detect gases like carbon monoxide, methane, and pollutants in the air. Used in pollution monitoring, fire alarms, and industrial safety applications.

Types of Arduino Sensors

- ▶ Pressure Sensors
 - ▶ Examples: BMP180, BMP280, MPX5700.
 - ▶ Purpose: Measure air or liquid pressure, often used in weather stations, altitude measurement, and industrial control systems.
- ▶ Sound Sensors
 - ▶ Examples: Microphone sensors, sound level meters.
 - ▶ Purpose: Detect sound intensity or frequency, enabling applications like voice recognition, noise monitoring, and sound-activated devices.

Types of Arduino Sensors

- ▶ Touch Sensors
 - ▶ Examples: Capacitive touch sensor (TTP223), Force-sensitive resistor.
 - ▶ Purpose: Detect touch or pressure. Used in smart home interfaces, mobile devices, and interactive displays.
- ▶ Vibration Sensors
 - ▶ Examples: Piezoelectric sensors, vibration switches.
 - ▶ Purpose: Detect vibrations and provide feedback for equipment diagnostics, industrial monitoring, and security systems.

Types of Arduino Sensors

- ▶ Flow Sensors
 - ▶ Examples: Water flow sensor (YF-S201), gas flow sensors.
 - ▶ Purpose: Measure the rate of fluid or gas flow. Commonly used in water management, irrigation, and industrial gas flow monitoring.
- ▶ Infrared Sensors
 - ▶ Examples: Infrared (IR) sensors, Flame sensors.
 - ▶ Purpose: Detect infrared light to identify heat or fire sources. Often used in security systems, fire alarms, and automation.

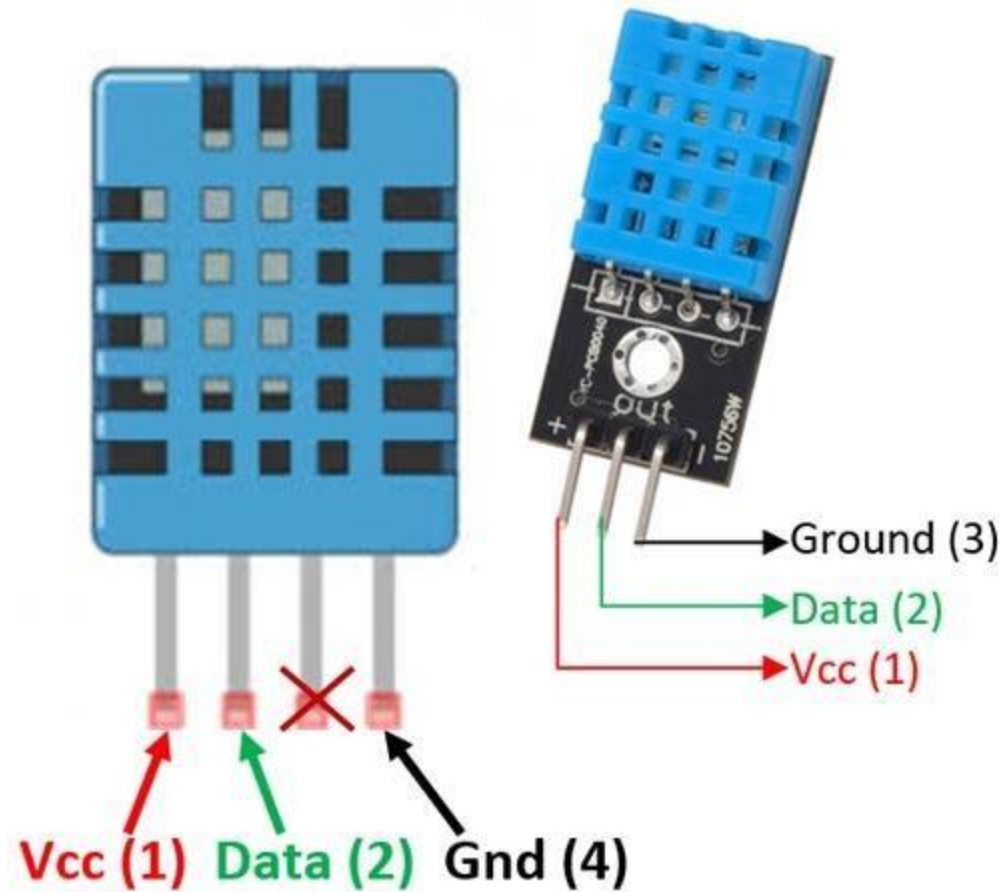
Types of Arduino Sensors

- ▶ Magnetic Sensors
 - ▶ Examples: Hall Effect sensors, magnetometers.
 - ▶ Purpose: Detect magnetic fields. These sensors are commonly used in compasses, automotive systems, and current-sensing applications.
- ▶ Image and Vision Sensors
 - ▶ Examples: Cameras, CCD sensors, CMOS sensors.
 - ▶ Purpose: Capture visual data for processing. Used in computer vision, surveillance, facial recognition, and smart vehicle applications.

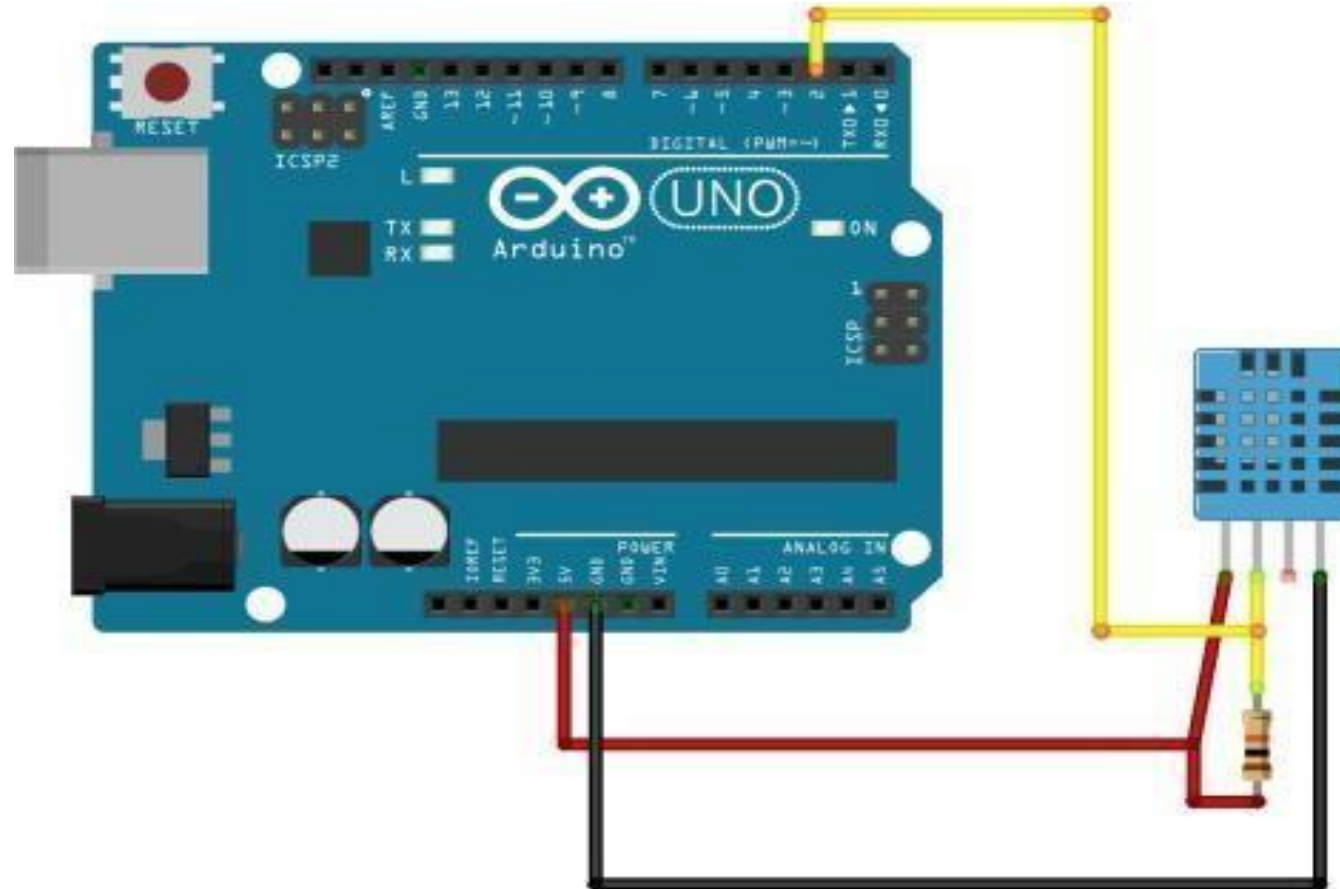
Types of Arduino Sensors

- ▶ Chemical Sensors
 - ▶ Examples: pH sensors, ion-selective electrodes.
 - ▶ Purpose: Detect chemical concentrations or compositions. Useful in agriculture, medical devices, and laboratory applications.

DHT11 Digital Humidity and Temperature Sensor



DHT11 Interfacing Diagram



DHT11 Interfacing Code

```
#include "DHT.h"
#define DHTPIN 2    // what digital pin we're connected to
#define DHTTYPE DHT11 // DHT 11 // set DHT sensor model to be DHT11
DHT dht(DHTPIN, DHTTYPE);

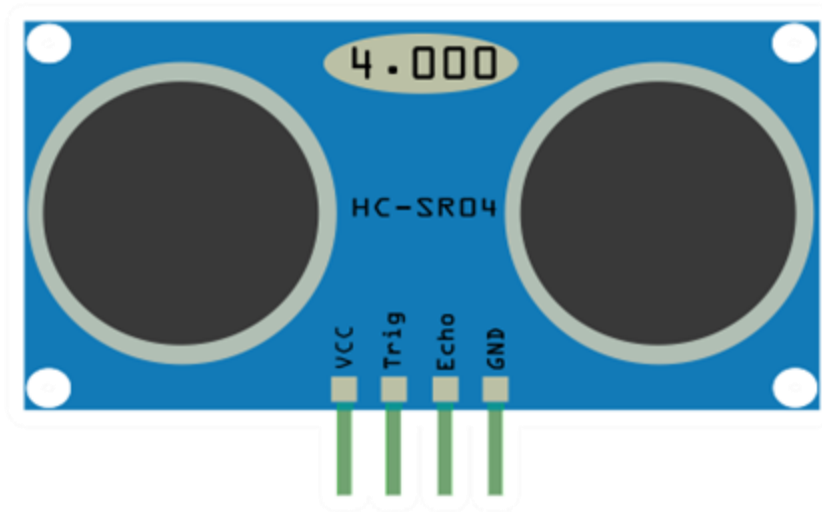
void setup() {
  Serial.begin(9600);
  Serial.println("DHTxx test!");
  dht.begin();
}

void loop() {
  delay(2000);
  float h = dht.readHumidity();
  float t = dht.readTemperature();

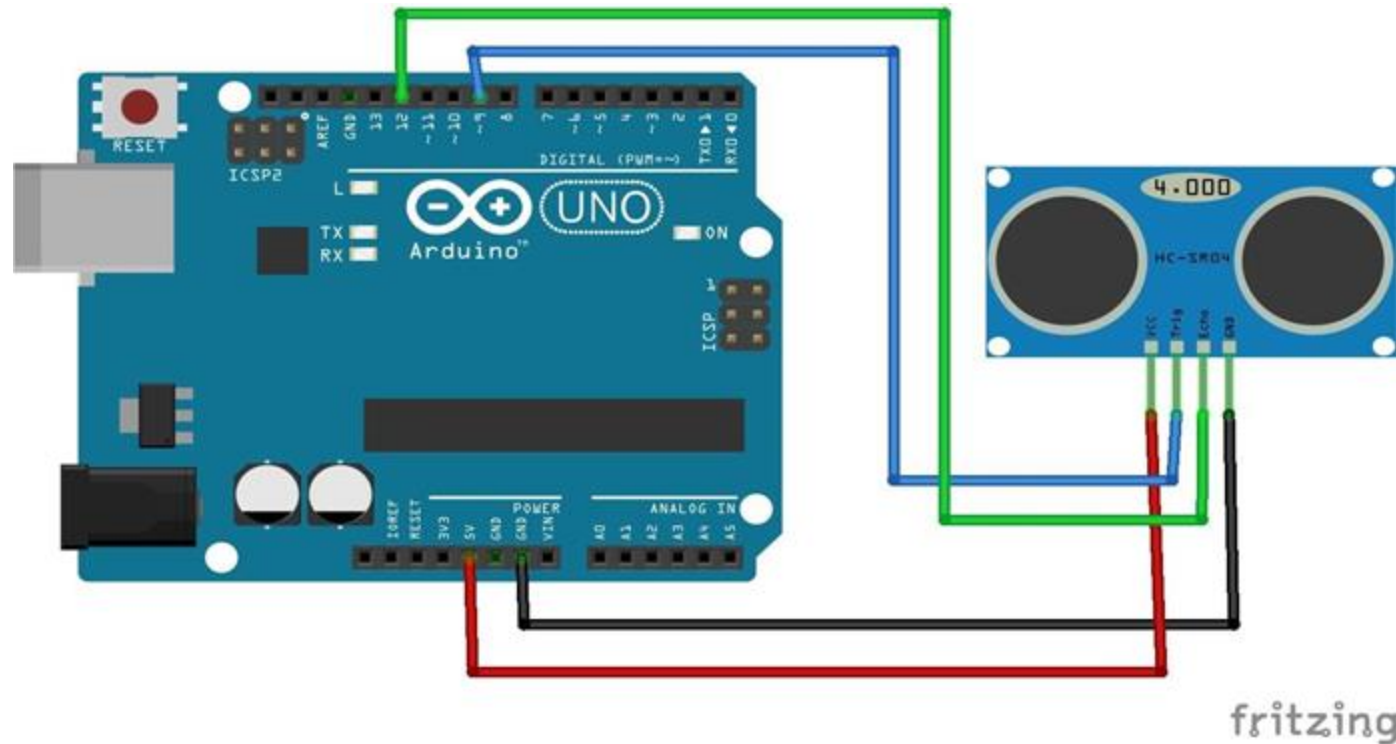
  if (isnan(h) || isnan(t) || isnan(f)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Humidity: ");
  Serial.print(h);
  Serial.print(" %\t");
  Serial.print("Temperature: ");
  Serial.print(t);
  Serial.print(" *C ");
}
```

HC-SR04 Ultrasonic Sensor



HC-SR04 Interfacing Diagram



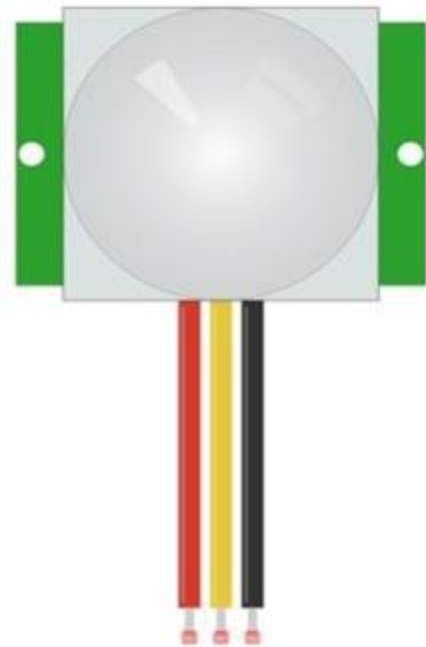
HC-SR04 Interfacing Code

```
int trigPin = 11;
int echoPin = 12;
long duration, cm;

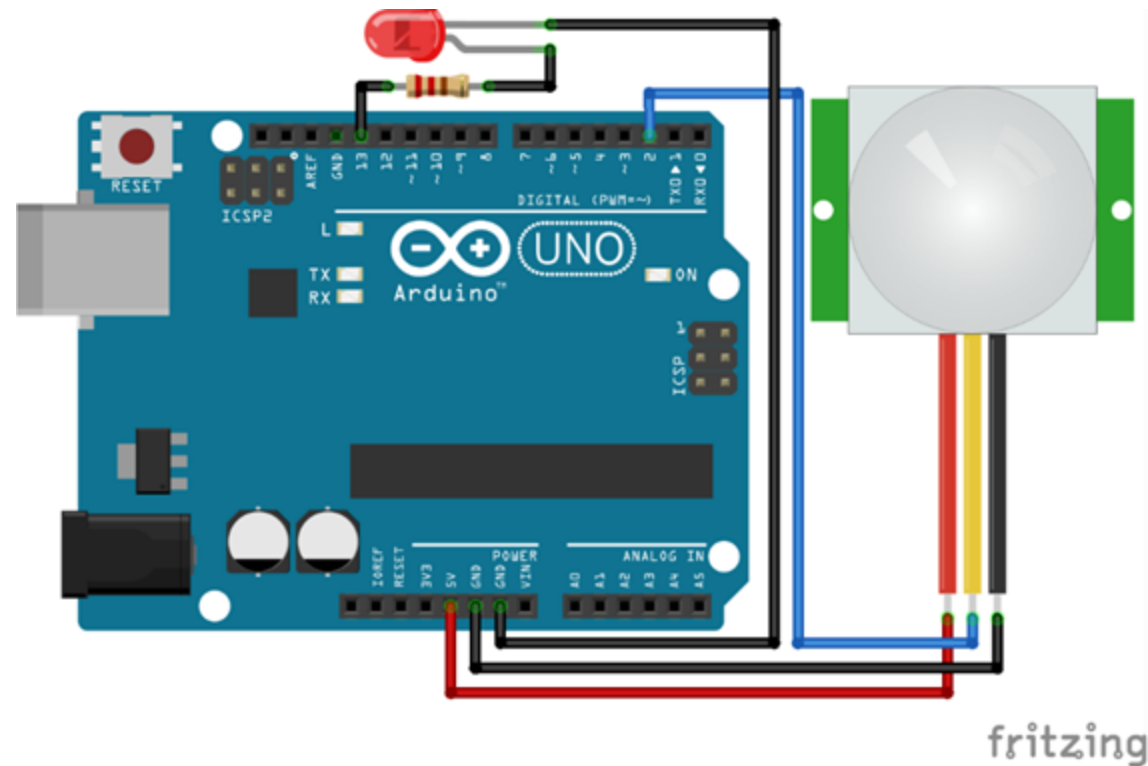
void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(5);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  pinMode(echoPin, INPUT);
  duration = pulseIn(echoPin, HIGH);
  cm = (duration/2) / 29.1;
  Serial.print(cm);
  Serial.print("cm");
  Serial.println();
  delay(250);
}
```

HC-SR501 Passive Infrared (PIR) Sensor



HC-SR501 Interfacing Diagram



HC-SR501 Interfacing Code

```
int led = 13;
int sensor = 2;
int state = LOW;
int val = 0;

void setup() {
  pinMode(led, OUTPUT);
  pinMode(sensor, INPUT);
  Serial.begin(9600);
}

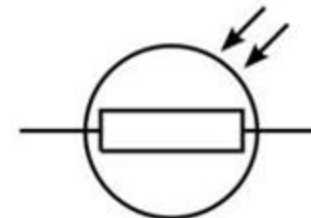
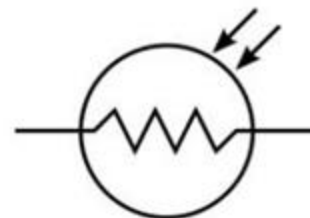
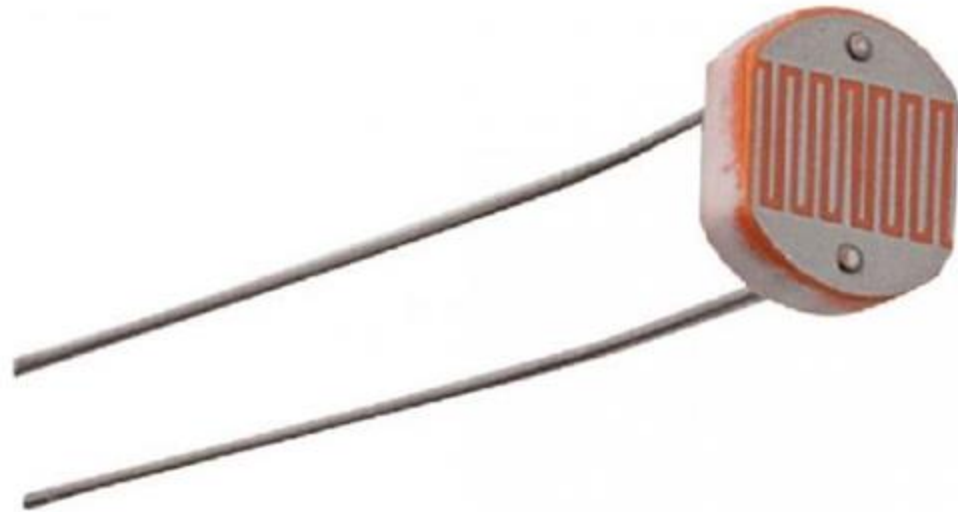
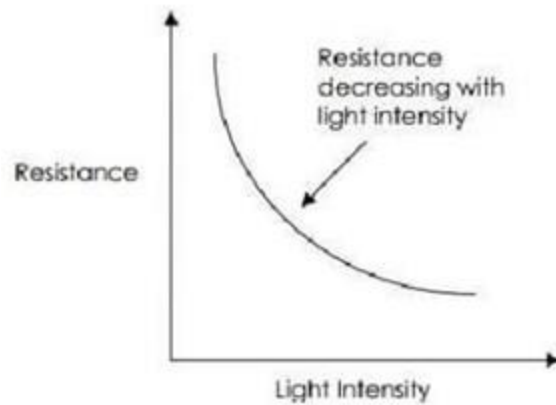
void loop() {
  val = digitalRead(sensor);

  if (val == HIGH) {
    digitalWrite(led, HIGH);
    delay(100);

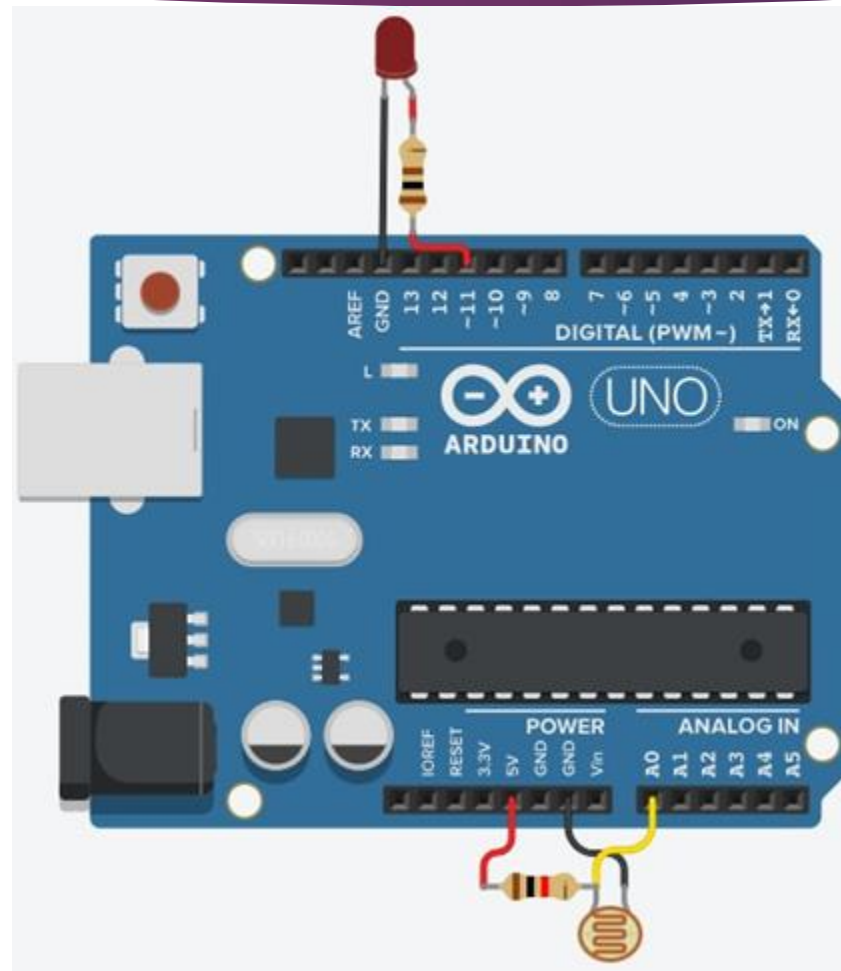
    if (state == LOW) {
      Serial.println("Motion detected!");
      state = HIGH;
    }
  }
  else {
    digitalWrite(led, LOW);
    delay(200);

    if (state == HIGH) {
      Serial.println("Motion ended!");
      state = LOW;
    }
  }
}
```

Light Dependent Resistor (LDR) Sensor



LDR Interfacing Diagram



LDR Interfacing Code

```
int ledPin = 11;
int ldrPin = A0;
int ldrStatus = 0;
int brightness = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  ldrStatus = analogRead(ldrPin);
  Serial.println(ldrStatus);
  brightness = map(ldrStatus, 344, 1017, 0, 255);
  analogWrite(ledPin, brightness);
  delay(100);
}
```

Introduction to Arduino Actuators

- Actuators bring life to electronic projects by converting electrical signals from the microcontroller into real-world physical actions.
- They enable devices to move, create sounds, produce heat, and perform various functions that allow a project to interact dynamically with its environment.
- Actuators are essential components in fields like robotics, automation, home devices, and IoT systems because they make it possible for devices to affect the physical world around them.

Introduction to Arduino Actuators

- Common types of Arduino-compatible actuators include motors (DC, servo, and stepper), which can rotate or move with precision, solenoids for linear pushing and pulling actions, relays to control high-power devices, and vibrators or buzzers for alerts and feedback.
- Together, these actuators empower hobbyists, engineers, and creators to build everything from robots and automated systems to interactive art installations and prototypes.
- With Arduino's simple interface, controlling actuators becomes accessible, opening up endless possibilities for innovation and practical applications.

Types of Arduino Actuators

► **DC Motors**

- **Description:** These motors convert electrical energy into rotational motion, making them useful for driving wheels, fans, and other rotational mechanisms.
- **Applications:** Robotic cars, fans, propellers.
- **Control:** Usually controlled through an H-bridge motor driver or a transistor circuit.

Types of Arduino Actuators

► Servo Motors

- **Description:** Servos allow for precise control of angular position, often within 0 to 180 degrees.
- **Applications:** Robotic arms, pan-tilt mechanisms for cameras, grippers.
- **Control:** Controlled by PWM (Pulse Width Modulation) signals, usually straightforward with Arduino Servo library.

Types of Arduino Actuators

► **Stepper Motors**

- **Description:** These motors rotate in small "steps," allowing for precise control over rotation.
- **Applications:** CNC machines, 3D printers, precise positioning systems.
- **Control:** Requires a driver, and control is typically through specific step and direction signals.

Types of Arduino Actuators

► Solenoids

- **Description:** Solenoids are electromagnets that create linear motion, pulling or pushing when powered.
- **Applications:** Locks, valves, pinball machines, electromechanical triggers.
- **Control:** Often controlled through a transistor or relay circuit for higher currents.

Types of Arduino Actuators

► Relays

- **Description:** An electromechanical switch that can turn on or off a high-power circuit using a low-power Arduino signal.
- **Applications:** Controlling high-power devices like lights, fans, or motors that operate at higher voltages.
- **Control:** Controlled by sending a signal to trigger the relay, typically through a transistor circuit.

Types of Arduino Actuators

▶ **Linear Actuators**

- **Description:** These actuators convert rotational motion into linear (back-and-forth) movement.
- **Applications:** Robotics, home automation (like moving windows or doors).
- **Control:** Requires a motor driver or relay circuit, depending on the type.

Types of Arduino Actuators

▶ **Piezo Buzzers and Vibration Motors**

- **Description:** Small actuators that produce sound or vibrations.
- **Applications:** Alerts, notifications, haptic feedback.
- **Control:** Directly controlled with digital pins or PWM for varying intensity.

Types of Arduino Actuators

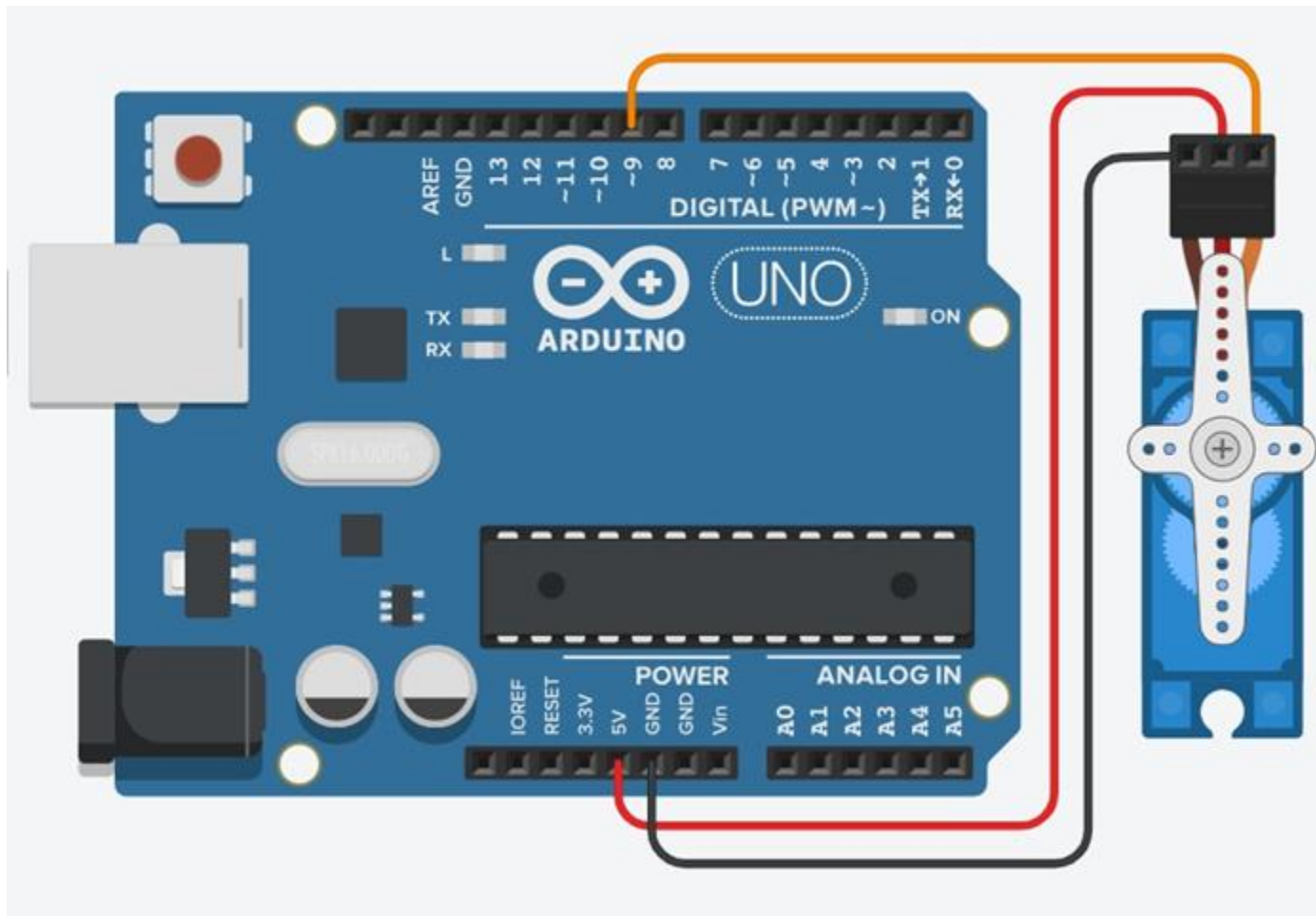
► Heating Elements

- **Description:** Heating elements generate heat instead of motion and are often used in applications that require temperature control.
- **Applications:** Soldering iron control, 3D printer heated beds.
- **Control:** Controlled through relays or MOSFETs for higher current handling.

SG90/MG996R Servo Motor



SG90/MG996R Servo Motor Interfacing Diagram



SG90/MG996R Servo Motor Interfacing Code

```
#include <Servo.h>

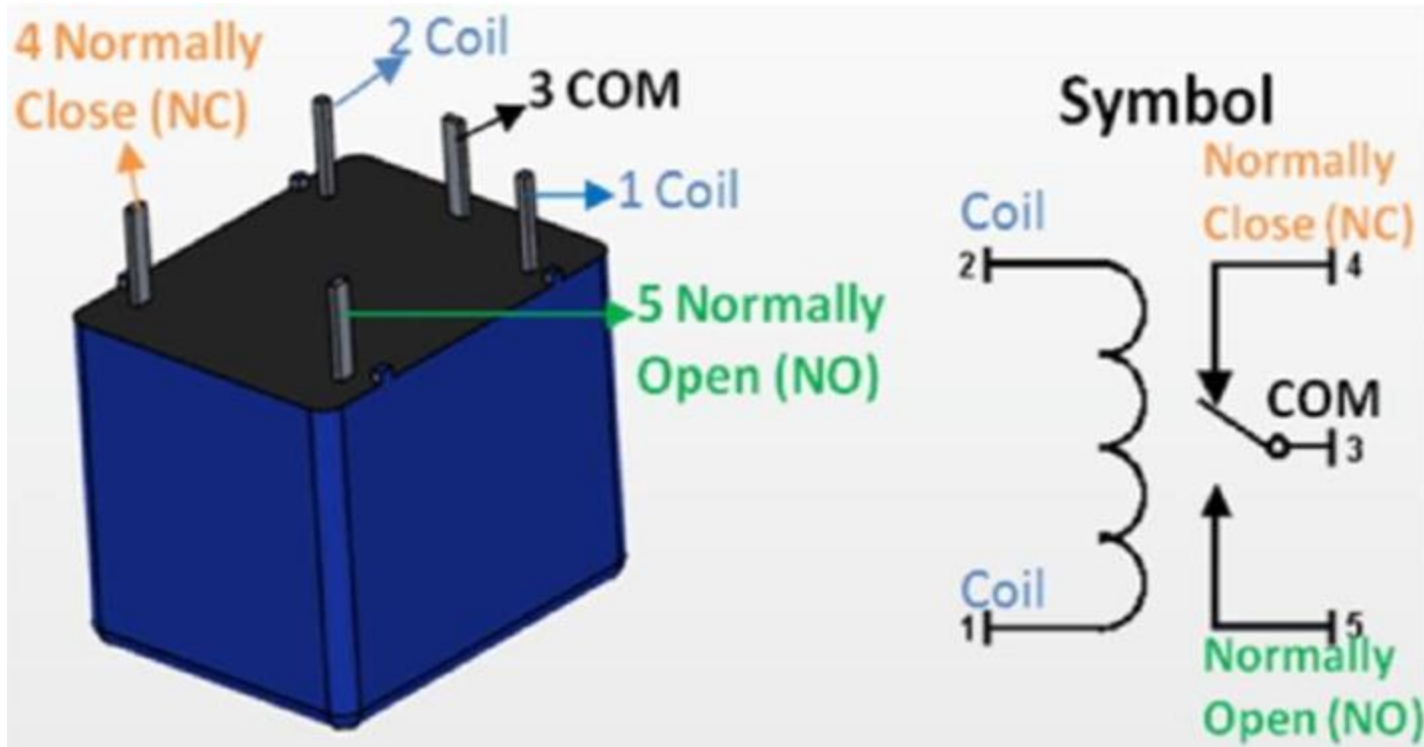
Servo myServo;

void setup() {
  myServo.attach(9);
}

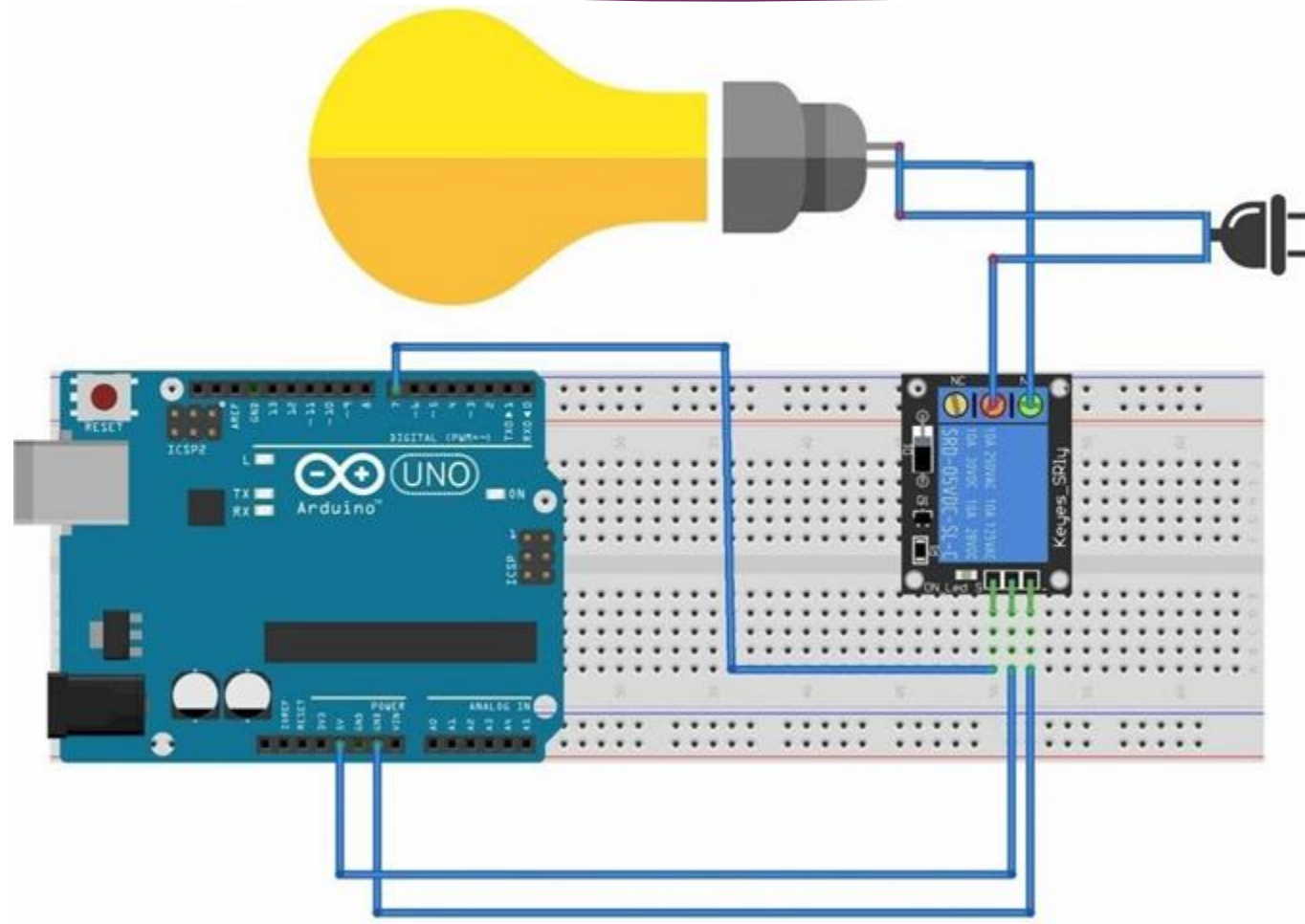
void loop() {
  for (int i = 0; i < 180; i++) {
    myServo.write(i);
    delay(10);
  }

  for (int i = 179; i > 0; i--) {
    myServo.write(i);
    delay(10);
  }
}
```

Electromagnetic Relay



Relay Interfacing Diagram



Relay Interfacing Code

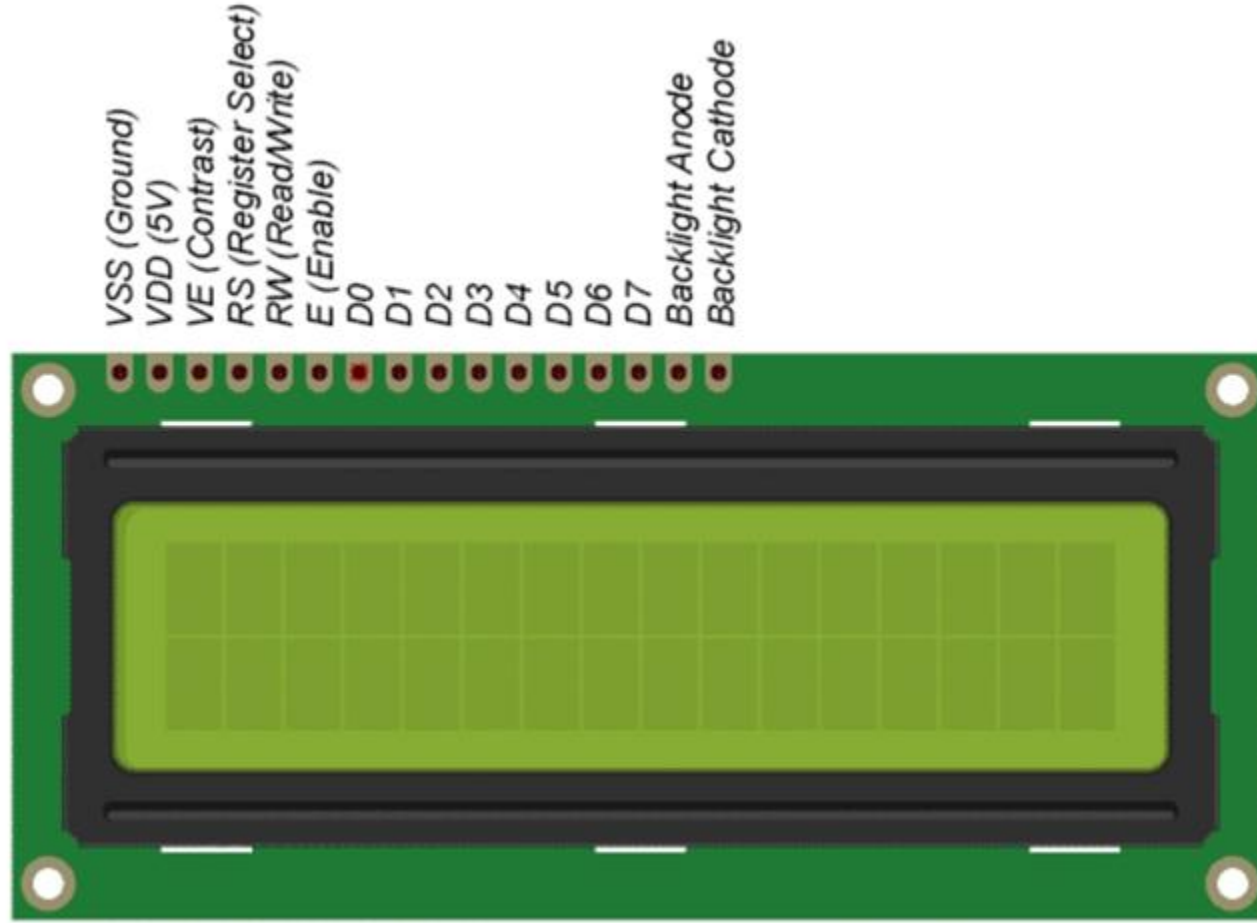
```
int RelayPin = 7;

void setup() {
    // Set RelayPin as an output pin
    pinMode(RelayPin, OUTPUT);
}

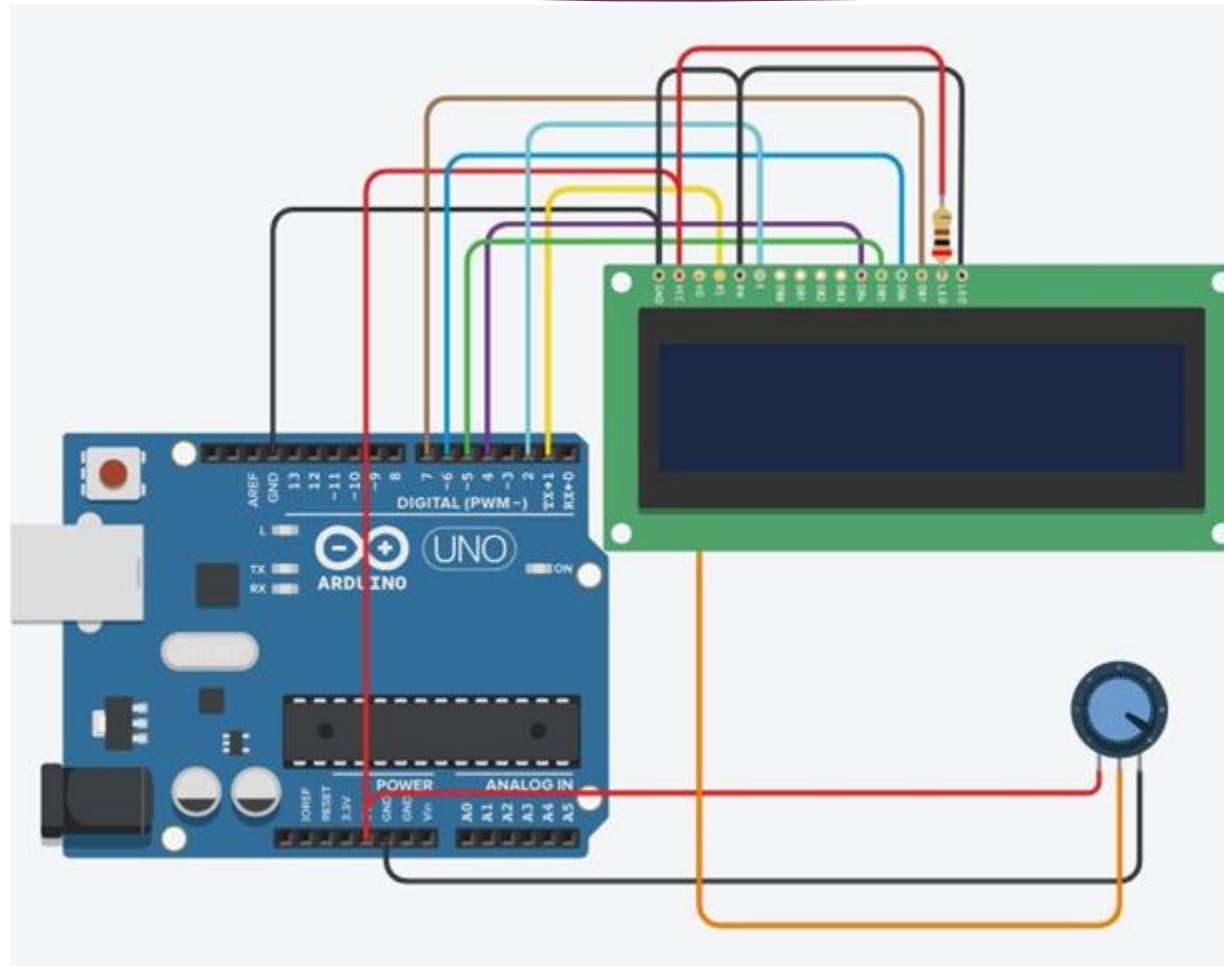
void loop() {
    // Let's turn on the relay...
    digitalWrite(RelayPin, LOW);
    delay(3000);

    // Let's turn off the relay...
    digitalWrite(RelayPin, HIGH);
    delay(3000);
}
```


16×2 LCD Display



16×2 LCD Interfacing Diagram



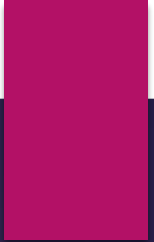
16×2 LCD Interfacing Code

```
#include <LiquidCrystal.h>

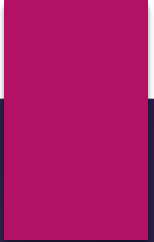
LiquidCrystal lcd(1, 2, 4, 5, 6, 7);

void setup() {
  lcd.begin(16, 2);
  lcd.clear();
  lcd.setCursor(2, 0);
  lcd.print("Hello World!");
  lcd.setCursor(0, 1);
  lcd.print("I am a 16x2 LCD!");
}

void loop() {
  // do nothing
}
```

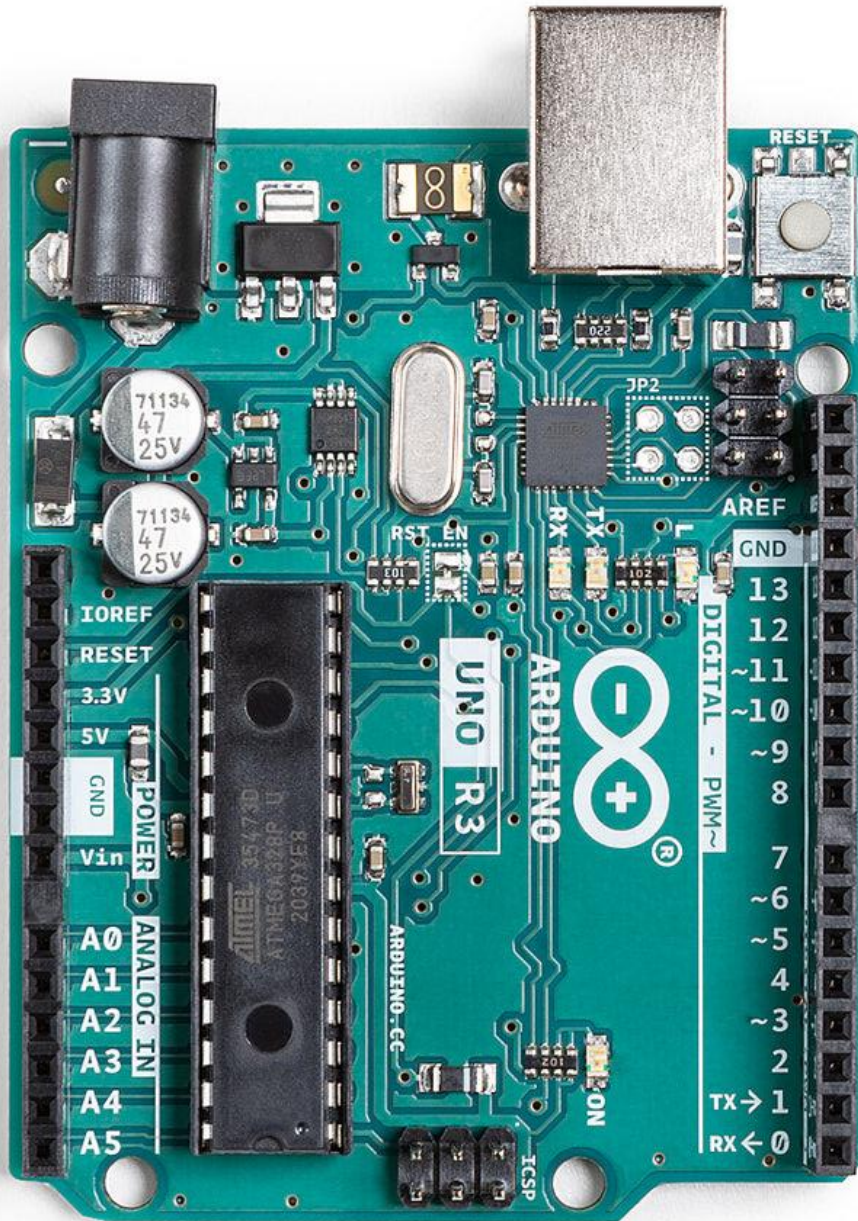


QUESTION
S?



THANK
YOU

Getting Started With Arduino and CEP Problem



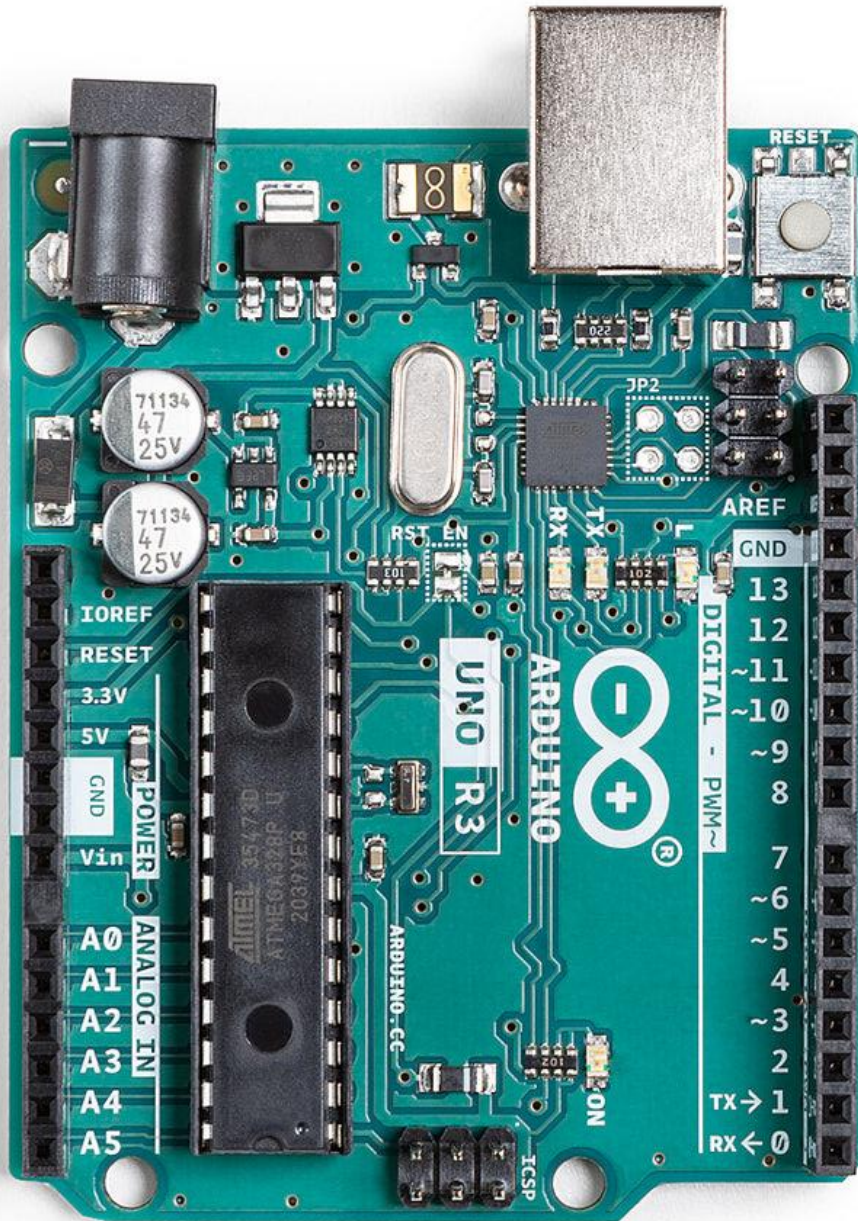
Digital Pins (0-13)

Pins 0 to 13: These are digital input/output pins that can be used as either inputs or outputs using `pinMode()`, `digitalWrite()`, and `digitalRead()` functions. Each pin operates at 5V and can provide or receive a maximum of 40 mA of current.

Pin 0 (RX) and Pin 1 (TX): Used for serial communication. RX is used for receiving data, and TX is used for transmitting data. These are connected to the USB-to-TTL serial chip, so avoid using them if you're using the USB port for programming.

PWM (Pulse Width Modulation) Pins (3, 5, 6, 9, 10, 11)

Pins 3, 5, 6, 9, 10, and 11: These digital pins can be used as PWM outputs using the `analogWrite()` function. PWM is used to simulate analog output, commonly used for dimming LEDs or controlling motor speed.



Analog Pins (A0-A5)

Pins A0 to A5: These are analog input pins, capable of reading a voltage between 0 and 5V using the `analogRead()` function. They have a 10-bit resolution, returning values from 0 to 1023.

Power Pins

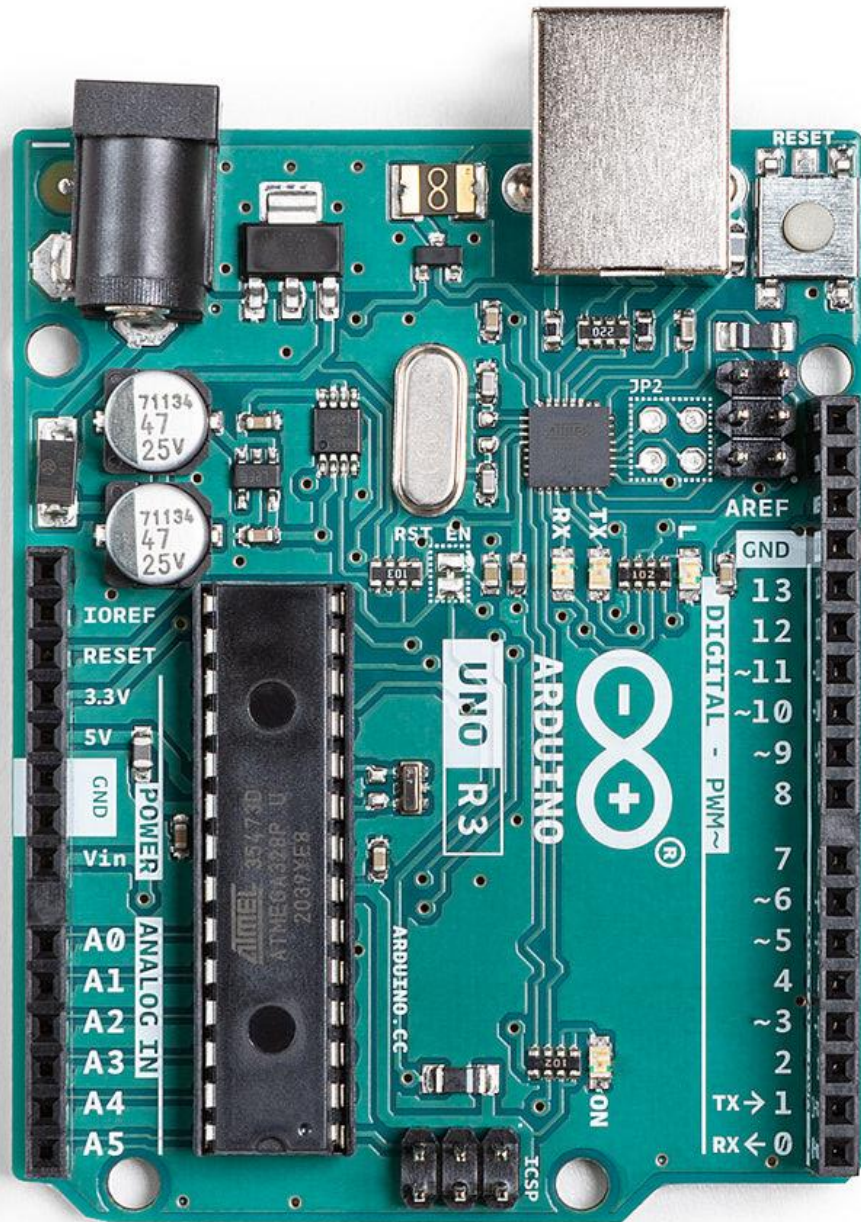
Vin: Input voltage to the Arduino board when using an external power source (6-20V). You can supply voltage through this pin or the power jack.

5V: The regulated power supply used to power the microcontroller and other components on the board. Can come from USB, Vin (through the regulator), or an external power supply.

3.3V: A 3.3V supply generated by the onboard regulator. Maximum current draw is 50 mA.

GND: Ground pins.

RESET: Resets the microcontroller. Can be used to restart the program from the beginning.



Other Pins

AREF (Analog Reference): Used to set an external reference voltage (0-5V) as the upper limit for analog inputs.

IOREF: Provides the voltage reference at which the microcontroller is operating (usually 5V on Arduino Uno). Allows the shields to adapt to the board's voltage.

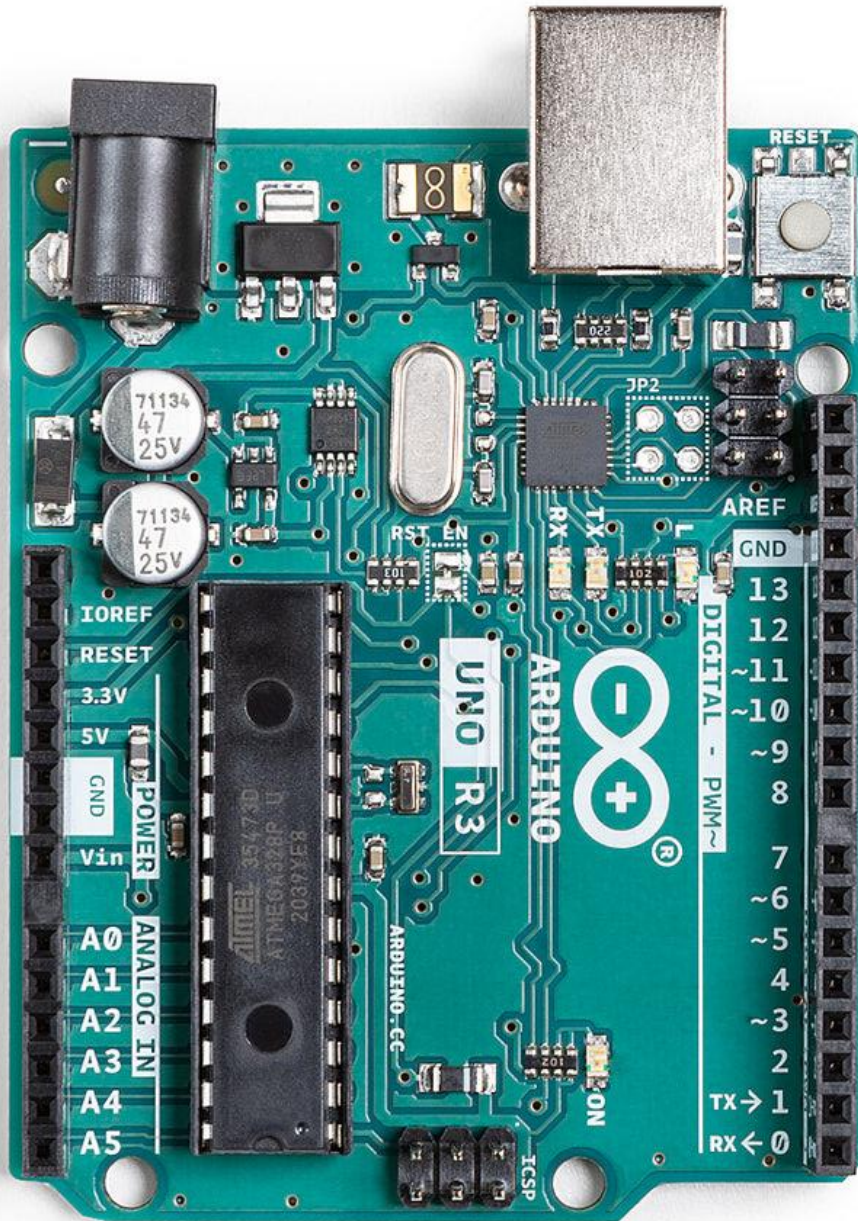
SPI Pins (10, 11, 12, 13)

Pin 10 (SS - Slave Select): Used to select the SPI slave device.

Pin 11 (MOSI - Master Out Slave In): Sends data from the Arduino to the connected SPI device.

Pin 12 (MISO - Master In Slave Out): Receives data from the connected SPI device.

Pin 13 (SCK - Serial Clock): Provides the clock signal used to synchronize data transmission.



I2C Pins (A4, A5)

SDA (A4): Serial Data Line used for I2C communication.

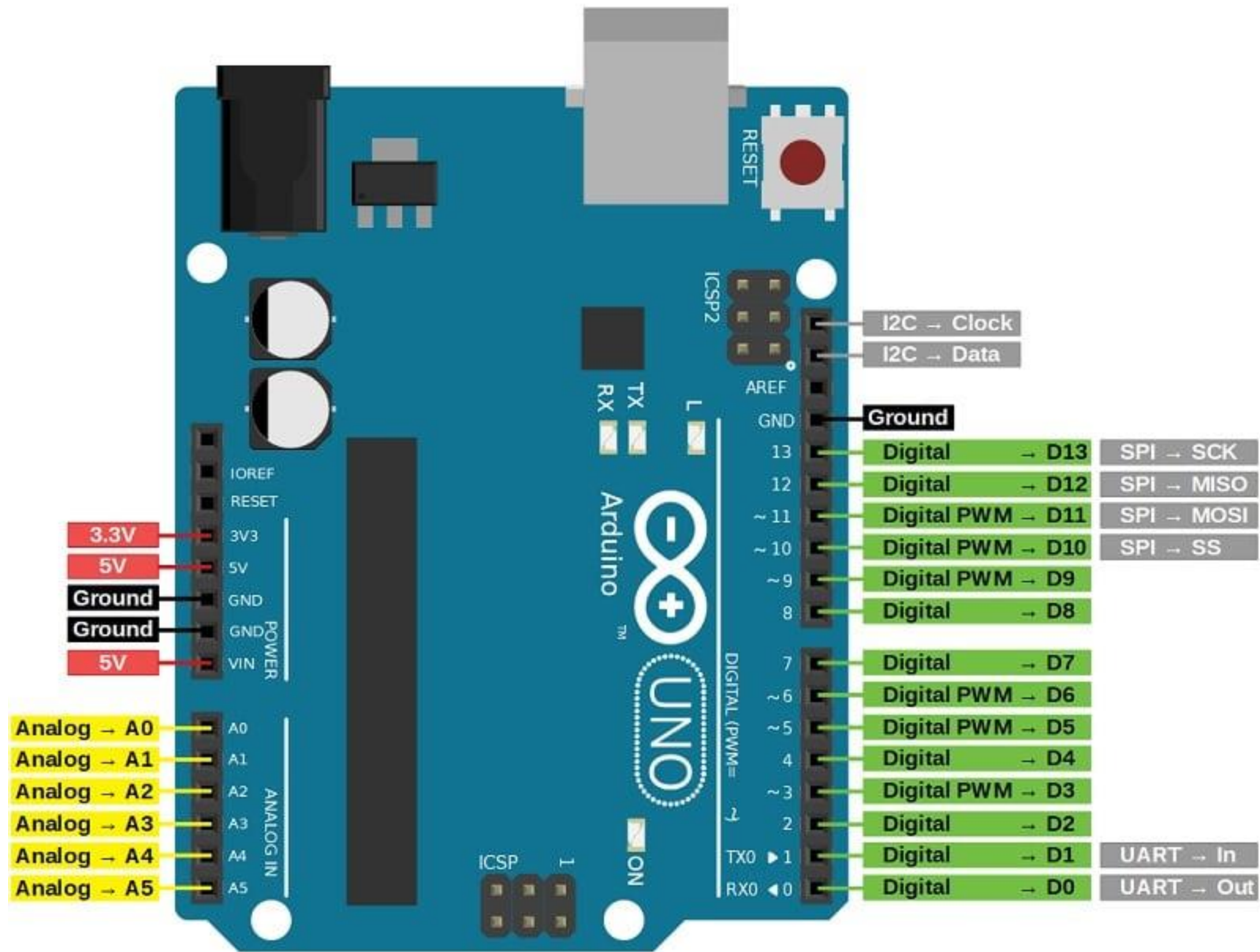
SCL (A5): Serial Clock Line used for I2C communication.

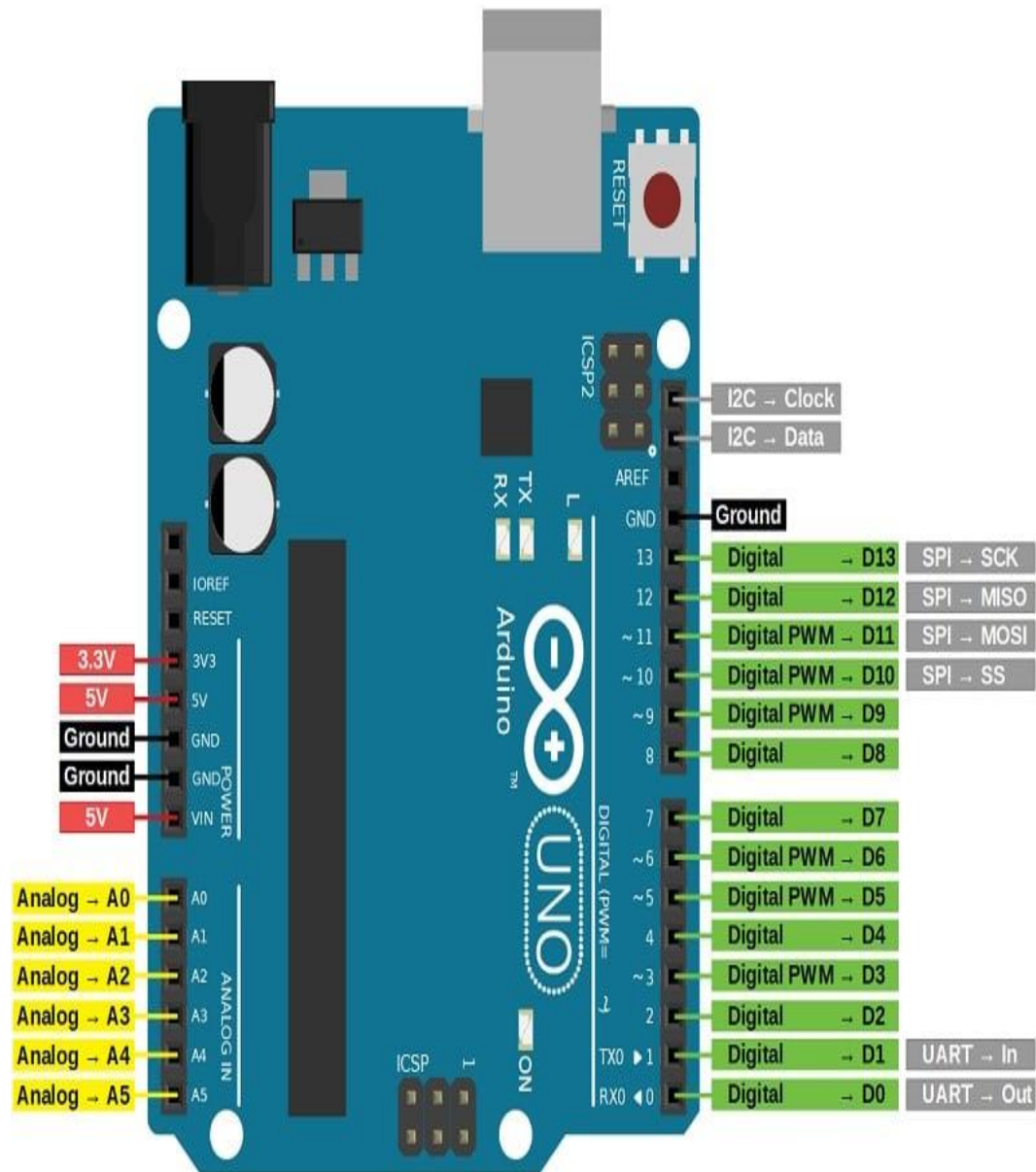
LED Indicators

Pin 13 (Built-in LED): Onboard LED connected to digital pin 13. This LED is handy for testing simple sketches without connecting external LEDs.

Communication Pins

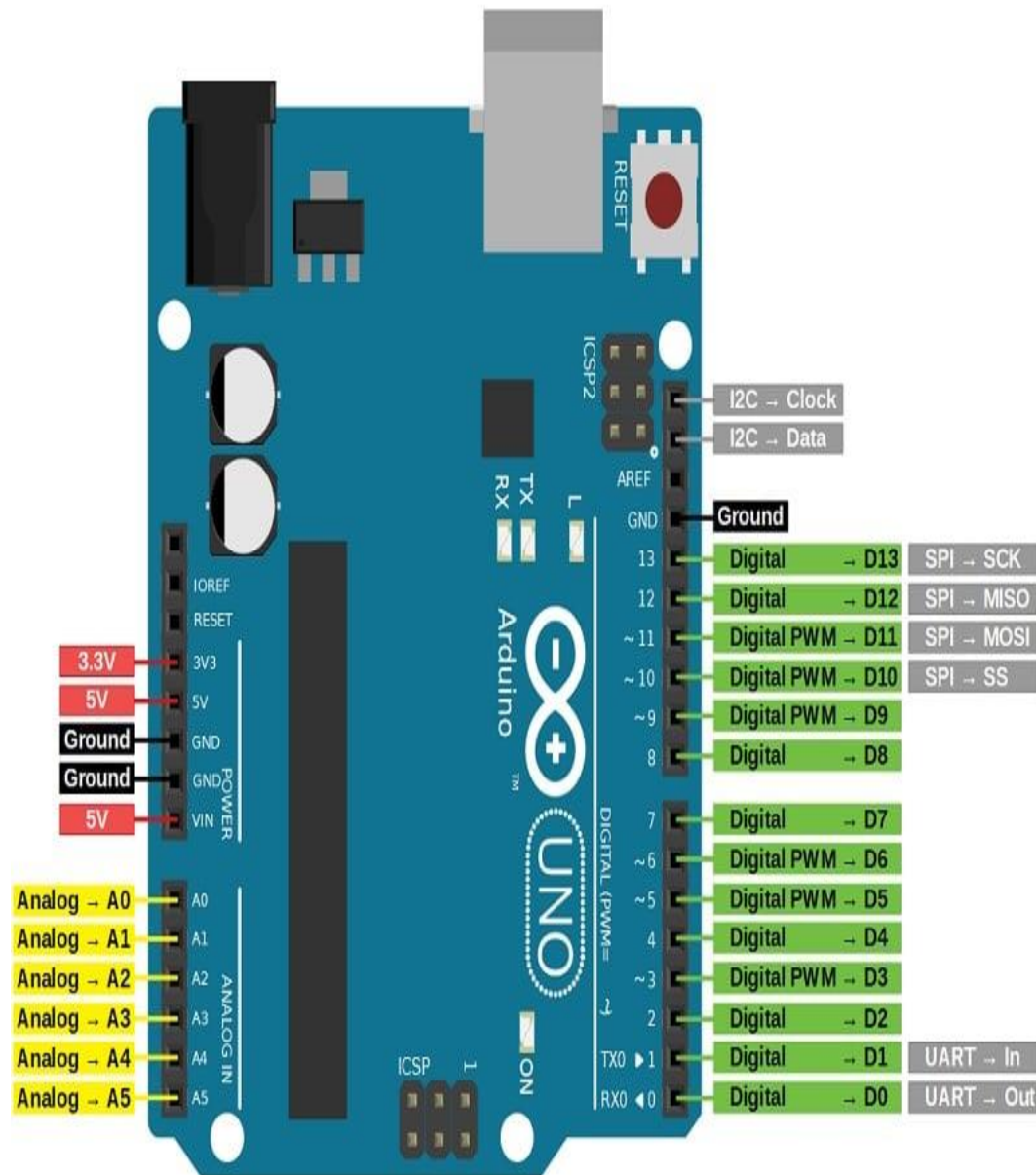
TX/RX LEDs: Indicate serial data transmission (TX) and reception (RX). These LEDs blink when data is being sent or received via the USB-to-serial chip and USB connection.





SPI (Serial Peripheral Interface) Devices

SPI (Serial Peripheral Interface) is a synchronous serial communication protocol used primarily to communicate between a master device (such as a microcontroller like the Arduino) and one or more peripheral devices (like sensors, displays, or memory chips). It's widely used because of its speed and simplicity, especially when high-speed data transfer is required.



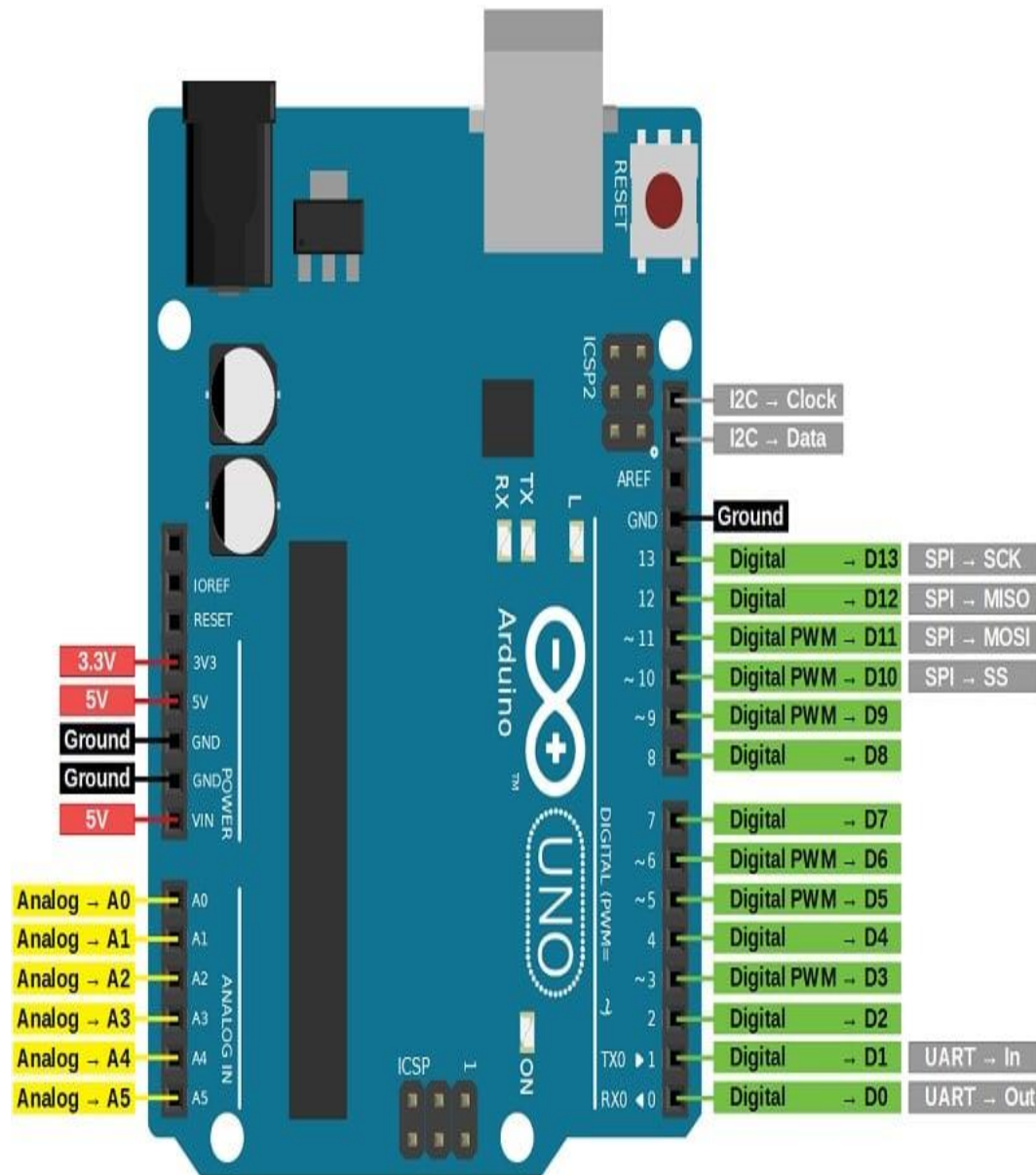
Key Components of SPI Communication:

1. Master and Slave Devices:

- **Master:** The device that controls the communication, typically a microcontroller (e.g., Arduino).
- **Slave(s):** The devices that respond to the master's commands, such as sensors, displays, SD cards, etc.

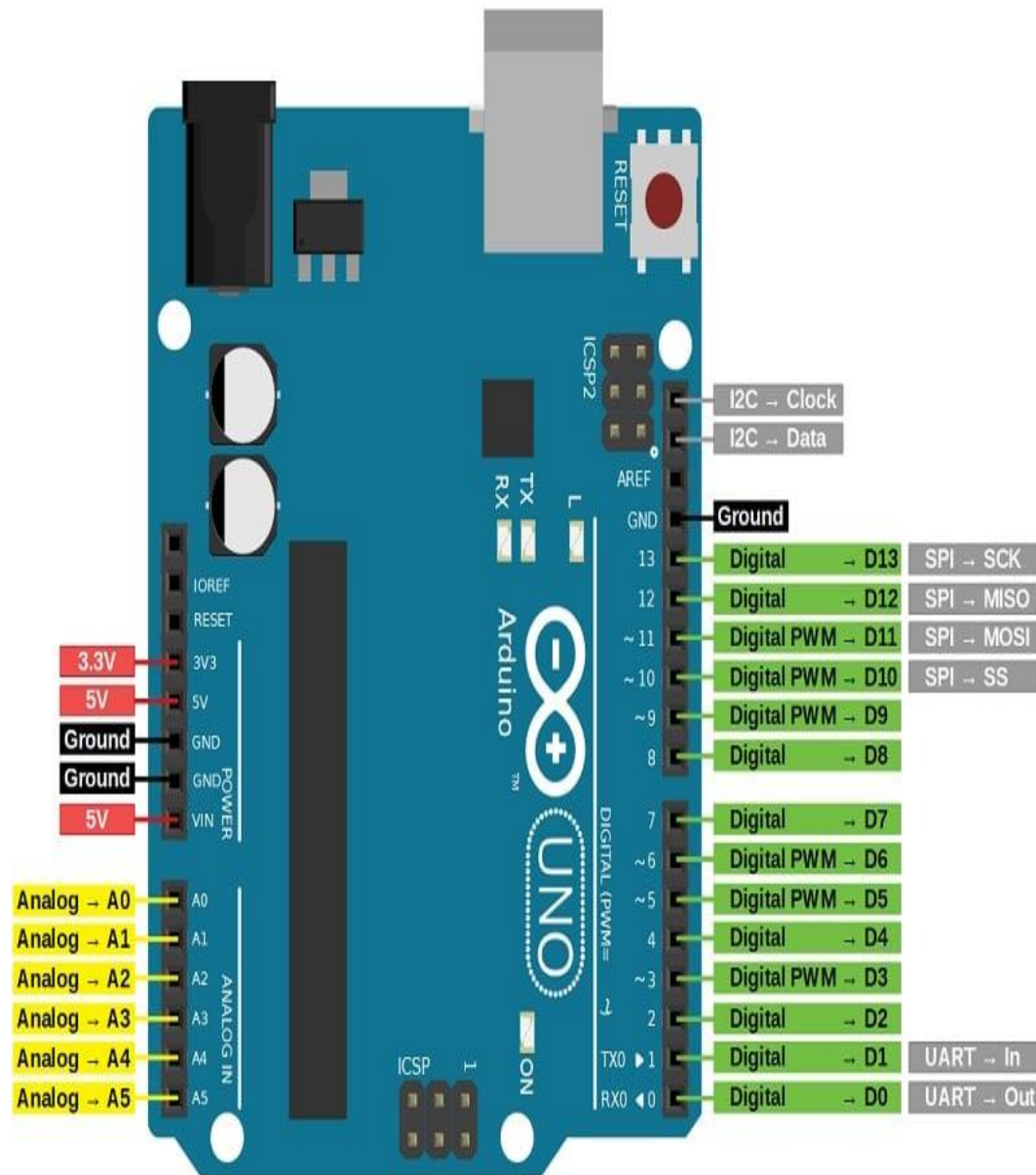
2. SPI Signals:

- **MOSI (Master Out Slave In):** The line through which data is sent from the master to the slave.
- **MISO (Master In Slave Out):** The line through which data is sent from the slave back to the master.
- **SCK (Serial Clock):** The clock signal generated by the master to synchronize data transfer.
- **SS (Slave Select):** A line used by the master to select which slave device to communicate with. When SS is low, the slave device is active; when high, it is inactive.



How SPI Works:

- 1. Data Transfer:** SPI is a full-duplex communication protocol, meaning data can be sent and received simultaneously.
- 2. Clock Control:** The master generates the clock signal on the SCK line, which dictates the timing of data transmission.
- 3. Slave Selection:** Only the slave with its SS line pulled low communicates with the master. Multiple slaves can be connected to the same SPI bus, and the master controls which one to communicate with by toggling the corresponding SS line.



Advantages of SPI:

- **High Speed:** SPI supports high data rates compared to other protocols like I2C.
- **Simplicity:** The protocol is straightforward, with minimal software overhead.
- **Full Duplex:** Data transmission in both directions simultaneously increases efficiency.

Common SPI Devices:

- **Sensors:** Gyroscopes, accelerometers, temperature sensors.
- **Memory Devices:** EEPROM, Flash memory.
- **Displays:** OLEDs, LCDs.
- **SD Cards:** Used for data storage in many embedded applications.

CEP Problem Based on Embedded System and IoT

Q1.

How would you design a smart irrigation system for an agricultural field using appropriate microcontrollers, sensors, and actuators to monitor soil moisture levels, weather conditions, and plant growth? The system should automatically regulate water flow and trigger alerts when critical thresholds are crossed, ensuring optimal water usage and plant health. Develop a detailed schematic of the system and provide a comprehensive list of required components.

CEP Problem Based on Embedded System and IoT

Solution:

Problem Overview:

The goal is to design a smart irrigation system that monitors soil moisture levels, weather conditions, and plant growth, automatically regulating water flow to ensure optimal water usage and plant health. The system will also trigger alerts when critical thresholds are crossed. Below is a detailed solution covering both hardware and software components.

CEP Problem Based on Embedded System and IoT

Solution:

1. System Requirements

The system should be capable of:

- **Monitoring soil moisture** to determine when watering is required.
- **Monitoring weather conditions** (rain, temperature, humidity) to adjust irrigation based on environmental conditions.
- **Monitoring plant growth** using relevant sensors to optimize watering based on plant maturity or health.
- **Controlling water flow** using solenoid valves to open or close irrigation lines.
- **Sending alerts** if soil moisture drops below a critical level or if weather conditions suggest extreme dryness or flooding.
- **Logging data** to cloud storage for analysis and long-term optimization of irrigation patterns.

CEP Problem Based on Embedded System and IoT

Solution:

2. Components Required

Microcontroller:

- **ESP32 or Arduino:** A microcontroller with built-in Wi-Fi for remote monitoring and control. The ESP32 is recommended for its better processing power, dual-core functionality, and built-in wireless capabilities.

Sensors:

- **Soil Moisture Sensor (Capacitive Type):** Measures soil moisture to determine when to irrigate. Example: Capacitive Soil Moisture Sensor (YL-69).
- **Temperature and Humidity Sensor (DHT22 or SHT31):** Monitors the ambient temperature and humidity to optimize irrigation based on weather conditions.

CEP Problem Based on Embedded System and IoT

Solution:

- **Rain Sensor:** Detects rainfall to prevent unnecessary watering.
- **Ultrasonic Distance Sensor** (optional): Monitors plant height/growth or could be replaced by more advanced sensors like NDVI (Normalized Difference Vegetation Index) sensors for plant health.
- **Weather Station** (optional): Integrates wind speed, rainfall, and UV sensors to optimize irrigation during extreme weather.

Actuators:

- **Solenoid Valve:** Controls the water flow in the irrigation system. It opens or closes based on signals from the microcontroller.
- **Relay Module:** Acts as a switch for high-power devices (solenoid valves or water pumps).

CEP Problem Based on Embedded System and IoT

Solution:

Power Supply:

- **12V or 24V DC Power Supply:** Powers the solenoid valves and the entire system. Solar power could be used for energy efficiency in remote fields.
- **5V Step-Down Converter:** To provide power to the microcontroller and sensors from the 12V/24V supply.

Communication:

- **Wi-Fi or LoRa Module:** Provides remote communication for data logging and alerting. ESP32 has built-in Wi-Fi, while LoRa is suitable for long-range applications.

Cloud Platform:

- **ThingSpeak/AWS IoT/MQTT Protocol:** For remote data logging and real-time alerting. The microcontroller sends sensor data to the cloud for analysis and triggers notifications via the cloud platform if required.

CEP Problem Based on Embedded System and IoT

Solution:

Power Supply:

- **12V or 24V DC Power Supply:** Powers the solenoid valves and the entire system. Solar power could be used for energy efficiency in remote fields.
- **5V Step-Down Converter:** To provide power to the microcontroller and sensors from the 12V/24V supply.

Communication:

- **Wi-Fi or LoRa Module:** Provides remote communication for data logging and alerting. ESP32 has built-in Wi-Fi, while LoRa is suitable for long-range applications.

Cloud Platform:

- **ThingSpeak/AWS IoT/MQTT Protocol:** For remote data logging and real-time alerting. The microcontroller sends sensor data to the cloud for analysis and triggers notifications via the cloud platform if required.

CEP Problem Based on Embedded System and IoT

Solution:

3. System Design

Main System Components:

1. **ESP32 Microcontroller** is the central brain that collects data from various sensors, processes it, and controls the solenoid valves via relays.
2. **Sensors:**
 - The **Soil Moisture Sensor** is placed near the plants and connected to the microcontroller's analog input to continuously monitor soil water content.
 - The **Temperature and Humidity Sensor (DHT22)** and **Rain Sensor** are placed in open-air locations to monitor environmental conditions.

CEP Problem Based on Embedded System and IoT

Solution:

1. **Water Flow Control:** The solenoid valves are connected to water pipes that distribute irrigation. The microcontroller, using a **Relay Module**, controls these valves based on sensor data.
2. **Power Supply:** A DC power supply unit is used to power the solenoid valves and sensors. The microcontroller is powered through a step-down converter.

CEP Problem Based on Embedded System and IoT

Solution:

4. Software Design

The software needs to perform the following tasks:

- **Read sensor data** for soil moisture, temperature, humidity, and rain status.
- **Process the sensor data** and decide when to water the plants.
- **Control solenoid valves** based on sensor thresholds and weather conditions.
- **Send alerts** if any parameters (like low soil moisture) exceed the predefined thresholds.
- **Log data to the cloud** for analysis and future optimization.

CEP Problem Based on Embedded System and IoT

Solution:

4.1 System Flow

1. Sensor Data Collection:

- Soil moisture is measured in real-time.
- Temperature, humidity, and rain sensors provide context for adjusting watering schedules based on environmental conditions.

2. Decision Making:

- If the soil moisture falls below a certain threshold, and the weather sensors indicate no rain, the system opens the solenoid valves to irrigate the field.
- If it's raining or soil moisture is adequate, irrigation is skipped.

CEP Problem Based on Embedded System and IoT

Solution:

3. Water Control:

- The relay module is triggered to open/close the solenoid valve for watering.
- Valves are automatically closed once soil moisture reaches a safe level.

4. Data Logging and Alerts:

- Data is logged to the cloud using an IoT platform (like ThingSpeak or AWS IoT).
- Alerts (via email, SMS, or app notification) are triggered if soil moisture or temperature reaches critical levels.

4.2 Pseudo Code

CEP Problem Based on Embedded System and IoT

Solution:

5. Conclusion

This smart irrigation system integrates sensors, actuators, and cloud connectivity to manage irrigation in an automated and efficient way. By monitoring soil moisture, weather conditions, and plant growth, the system optimizes water usage while ensuring plant health. Real-time alerts and data logging to the cloud make it robust, allowing remote monitoring and adjustments.

This system demonstrates how modern agricultural practices can be enhanced using IoT, real-time data, and automation to conserve water, improve crop yield, and reduce manual labor.

CEP Problem Based on Embedded System and IoT

Q2.

How would you design a smart energy management system for a commercial building using appropriate microcontrollers, sensors, and actuators to monitor and optimize energy consumption? The system should track occupancy, lighting, HVAC (Heating, Ventilation, and Air Conditioning) usage, and power loads from various appliances, automatically adjusting based on occupancy and time of day. It should also trigger alerts when energy usage exceeds certain limits or when equipment failures are detected. Develop a detailed schematic of the system and provide a comprehensive list of required components



Advanced Arduino Programming for Communications

Course Title: Embedded Systems & IoT
Course Code: CSE233

Dept. of CSE, DIU

Table of contents

- ▶ **HC-05 Bluetooth Module** Interfacing with Arduino
- ▶ **SIM800L GSM/GPRS Module** Interfacing with Arduino
- ▶ **LoRa RF Module** Interfacing with Arduino
- ▶ **Generic RF Module** Interfacing with Arduino
- ▶ **nRF24L01 RF Module** Interfacing with Arduino

HC-05 Bluetooth Module

A Bluetooth module is a device that allows wireless communication between electronic devices over short distances using Bluetooth technology. It is commonly used in IoT, robotics, and home automation applications to enable wireless data transfer between devices like smartphones, computers, and microcontrollers.

Key Features:

- **Wireless Communication:** Operates over short distances (typically 10m - 100m depending on the module).
- **Low Power Consumption:** Suitable for battery-operated devices.
- **Data Transfer:** Supports serial communication (UART), allowing data transfer between a microcontroller (e.g., Arduino) and other Bluetooth-enabled devices.
- **Bluetooth Versions:** Common modules use Bluetooth 2.0, 3.0, 4.0 (Low Energy), or 5.0 (latest version with increased range and speed).

HC-05 Bluetooth Module

- ✓ Supports **Master & Slave** modes
- ✓ **UART (Serial Communication)** → Default Baud Rate: **9600**
- ✓ **Range:** ~10 meters
- ✓ **Power:** 3.3V (Tolerates 5V on RX)

- Easy-to-use wireless module
- Compatible with Arduino, Raspberry Pi, Microcontrollers
- Ideal for IoT & Embedded Projects

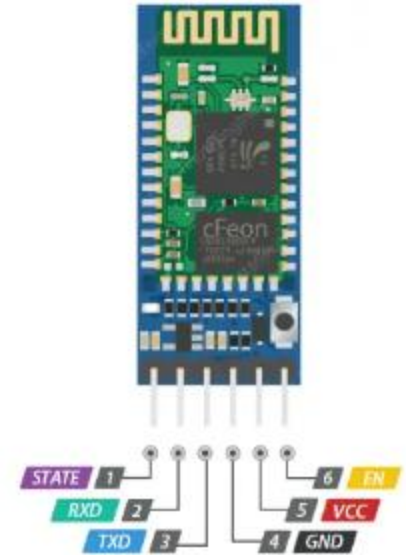


HC-05 Working Principle

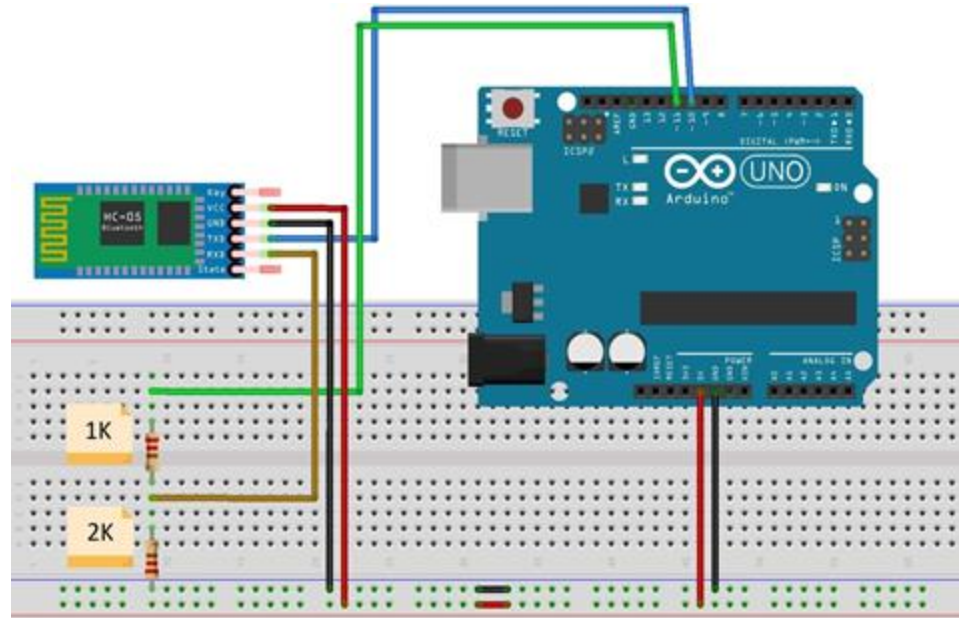
- **Pairing & Connection:** The module discovers nearby devices and establishes a connection.
- **Data Transfer:** Uses serial communication (UART) to send/receive data via TX and RX pins.
- **Bluetooth Profiles:** Utilizes protocols like SPP (Serial Port Profile) for wireless serial communication.
- **Wireless Range:** Typically works within a range of up to 10 meters.
- **Bidirectional Communication:** Enables two-way data transfer between devices.

HC-05 Bluetooth Module Pin Configuration

Pin	Function
VCC	Power (3.3V-5V)
GND	Ground
TX	Transmit Data
RX	Receive Data
EN (Key)	Enables AT mode



HC-05 Interfacing with Arduino Circuit Diagram



HC-05 Interfacing with Arduino Microcontroller Code

```
#include <SoftwareSerial.h>

// Create software serial object to communicate with HC-05
SoftwareSerial mySerial(10, 11); // HC-05 Tx -> Arduino 10, HC-05 Rx -> Arduino 11

void setup() {
  // Begin serial communication with Arduino IDE (Serial Monitor)
  Serial.begin(9600);
  // Begin serial communication with HC-05
  mySerial.begin(9600);

  Serial.println("Initializing...");
  Serial.println("The device started, now you can pair it with Bluetooth!");
}
```

```
void loop() {
  // If data is received from Serial Monitor, send it to Bluetooth
  if (Serial.available()) {
    mySerial.write(Serial.read());
  }

  // If data is received from Bluetooth, send it to Serial Monitor
  if (mySerial.available()) {
    Serial.write(mySerial.read());
  }

  delay(20); // Small delay to improve stability
}
```

HC-05 Bluetooth Module Applications

- **Industrial Automation:** Enables remote monitoring and control of machines, improving efficiency and reducing manual intervention.
- **IoT & Home Automation:** Facilitates wireless control of smart home devices, allowing automation of appliances via mobile apps.
- **Robotics & Remote Control:** Supports wireless communication for robot navigation, making remote operation and data exchange seamless.
- **Healthcare & Wearable Devices:** Used in smart healthcare systems for wireless patient monitoring and health tracking through Bluetooth-enabled devices.
- **Wireless Data Transfer:** Replaces wired UART communication, enabling secure and efficient data exchange between microcontrollers and smart devices.

SIM800L GSM Module

The SIM800L is a GSM/GPRS module that allows your Arduino to connect to cellular networks for communication, making it suitable for sending SMS, making calls, and connecting to the internet via GPRS. Here's an overview and a simple code example to get you started.

Key Features:

- GSM/GPRS Communication: Send SMS, make calls, and use data.
- Voltage: 3.4V to 4.4V (often powered with 4V).
- Communication Interface: Serial (UART).
- Functions: SMS, voice calls, data (GPRS), and more.



SIM800L Pin Configuration

SIM800L Pin	Arduino Pin
VCC	4V (Use a separate power supply if needed)
GND	GND
TXD	RX (Pin 10)
RXD	TX (Pin 11)
DTR	Not connected (optional)



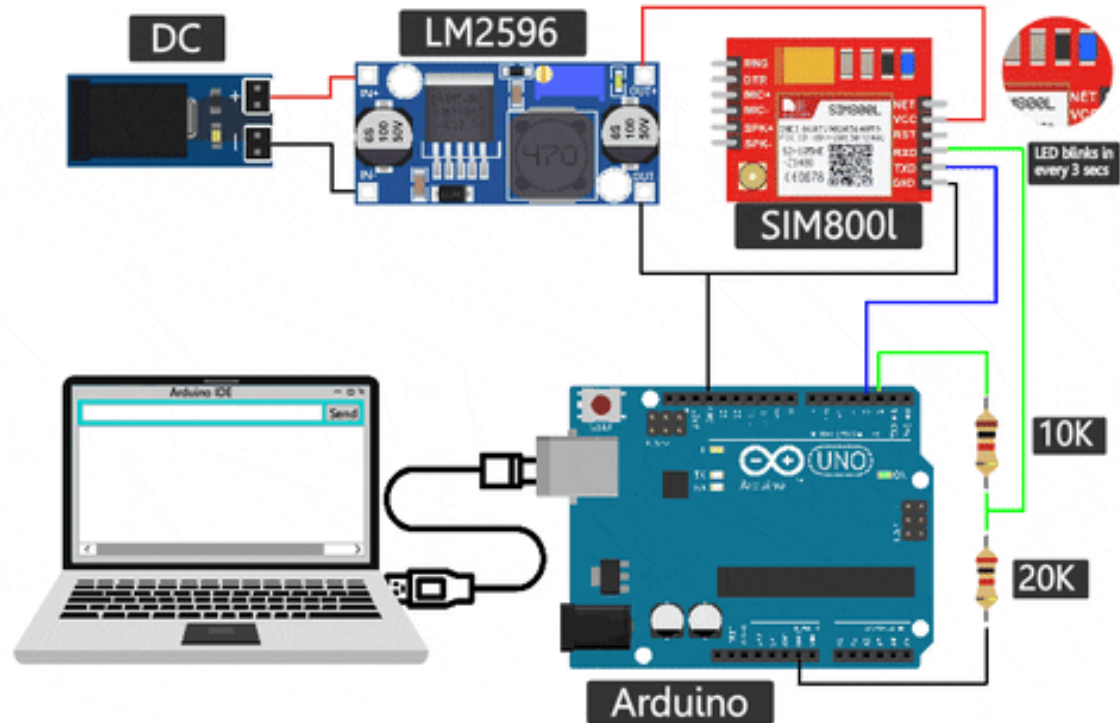
- **Power Supply:** The SIM800L requires stable voltage (3.4V to 4.4V) and a separate power source (as Arduino's 5V pin may not supply enough current).
- **Antennas:** Ensure connect the antenna for better reception.
- **GSM Signal:** Make sure SIM card is active and has a good signal.

SIM800L Working Principle

How it works:

1. **AT Commands:** Sends basic **AT** commands to the SIM800L for SMS functionality.
1. **Serial Communication:** Uses **SoftwareSerial** to communicate with the SIM800L, forwarding data between the Arduino and SIM800L, and the Arduino and the Serial Monitor.
1. **SMS:** The code sends the "Hello World!" SMS message to a phone number (+ZZxxxxxxxxxx). Make sure to replace it with an actual phone number with country code.

SIM800L with Arduino



Connection Diagram of SIM800L with Arduino

To interface the **SIM800L GSM module with Arduino**:

✓ **Power:**

- SIM800L requires **3.7V–4.2V stable supply** (Li-ion battery with LM2596 step-down voltage regulator set to 4V).
- **SIM800L GND and LM2596 GND connects to Arduino GND** to maintain a common ground.

✓ **Communication:**

- **SIM800L TX (e.g., Pin D10) → Arduino RX** (direct connection).
- **Arduino TX → SIM800L RX (e.g., Pin D11) via voltage divider** to reduce 5V to ~3.3V for safe operation.

Connection Diagram of SIM800L with Arduino



Voltage Divider for Arduino TX → SIM800L RX:

- Use **10kΩ resistor** (Arduino TX to SIM800L RX) and
- **20kΩ resistor** (SIM800L RX to GND).



Optional:

- Connect an antenna to SIM800L for better signal.
- Add a **1000μF capacitor** across SIM800L VCC-GND to prevent resets during transmission.

SIM800L with Arduino Code



```
#include <SoftwareSerial.h>

SoftwareSerial sim800l(3, 2); // RX, TX pins connected to SIM800L

void setup() {
  // Begin serial communication with Serial Monitor and SIM800L
  Serial.begin(9600);
  sim800l.begin(9600);

  // Wait for the SIM800L module to initialize
  delay(1000);

  // Send SMS
  sendSMS("1234567890", "Hello from SIM800L!");

  Serial.println("SMS sent!");
}
```

```
void loop() {
  // You can add code to listen for incoming messages or calls
}

// Function to send SMS
void sendSMS(String phoneNumber, String message)
{
  sim800l.print("AT+CMGF=1\r"); // Set SMS format to text mode
  delay(500);
  sim800l.print("AT+CMGS=\"" + phoneNumber + "\"\r"); // Set the recipient phone number
  delay(500);
  sim800l.print(message); // Send message
  delay(500);
  sim800l.write(26); // ASCII code for sending the SMS
  delay(500);
}
```

SIM800L Applications

The SIM800L module has several applications, including:

- ✓ **SMS-based Communication:** Send SMS alerts, control devices remotely, or create alert systems for emergencies.
- ✓ **Voice Communication:** Enable voice calls or intercom systems.
- ✓ **GPRS Data Communication:** Provide internet access, send data from sensors, or enable GPS tracking.
- ✓ **IoT Projects:** Used in remote IoT devices for monitoring and automation, such as in agriculture or smart home systems.
- ✓ **Home Automation & Security:** Control appliances via SMS or phone calls and send alerts for

LoRa Module

LoRa (Long Range) Module is a wireless communication technology designed for **long-distance, low-power** data transmission.

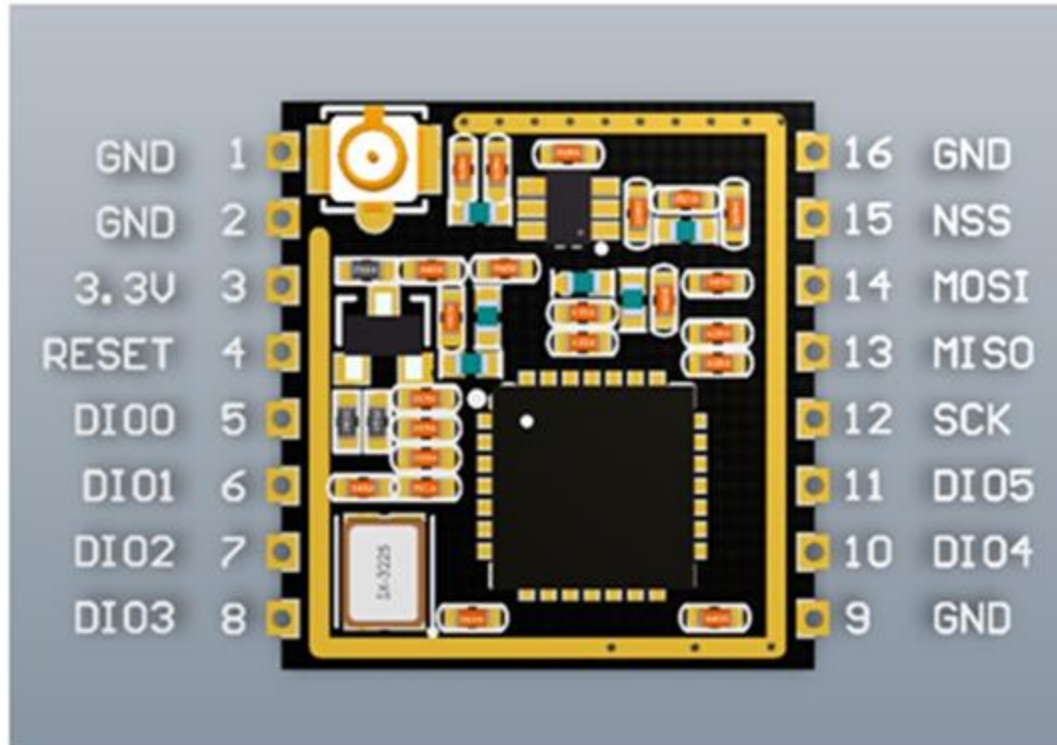
- It operates on unlicensed **ISM bands** (e.g., **433MHz, 868MHz, 915MHz**) and is widely used in IoT applications.

❑ How LoRa Works

- Uses Chirp Spread Spectrum (CSS) modulation for robust, interference-resistant communication.
- Supports Point-to-Point (direct node communication) and LoRaWAN (network-based) communication.
- Can transmit data up to 10-15 km in open areas with low power consumption.



Pin Diagram of LoRa Module



Pin Diagram of LoRa Module



Pin Diagram of LoRa Module with Arduino

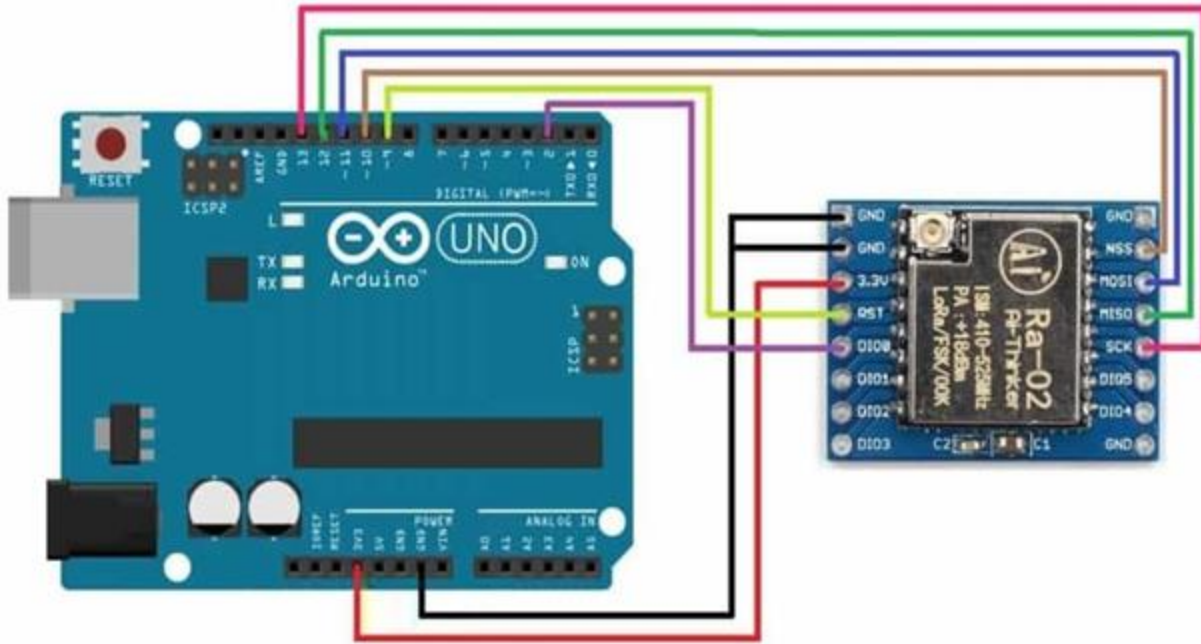
LoRa Module Pin

Arduino Pin

- | | |
|--------------|-------------------|
| • MISO | Pin 12 |
| • MOSI | Pin 11 |
| • SCK | Pin 13 |
| • NSS (CS) | Pin 10 |
| • RESET | Pin 9 |
| • DIO0 | Pin 2 |
| • GND | GND |
| • VCC (3.3V) | 3.3V (Important!) |

Note: The LoRa module operates at 3.3V, so ensure you're using the 3.3V pin for power. Also, the logic level is 3.3V, so if you are using a 5V Arduino, it's safer to use level shifters for the SPI pins (although many users get by without them).

LoRa Module with Arduino



LoRa Module with Arduino Code

Transmitter Code:

```
#include <SPI.h>
#include <LoRa.h>

void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("LoRa Transmitter");

  if (!LoRa.begin(433E6)) { // Set frequency (433MHz, 868MHz, or 915MHz)
    Serial.println("Starting LoRa failed!");
    while (1);
  }
}

void loop() {
  LoRa.beginPacket();
  LoRa.print("Hello World!");
  LoRa.endPacket();
  delay(1000); // Longer delay for stable transmission
}
```

Receiver Code:

```
#include <SPI.h>
#include <LoRa.h>

void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("LoRa Receiver");

  if (!LoRa.begin(433E6)) { // Set frequency (must match the transmitter)
    Serial.println("Starting LoRa failed!");
    while (1);
  }
}

void loop() {
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    String receivedMessage = "";
    while (LoRa.available()) {
      char inChar = (char)LoRa.read();
      receivedMessage += inChar;
    }
    Serial.println("Received: " + receivedMessage);
  }
}
```

Applications of LoRa Module

- ✓ **Smart Agriculture:** Remote monitoring of soil, weather, and irrigation.
- ✓ **Industrial Automation:** Wireless sensor networks for equipment monitoring.
- ✓ **Smart Cities:** Parking systems, waste management, and streetlights.
- ✓ **Environmental Monitoring:** Air pollution, water quality, and weather stations.

Generic RF Module

A Generic RF (Radio Frequency) Module is a wireless communication device that transmits and receives data over radio waves.

It consists of:

1. RF Transmitter (TX): Sends encoded signals.
2. RF Receiver (RX): Captures and decodes signals.

Common Frequencies: 315MHz, 433MHz, 868MHz, 915MHz.

Used for: Low-power, short-range communication in IoT, remote controls, and automation.

Generic RF Module

Signal Transmission:

- ✓ The transmitter converts digital data into an RF signal and sends it via an antenna.
- ✓ The receiver detects and decodes the signal into usable data.

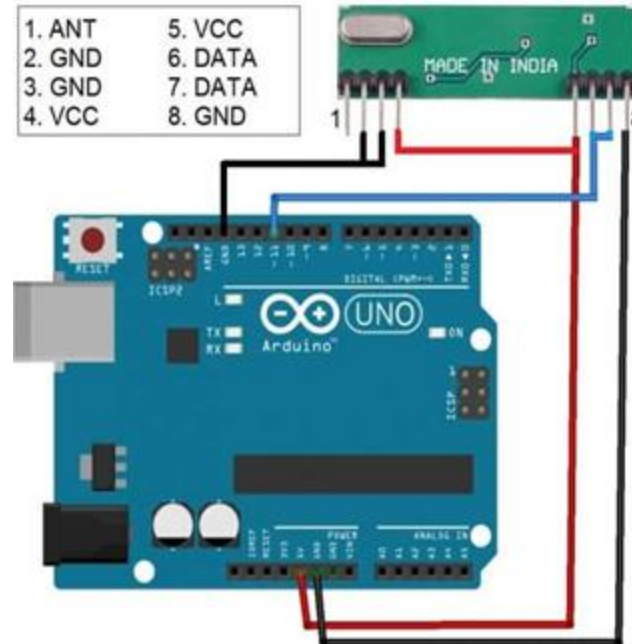
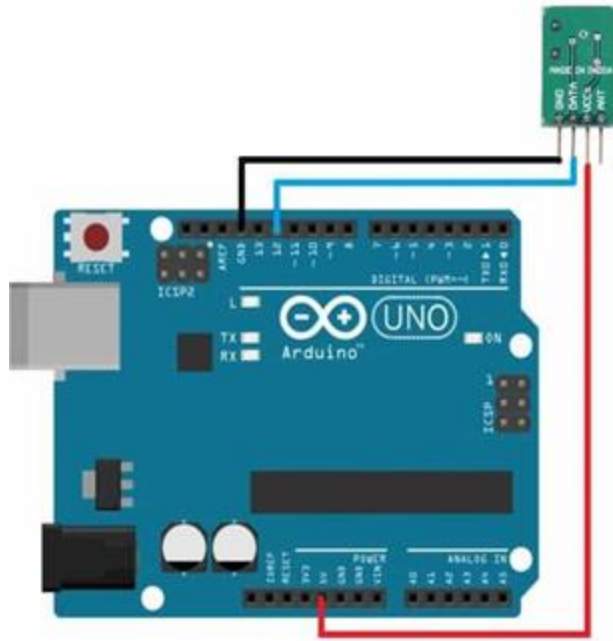
Modulation Techniques:

- ASK (Amplitude Shift Keying): Simple, low-cost, but prone to interference.
- FSK (Frequency Shift Keying): More reliable and resistant to noise.

Microcontroller Interface:

- Can be easily connected to Arduino, Raspberry Pi, or any microcontroller using simple digital I/O pins.

Generic RF Module



Generic RF Module

Transmitter Code (Arduino)

```
#include <RH_ASK.h>    // Include RadioHead ASK Library
#include <SPI.h>        // Include SPI Library (Required for RH_ASK)

RH_ASK rf_driver;      // Create ASK object

void setup() {
    rf_driver.init();   // Initialize RF transmitter
}

void loop() {
    const char *msg = "Hello World"; // Message to send
    rf_driver.send((uint8_t *)msg, strlen(msg)); // Transmit data
    rf_driver.waitPacketSent(); // Wait for transmission to complete
    delay(1000); // Send message every second
}
```

Receiver Code (Arduino)

```
#include <RH_ASK.h>    // Include RadioHead ASK Library
#include <SPI.h>        // Include SPI Library (Required for RH_ASK)

RH_ASK rf_driver;      // Create ASK object

void setup() {
    rf_driver.init();   // Initialize RF receiver
    Serial.begin(9600); // Start serial monitor
}

void loop() {
    uint8_t buf[11]; // Buffer to store received data
    uint8_t buflen = sizeof(buf); // Length of expected message

    if (rf_driver.recv(buf, &buflen)) { // Check if data received
        Serial.print("Message Received: ");
        Serial.println((char*)buf); // Print received message
    }
}
```


Applications of Generic RF Module

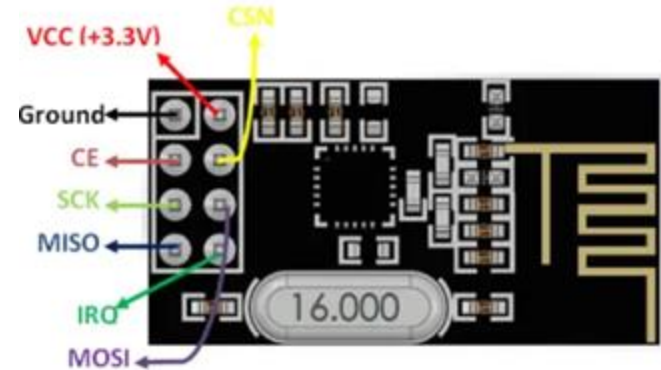
1. **Remote Control Systems** - Used in wireless remotes for TVs, gates, and car locking systems.
2. **Home Automation** - Enables smart home devices like wireless switches and alarms.
3. **Wireless Sensor Networks** - Transfers data from remote sensors for monitoring.
4. **Industrial Automation** - Provides machine-to-machine (M2M) communication.
5. **Security Systems** - Used in RFID-based access control and burglar alarms.

nRF24L01 RF Module

The nRF24L01 is a low-power, highly integrated 2.4 GHz transceiver module designed for wireless communication.

Key Features:

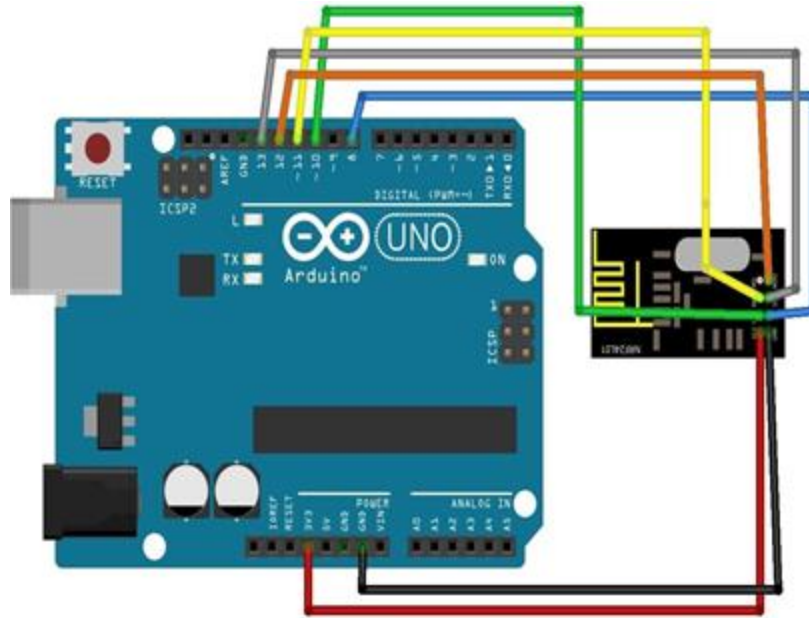
- **Frequency Range:** 2.4 GHz ISM band
- **Communication Distance:** Up to 100 meters (line of sight)
- **Data Rate:** 250 kbps, 1 Mbps, 2 Mbps
- **Low Power Consumption:** 12.3 μ A in power-down mode
- Supports Point-to-Point & Multi-Point Communication



nRF24L01 RF Module Advantages

- ❑ **Low Power:** Perfect for battery-operated devices.
- ❑ **High Communication Range:** Ideal for IoT applications.
- ❑ **Easy to Integrate:** Supports SPI for simple communication with microcontrollers like Arduino and Raspberry Pi.
- ❑ **Reliable:** Uses Frequency Hopping Spread Spectrum (FHSS) to reduce interference.

nRF24L01 RF Module



nRF24L01 RF Module

Transmitter Code:

```
// Include Libraries
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

// Create an RF24 object
RF24 radio(9, 8); // CE, CSN

// Address through which two modules communicate
const byte address[6] = "00001";

void setup() {
  radio.begin();

  // Set the address
  radio.openWritingPipe(address);

  // Set module as transmitter
  radio.stopListening();
}

void loop() {
  // Send message to receiver
  const char text[] = "Hello World";
  radio.write(&text, sizeof(text));
  delay(1000);
}
```

Receiver Code:

```
// Include Libraries
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

// Create an RF24 object
RF24 radio(9, 8); // CE, CSN

// Address through which two modules communicate
const byte address[6] = "00001";

void setup() {
  while (!Serial);
  Serial.begin(9600);
  radio.begin();

  // Set the address
  radio.openReadingPipe(0, address);

  // Set module as receiver
  radio.startListening();
}

void loop() {
  // Read the data if available in buffer
  if (radio.available()) {
    char text[32] = {0};
    radio.read(&text, sizeof(text));
    Serial.println(text);
  }
}
```

nRF24L01 RF Module Applications

- **Wireless Sensor Networks** - Remote data collection from environmental or industrial sensors.
- **Home Automation** - Wireless control of smart devices like lights, fans, and door locks.
- **IoT Applications** - Seamless communication between IoT devices for real-time monitoring and control.
- **Robotics and Automation** - Wireless control and coordination of robots and automated systems.
- **Remote Control Systems** - Stable, long-range control for RC cars, drones, and robotic projects.



THANK YOU



Introduction to ESP32

Course Title: Embedded Systems & IoT
Course Code: CSE233

Dept. of CSE, DIU

Table of contents

- ▶ Introduction to ESP32
- ▶ Why ESP32
- ▶ ESP32 Specifications
- ▶ ESP32 vs ESP8266
- ▶ Pin Diagram
- ▶ ESP Peripherals
- ▶ Esp Based projects Ideas

Introduction

- ESP32 is a series of low-cost, low-power system on chip microcontrollers with integrated **Wi-Fi** and dual-mode **Bluetooth**.
- ESP32 is created and developed by **Espressif** Systems.
- It is a successor to the **ESP8266** microcontroller.



Why ESP32

- Low Cost
- Low Power
- Wi-Fi Connectivity
- Bluetooth
- Dual-Core
- Compatible with Arduino Programming



ESP32 Specifications

- ❑ Wireless connectivity ([WiFi](#)): 150.0 Mbps data rate with HT40

Bluetooth: [BLE \(Bluetooth Low Energy\)](#) and [Bluetooth Classic](#).

Processor: Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor, running at 160 or 240 MHz.

- ❑ ROM: 448 KB (for booting and core functions)
- ❑ SRAM: 520 KB (for data and instructions)
- ❑ Low Power: ensures that you can still use ADC conversions, for example, during [deep sleep](#).

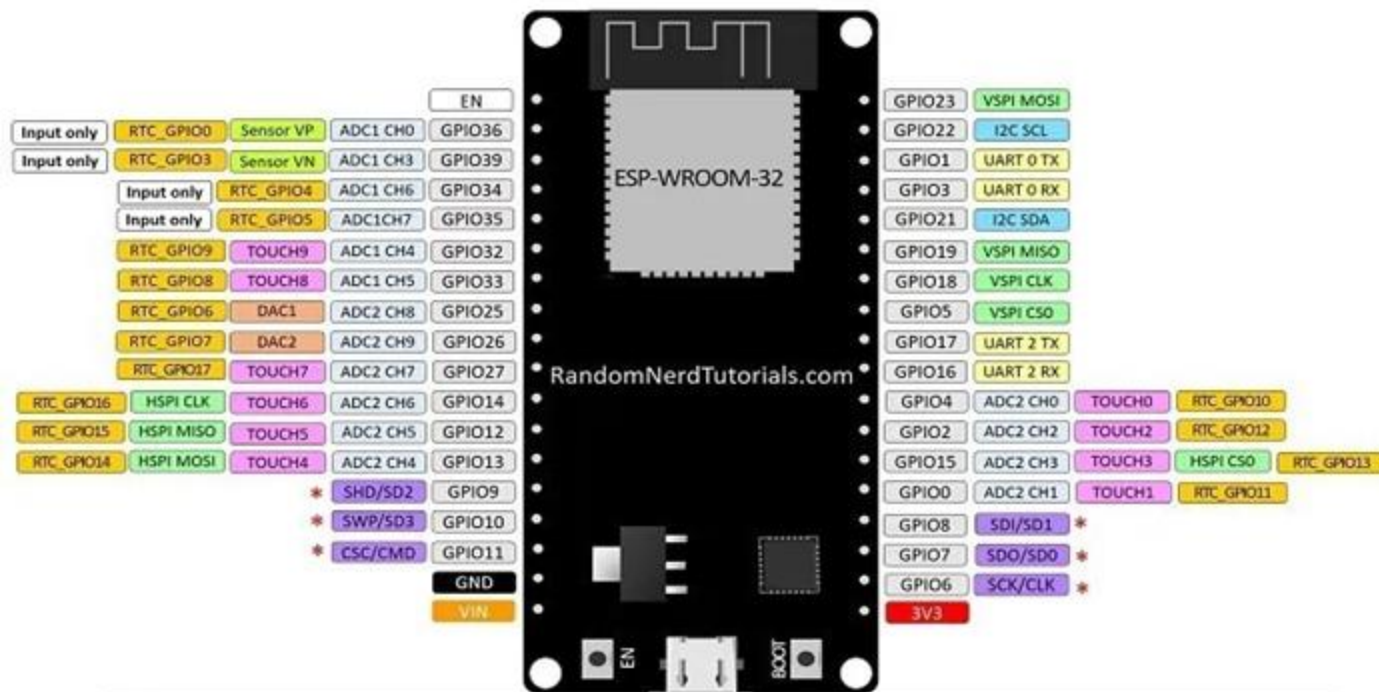


ESP32 vs ESP8266

- ❑ The ESP32 adds an **extra CPU core**, faster **Wi-Fi**, **more GPIOs**, and supports **Bluetooth 4.2** and **Bluetooth low energy**. Additionally, the ESP32 comes with **touch-sensitive pins** that can be used to **wake up the ESP32 from deep sleep**, and **built-in hall effect sensor** (A Hall effect sensor is a transducer that detects and measures magnetic fields, commonly used for sensing speed, position, and direction of moving parts.).
- ✓ The **ESP32 is faster** than the ESP8266.
- ✓ The ESP32 comes with **more GPIOs** with multiple functions.
- ✓ The **ESP32 supports analog measurements on 18 channels** (analog-enabled pins) versus just one 10-bit ADC pin on the ESP8266.
- ✓ The **ESP32 supports Bluetooth** while the ESP8266 doesn't.
- ✓ The **ESP32 is dual-core** (most models), and the ESP8266 is single core.
- ✓ The ESP32 is a bit more expensive than the ESP8266.

Pin Diagram

ESP32 DEVKIT V1 – DOIT version with 36 GPIOs



* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and CSC/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

ESP Peripherals

- **18** - Analog-to-Digital Converter (ADC) channels
- **3** - SPI interfaces (SPI, or Serial Peripheral Interface, is a synchronous serial communication interface commonly used in embedded systems to transfer data between a microcontroller and peripheral devices.)
- **3** - UART interfaces A UART (Universal Asynchronous Receiver/Transmitter) is a hardware device or circuit used for asynchronous serial communication between devices.

ESP Peripherals

- **2** - I2C interfaces I2C (Inter-Integrated Circuit) is a general-purpose, two-wire serial communication protocol primarily used for connecting various low-speed peripherals like sensors, EEPROMs, and real-time clocks.)
- **16** - PWM output channels
- **2** - Digital-to-Analog Converters (DAC)
- **2** - I2S interfaces I2S (Inter-IC Sound) is specifically designed for transmitting digital audio data between devices, such as between a microcontroller and an audio codec.)
- **10** - Capacitive sensing GPIOs

Generalized Description of ESP Board Pinout

While specific pinouts vary between different ESP boards, they generally share some common characteristics and functionalities. Here's a generalized overview:

Number of Pins:

- ESP8266 boards typically have **32 pins** in a QFN package.
- ESP32 boards usually have more pins, ranging from **30 to 40** depending on the variant.

Power Supply Pins:

- Most ESP boards have separate power supply pins for digital (VDD) and analog (VDD_ANA) functions.
- Typical voltage levels are **3.3V for VDD** and **2.5V for VDD_ANA**.
- Ground pins (GND) are essential for circuit stability and are often located around the board.

General Purpose Input/Output (GPIO) Pins:

- These are the most versatile pins, allowing for digital input, output, SPI, I2C, and PWM (Pulse Width Modulation) functionalities.
- The number of GPIO pins varies between boards, typically ranging from 8 to 32.
- Each GPIO pin has a unique number and label for identification.
- Some ESP boards have specific pins that need to be set to certain logic levels (high or low) during boot for proper operation.
- These pins are crucial for programming and uploading firmware.

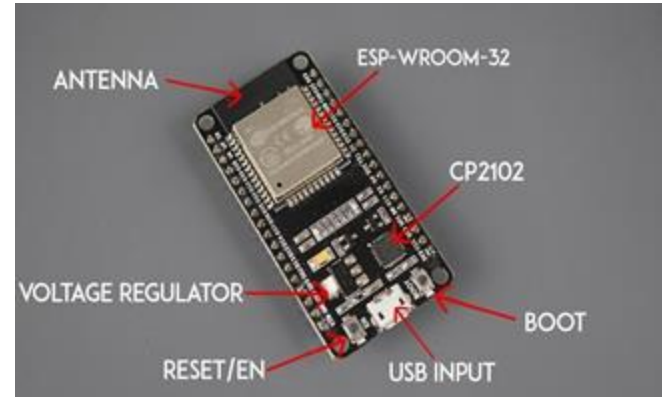
Other Functional Pins: Some boards may have additional pins for specific functions like:

- Reset pins
- LED control pins
- ADC (Analog-to-Digital Converter) channels
- SPI flash memory interface

Pinout Diagrams:

It's crucial to consult the specific pinout diagram for ESP board, as functionalities and pin assignments can differ.

- ESP8266 NodeMCU:
- ESP32 DevKitC:



Remember, this is a generalized overview, and the specific pinout for your board might differ. Always refer to the official documentation or resources provided by the manufacturer for accurate and up-to-date information.

OS vs Firmware vs IDE (From Operating System)



OS:

An OS is a software program that manages computer hardware and software resources, providing a user interface and enabling applications to run.

Examples:

Popular operating systems include Windows, macOS, Linux, Android, and iOS.

FreeRTOS on ESP32 and STM32

Embedded Linux on Raspberry Pi

OS vs Firmware vs IDE



Firmware:

A Firmware is a low level software permanently programmed into **ROM/Flash** that directly controls hardware. In ES it runs directly on microcontrollers without an OS or under a minimal RTOS.

Examples:

Firmware is found in devices like printers, routers, cameras etc. It's what allows your printer to receive and process print jobs, or your router to manage network connections. 14

OS vs Firmware vs IDE



IDE:

An IDE is a software tool used for writing, compiling, testing, debugging, and uploading code to embedded systems.

Examples:

Arduino IDE

STM32CubeIDE

Dual-Core in ESP32 under Arduino IDE



ESP32 has 2 cores:

- Core 0 (PRO_CPU): WiFi, Bluetooth, system tasks.
- Core 1 (APP_CPU): Runs Arduino 'setup()' and 'loop()' by default.

Using Arduino IDE:

- No extra setup needed for dual-core.
- Arduino code runs on Core 1 automatically.
- Core 0 handles WiFi/Bluetooth/system tasks in the background.

Dual-Core in ESP32 under Arduino IDE



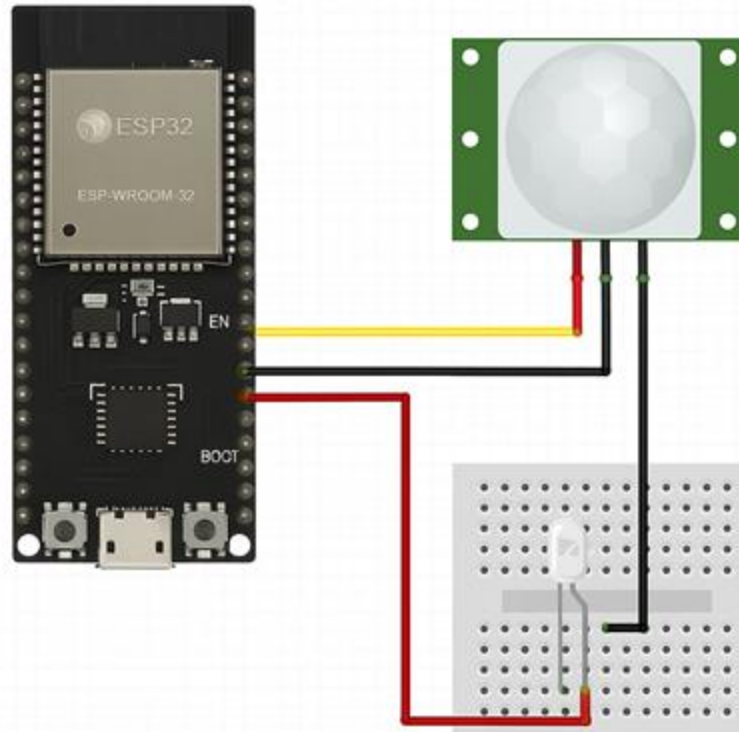
Manual dual-core usage needs FreeRTOS tasks:

- Use 'xTaskCreatePinnedToCore(...)' to assign tasks to Core 0 or 1.
- Not required for basic lab projects like PIR + LED+Buzzer etc

Key Points:

- ESP32 dual-core support works automatically under Arduino IDE.
- Default Arduino structure is sufficient for most labs.
- Advanced projects can explore FreeRTOS task handling.

Connection diagram of an dual core ESP32 projects with PIR sensor, LEDs



Code of a dual core ESP32 project with PIR sensor, LEDs



```
int led = 13;    // LED connected to pin 13
int PIR = 2;    // PIR sensor output connected to pin 2

void setup() {
  pinMode(PIR, INPUT);  // PIR sensor as input
  pinMode(led, OUTPUT);  // LED as output

  Serial.begin(9600);
  Serial.println("Calibrating Sensor...");

  for (int i = 0; i < 30; i++) {  // Wait for PIR to calibrate (~30 sec)
    Serial.print(".");
    delay(1000);
  }
```

Code of a dual core ESP32 project with PIR sensor, LEDs

```
Serial.println();  
Serial.println("Calibration done");  
Serial.println("Sensor Activated");  
}  
  
void loop() {  
  int r = digitalRead(sensor);  // Read PIR sensor  
  
  if (r == HIGH) {              // If motion detected  
    digitalWrite(led, HIGH);    // Turn on LED  
    delay(5000);                // Keep LED on for 5 seconds  
  } else  
  
  {  
    digitalWrite(led, LOW);     // Turn off LED  
  }  
}
```

ESP Based Projects

❑ Smart Home & Automation Projects

1.Voice-Controlled Home Automation - Control lights, fans, and other appliances using Google Assistant or Alexa with ESP32 and MQTT.

1.Smart Door Lock System - Use RFID, fingerprint sensor, or facial recognition to secure your door.

1.IoT-Based Gas Leakage Detector - Detect gas leaks and send alerts via an app using an MQ-2 sensor.

1Automatic Curtain Control - Use a servo motor to open/close curtains based on sunlight intensity or voice commands.

1.Water Quality Monitoring System - Measure pH, TDS, and temperature of water in real-time and display data on a mobile app.

ESP Based Projects

❑ Agriculture & Environmental Monitoring

6.Smart Irrigation System - Automate watering based on soil moisture levels using soil moisture sensors and ESP32.

6.Greenhouse Monitoring System - Monitor temperature, humidity, and CO2 levels inside a greenhouse and control ventilation automatically.

6.Weather Station - Collect real-time weather data (temperature, humidity, pressure, rainfall) and display it on a web dashboard.

6.IoT-Based Pest Detection - Use a camera module and AI to detect pests in fields and alert farmers.

6.Smart Aquaponics System - Monitor water pH, temperature, and oxygen levels for fish farming and hydroponics.

ESP Based Projects

❑ Health & Wearable Technology

11.ESP32 Smart Health Monitoring System - Track heart rate, blood oxygen, and temperature using sensors and send data to a mobile app.

11.Fall Detection for Elderly People - Use an accelerometer and ESP32 to detect falls and send emergency alerts.

11.AI-Based Gesture Recognition - Detect hand gestures using an accelerometer and use them for controlling smart devices.

ESP Based Projects

❑ Industrial & Security Applications

14.IoT-Based Fire Detection System - Use temperature and flame sensors to detect fires and send alerts via Wi-Fi.

14.Smart Factory Monitoring System - Monitor machine temperature, vibrations, and power consumption in industries.

14.IoT-Based Asset Tracking System - Track valuable assets in real-time using GPS and ESP32.

14.RFID-Based Attendance System - Track employee/student attendance using RFID and store data in Firebase.

14.IoT-Based Air Pollution Monitoring - Measure air quality (CO₂, PM_{2.5}, NO₂) and display data on a web dashboard.

ESP Based Projects

❑ Transportation & Mobility

19.Smart Parking System - Detect free parking slots using ultrasonic sensors and display availability in an app.

19.Vehicle Tracking System - Track a vehicle's real-time location and speed using GPS and ESP32.

19Automatic Toll Collection System - Use RFID tags for contactless toll payment and reduce congestion.

ESP Based Projects

❑ Fun & Experimental Projects

22.ESP32-Based AI Camera - Use an ESP32-CAM module to recognize faces and objects using AI.

22.ESP32-Based Gaming Console - Build a retro gaming console with an OLED display and push buttons.

22.IoT-Based Smart Mirror - Display weather, calendar, and notifications on a smart mirror using ESP32.

22.ESP32-Powered Robot Car - Control a robot car using Bluetooth or Wi-Fi via a mobile app.



THANK YOU

Department of Computer Science and Engineering

Daffodil International University

IoT Data Communication and Security

Course Title: Embedded Systems and IoT

Course Code: CSE233

Dept. of CSE, DIU

Department of Computer Science and Engineering

Daffodil International University

Introduction to IoT Data Communication

☐ What is IoT Communication?

- ☐ The process of transmitting data between IoT devices, sensors, and cloud platforms.
- ☐ Enables seamless interaction and automation in smart environments.
- ☐ Uses various communication protocols like MQTT, CoAP, HTTP, LoRaWAN, and Bluetooth.

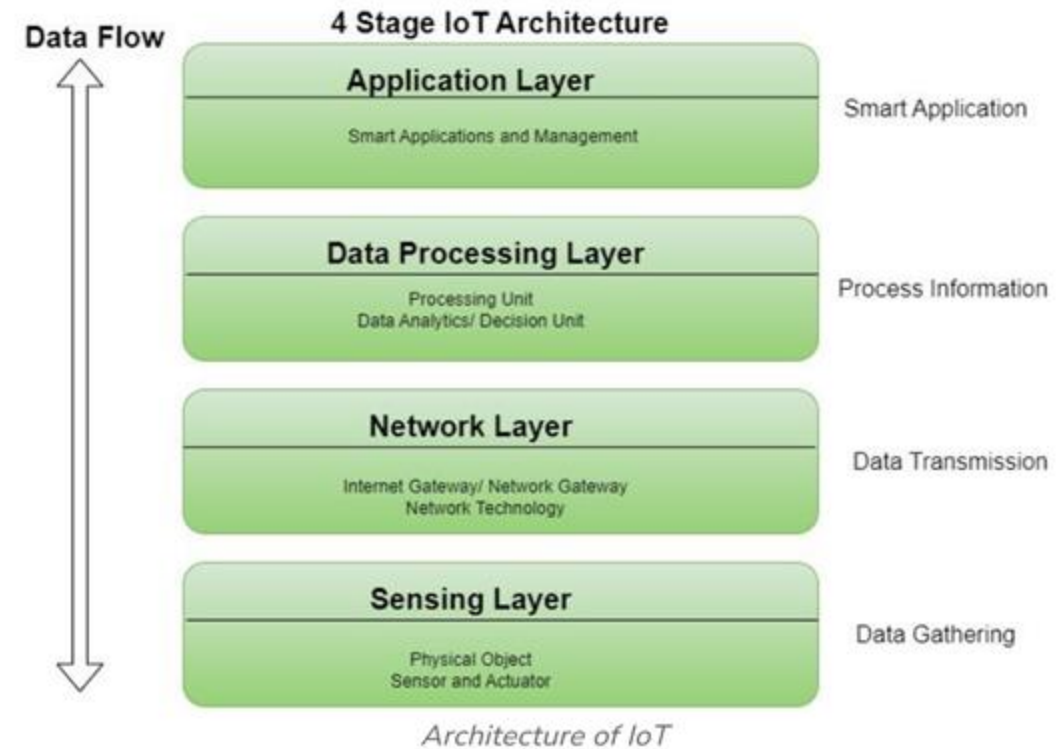
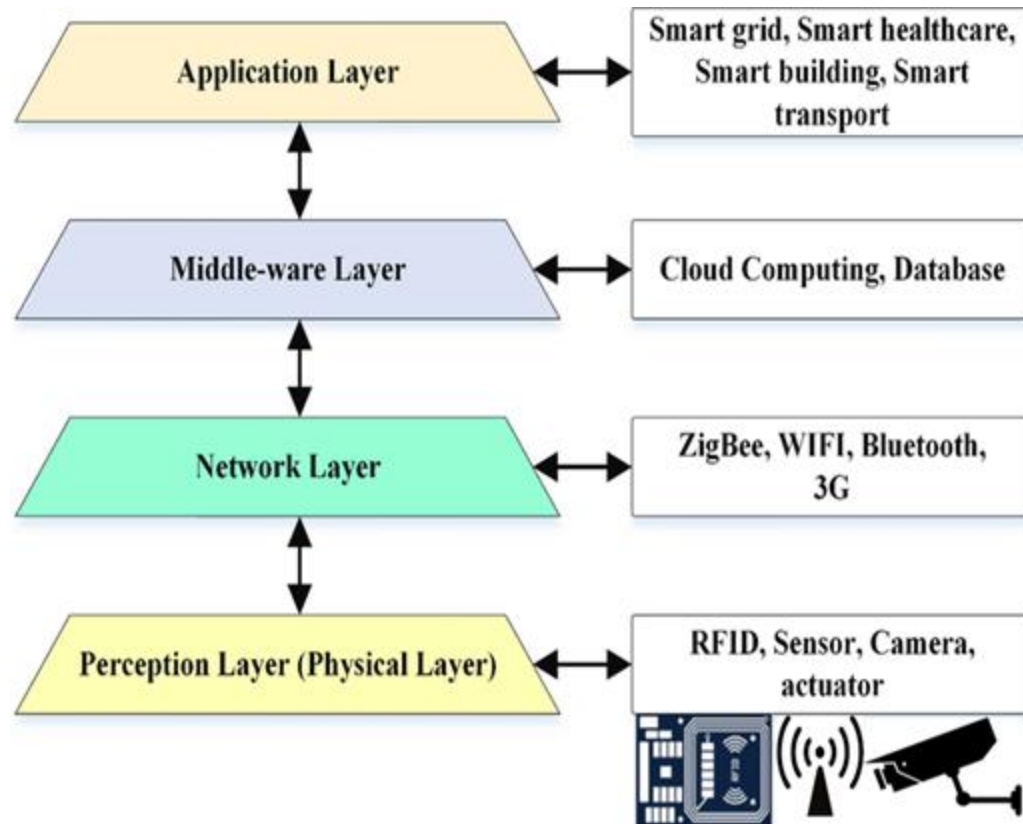
☐ Key Components:

- ☐ Sensors and actuators (For **data collection and action execution**).
- ☐ Connectivity Module (For **network communication**- Wi-Fi, Bluetooth, GSM, LoRa, Generic RF).
- ☐ Cloud Platform (For **data storage, processing, analytics, and visualization**).
- ☐ User Interface (For **monitoring, controlling, and interacting** with the IoT system).

Department of Computer Science and Engineering

Daffodil International University

IoT Architecture



Data importance in IoT Ecosystems

❑ **Enhancing Connectivity:**

- Facilitates real-time monitoring and decision-making.
- Enables remote access and control.

❑ **Efficiency and Automation:**

- Reduces manual intervention, optimizing processes.
- Supports predictive maintenance and resource management.

❑ **Security and Scalability:**

- Ensures secure data transmission with encryption and authentication.
- Scales across multiple devices and applications.

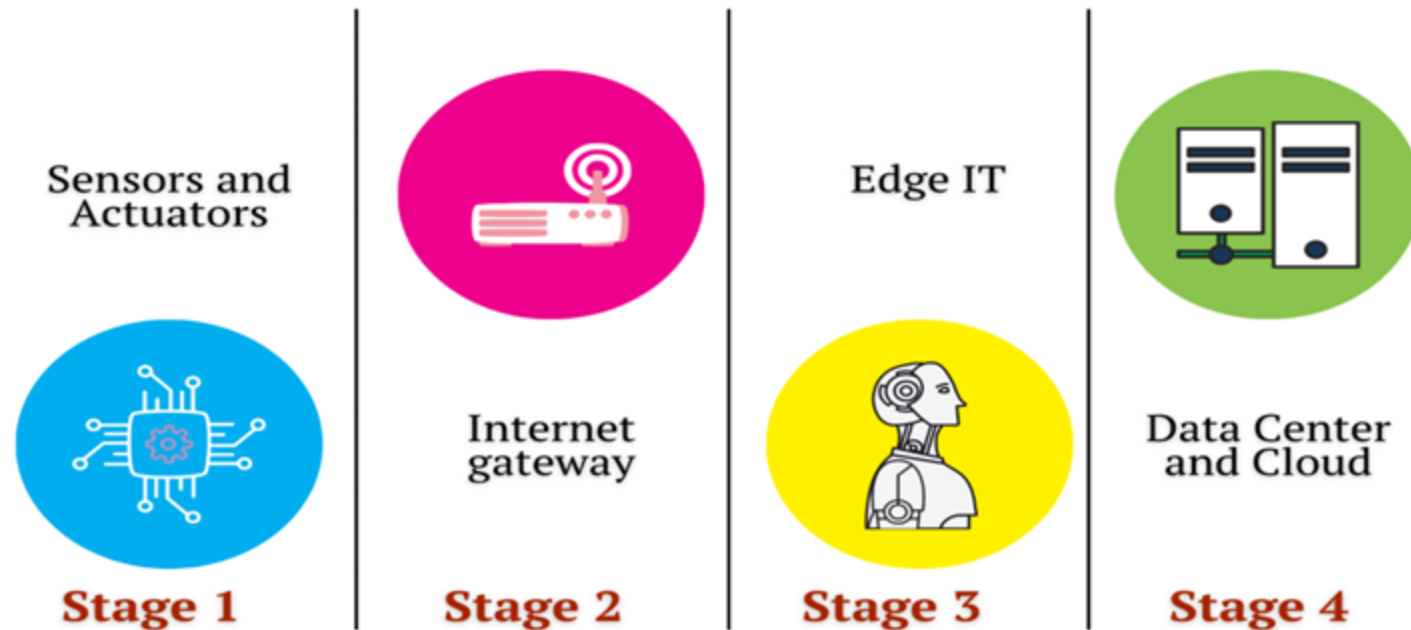
Department of Computer Science and Engineering

Daffodil International University

IoT Architecture Main Stages Overview

❑ What is IoT Architecture?

- A structured framework that defines how IoT devices communicate and process data.
- Comprises four main stages: Sensors & Actuators, Internet Gateway, Edge IT, and Data Center & Cloud.



Department of Computer Science and Engineering

Daffodil International University

IoT Architecture Main Stages Overview (Contd.)

❑ Stage 1: Sensors & Actuators

- Sensors collect environmental data (e.g., temperature, humidity, motion).
- Actuators respond based on processed data (e.g., motors, alarms, smart switches).
- Examples: RFID, infrared sensors, cameras.

❑ Stage 2: Internet Gateway

- Bridges sensor networks and the internet for data transmission.
- Uses protocols like Wi-Fi, Bluetooth, Zigbee, LoRaWAN, and 5G.
- Ensures secure and reliable data transfer to the next stage.

❑ Stage 3: Edge IT

- Processes data near the source, reducing latency.
- Uses edge computing devices like gateways and microcontrollers.
- Filters and pre-processes data before sending to the cloud.

IoT Architecture Main Stages Overview (Contd.)

☐ Stage 4: Data Center & Cloud

- Centralized storage and analysis of IoT data.
- Supports AI-driven analytics, machine learning, and predictive insights.
- Enables remote monitoring and control of IoT devices.

Department of Computer Science and Engineering

Daffodil International University

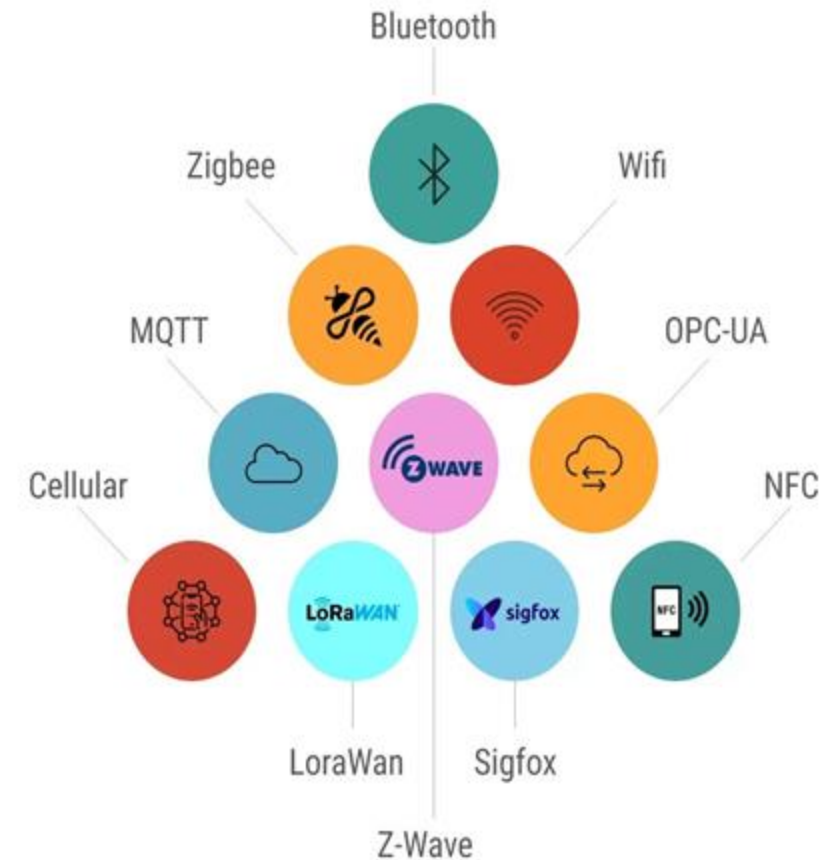
IoT Communication Protocols Overview

❑ What are IoT Communication Protocols?

- Rules that define how IoT devices exchange data efficiently and securely.
- Categorized into Short-Range, Long-Range, and Data communication Protocols.

❑ Why are they Important?

- Ensure seamless data transfer between IoT components.
- Optimize power consumption, bandwidth, and security.



Department of Computer Science and Engineering

Daffodil International University

Short-Range Communication Protocols

❑ Wi-Fi:

- **Range & Frequency:** Up to 100m, operates in 2.4 GHz & 5 GHz bands.
- **Use Cases:** High-speed connectivity for smart homes, industrial IoT, and multimedia streaming.
- **Pros:** High data rate, widely available.
- **Cons:** Limited range, high power consumption.

❑ Bluetooth:

- **Range & Frequency:** Up to 100m (Class 1), operates in the 2.4 GHz band.
- **Use Cases:** Short-range communication for wearables, sensors, and smart devices.
- **Variants:** Bluetooth Classic (higher throughput), Bluetooth Low Energy (BLE) (optimized for low power).
- **Pros:** Energy-efficient (BLE), widely supported.
- **Cons:** Limited range and data rate.

Short-Range Communication Protocols(Contd.)

❑ Zigbee:

- **Range & Frequency:** Up to 100m, operates in the 2.4 GHz band.
- **Use Cases:** Mesh networking for home automation, smart lighting, and industrial IoT.
- **Pros:** Low power, supports large networks.
- **Cons:** Lower data rate than Wi-Fi.

❑ Z-Wave:

- **Range & Frequency:** Up to 100m, operates in sub-GHz bands (e.g., 908.42 MHz in the U.S.).
- **Use Cases:** Smart home automation (security systems, lighting, HVAC control).
- **Pros:** Low power, strong mesh networking.
- **Cons:** Proprietary standard, fewer devices compared to Zigbee.

Department of Computer Science and Engineering

Daffodil International University

Long-Range Communication Protocols

❑ LPWAN (Low Power Wide Area Network)

- **Range & Frequency:** 10-40 km, varies by technology and region.
- **Use Cases:** Smart cities, agriculture, logistics, and remote monitoring.
- **Pros:** Low power, long-range communication.
- **Cons:** Limited data rates, high latency.

❑ LoRaWAN (Long Range Wide Area Network)

- **Range & Frequency:** Up to 15 km, operates on unlicensed sub-GHz bands (e.g., 868 MHz in Europe, 915 MHz in the U.S.).
- **Use Cases:** IoT sensors, environmental monitoring, asset tracking.
- **Pros:** Low power, long-range, bidirectional communication.
- **Cons:** Lower data rate than cellular technologies.

Department of Computer Science and Engineering

Daffodil International University

Long-Range Communication Protocols (Contd.)

❑ NB-IoT (Narrowband IoT)

- **Range & Frequency:** Several km, operates on licensed cellular bands (e.g., LTE bands).
- **Use Cases:** Smart meters, asset tracking, industrial IoT, smart parking.
- **Pros:** Optimized for deep indoor coverage, low power, secure.
- **Cons:** Requires carrier support, slightly higher cost than LoRaWAN.

❑ Sigfox

- **Range & Frequency:** Up to 50 km in rural areas, 10 km in urban areas; operates on sub-GHz ISM bands (e.g., 868 MHz in Europe, 915 MHz in the U.S.).
- **Use Cases:** Asset tracking, smart agriculture, environmental monitoring, logistics.
- **Pros:** Ultra-low power, cost-effective, global network availability.
- **Cons:** Limited data rate (few bytes per message), not ideal for real-time applications.

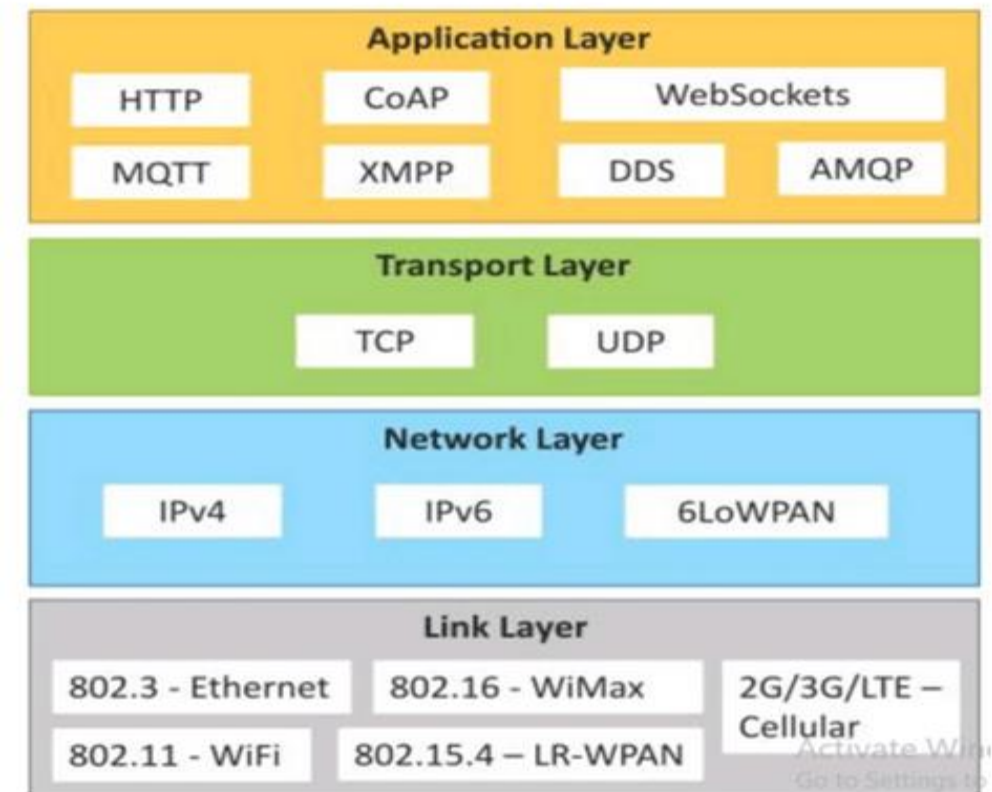
Department of Computer Science and Engineering

Daffodil International University

IoT Application Layer Protocols

❑ There are some Application protocols in IoT. The major protocols that are used in IoT are-

1. Message Queue Telemetry Transport (MQTT)
2. HyperText Transfer Protocol (HTTP)
3. WebSocket
4. Constrained Application Protocol (CoAP)
5. Data Distribution Service (DDS)
6. Advanced Message Queue Protocol (AMQP)
7. Extensible Messaging and Presence Protocol (XMPP)

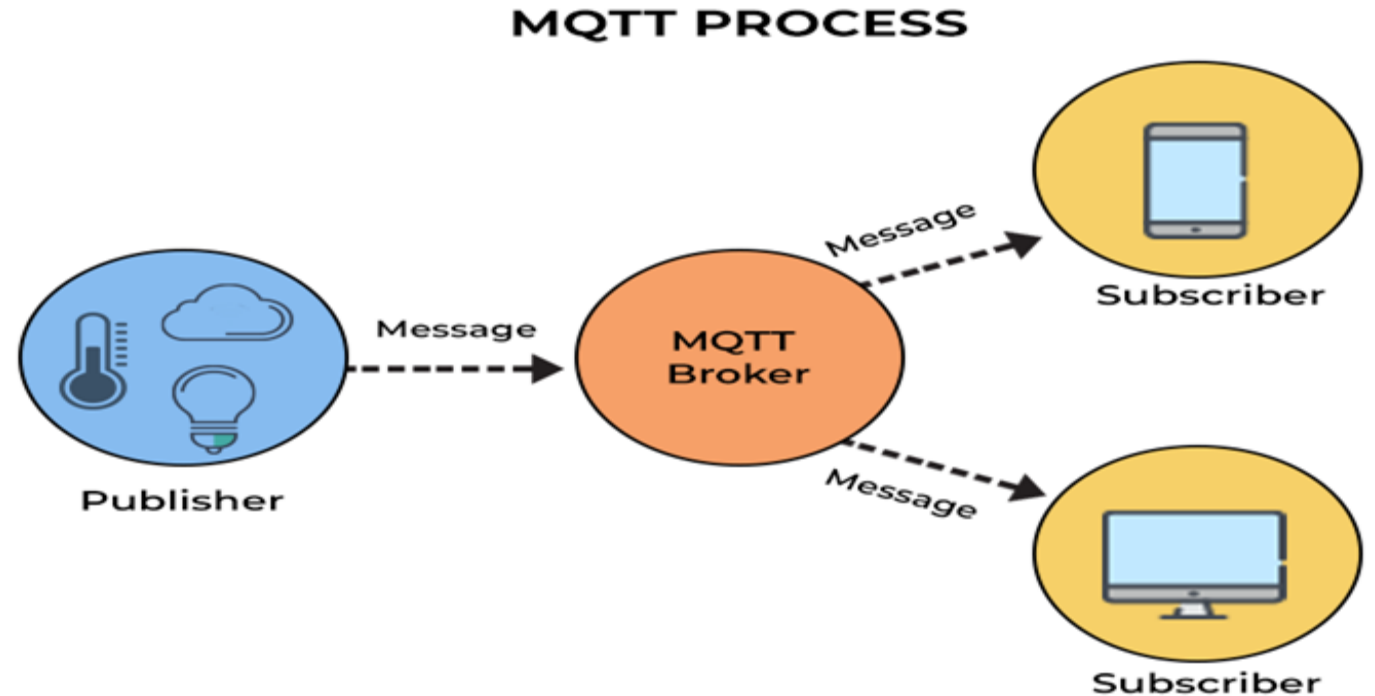


Department of Computer Science and Engineering

Daffodil International University

MQTT

MQTT, or Message Queuing Telemetry Transport, is a lightweight, publish-subscribe messaging protocol designed for resource-constrained devices and low-bandwidth networks

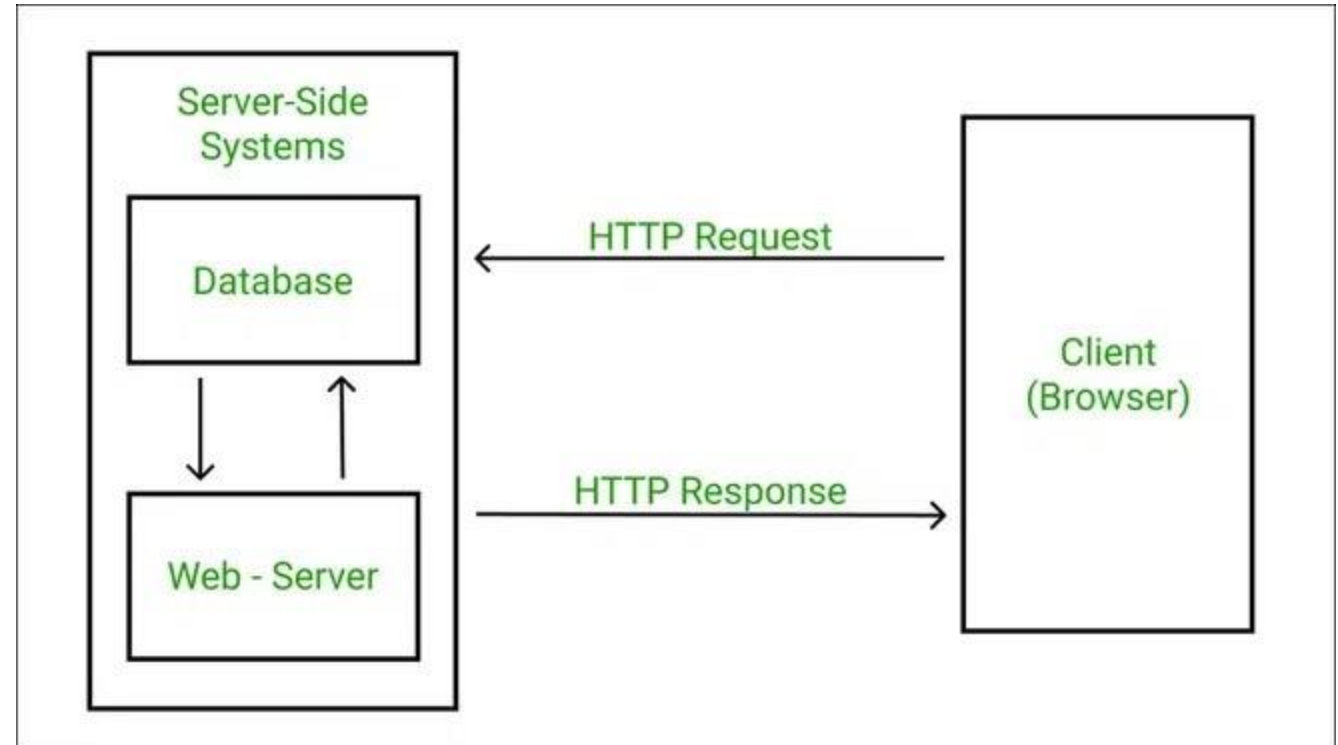


Department of Computer Science and Engineering

Daffodil International University

HTTP

HTTP (Hypertext Transfer Protocol) is a widely used application-layer protocol for communication between devices and servers, particularly through RESTful APIs.

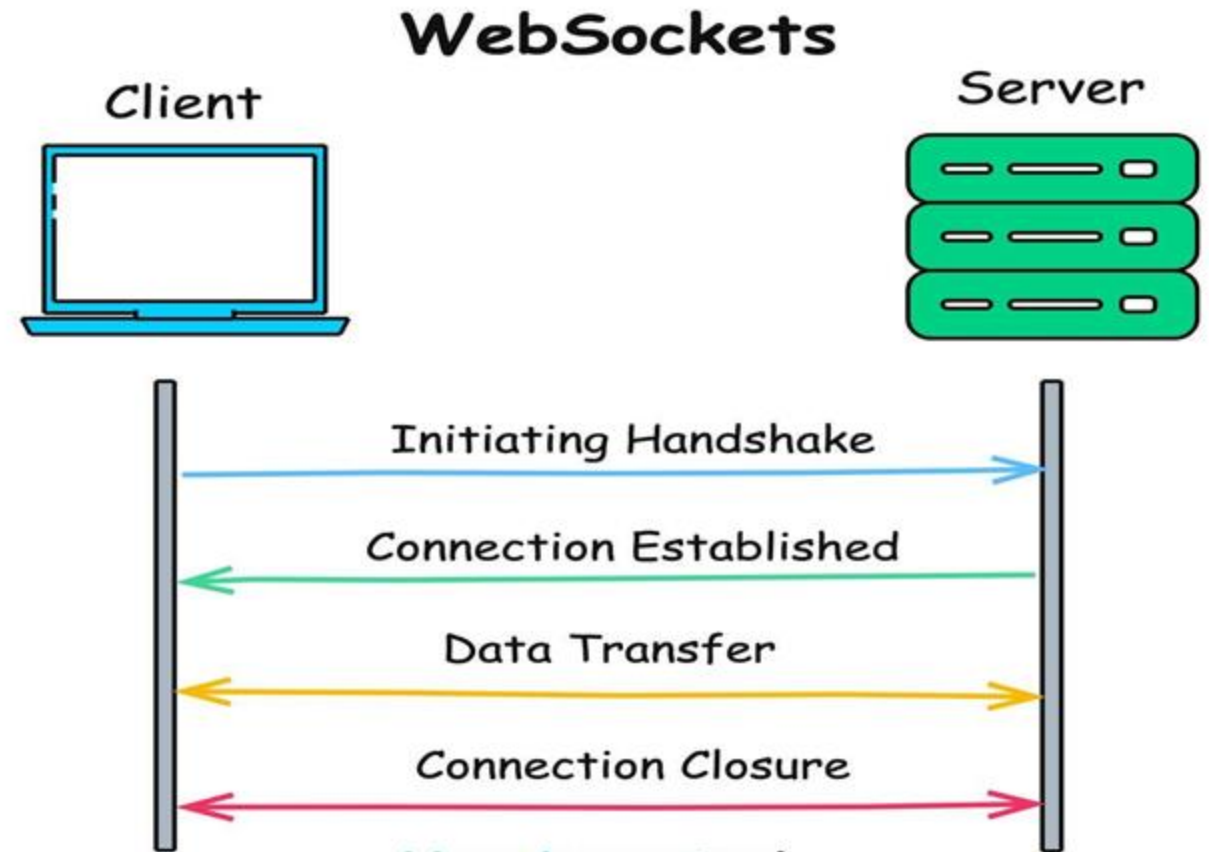


Department of Computer Science and Engineering

Daffodil International University

WebSocket

A WebSocket is a communication protocol that enables persistent, two-way (full-duplex) communication channels over a single TCP connection.

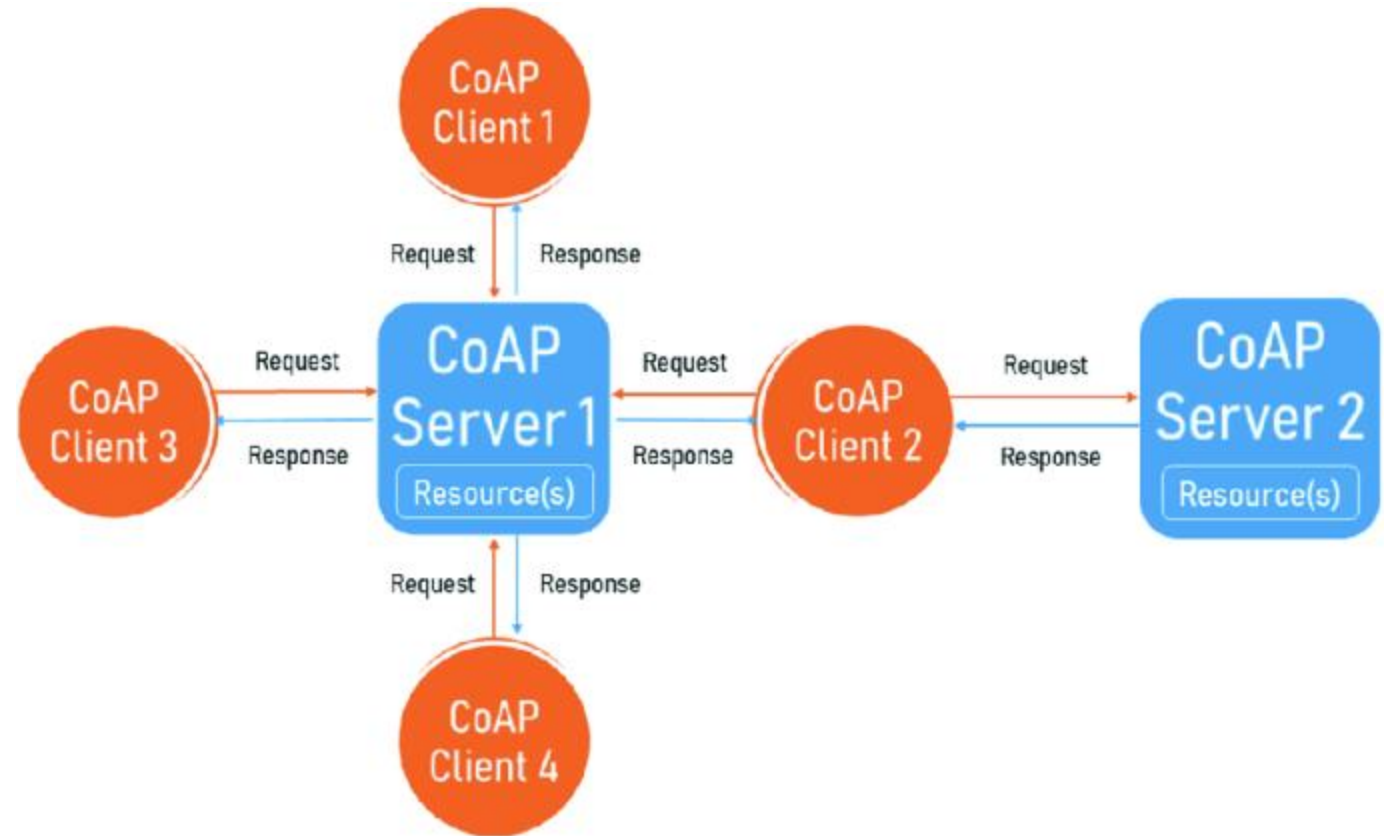


Department of Computer Science and Engineering

Daffodil International University

COAP

COAP, or Constrained Application Protocol, is a lightweight web transfer protocol designed for constrained devices and networks within the Internet of Things (IoT).

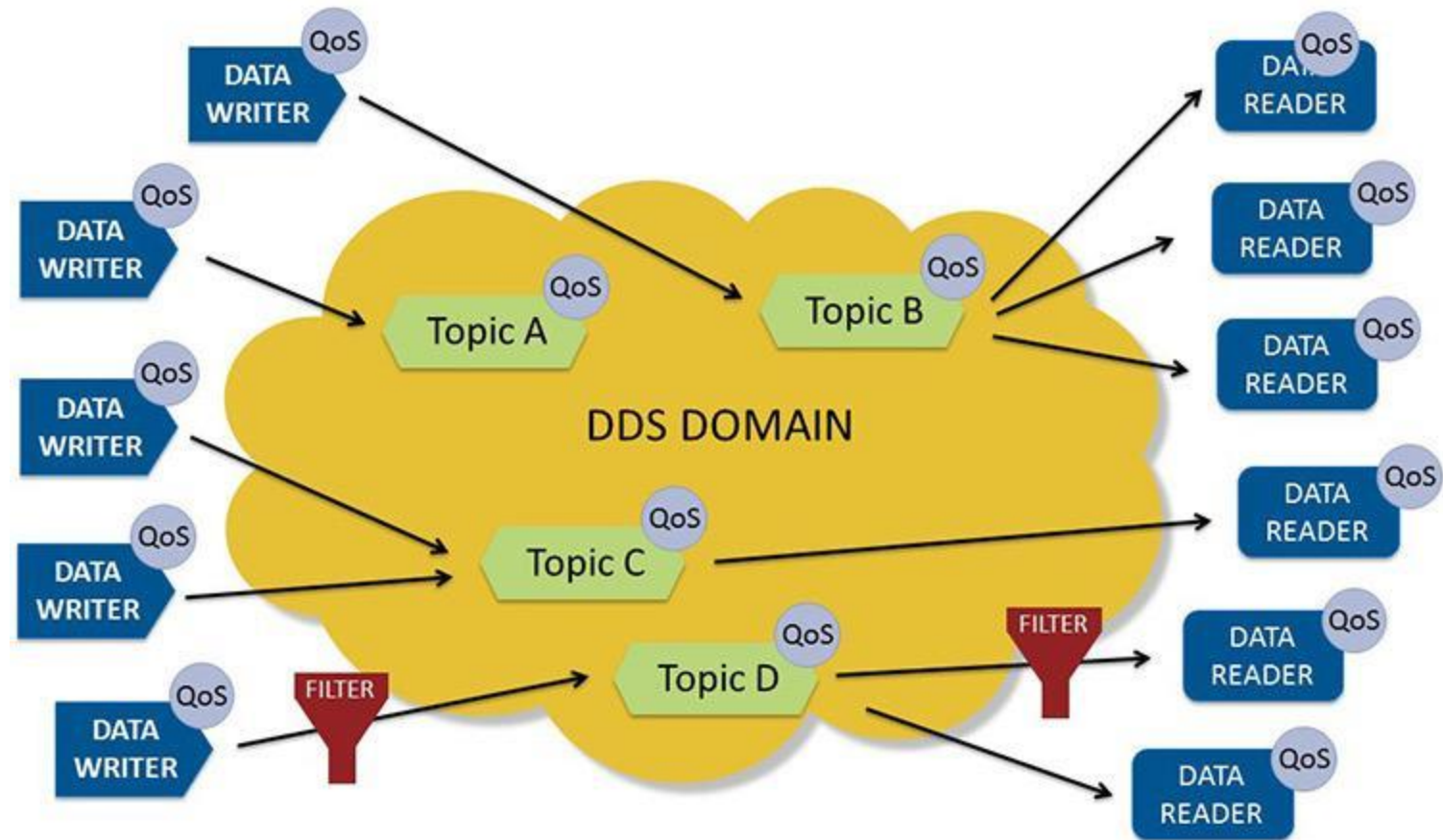


Department of Computer Science and Engineering

Daffodil International University

DDS

DDS (Data Distribution Service) is a protocol for IoT that enables interoperability between connected machines, enterprise systems, and mobile devices.

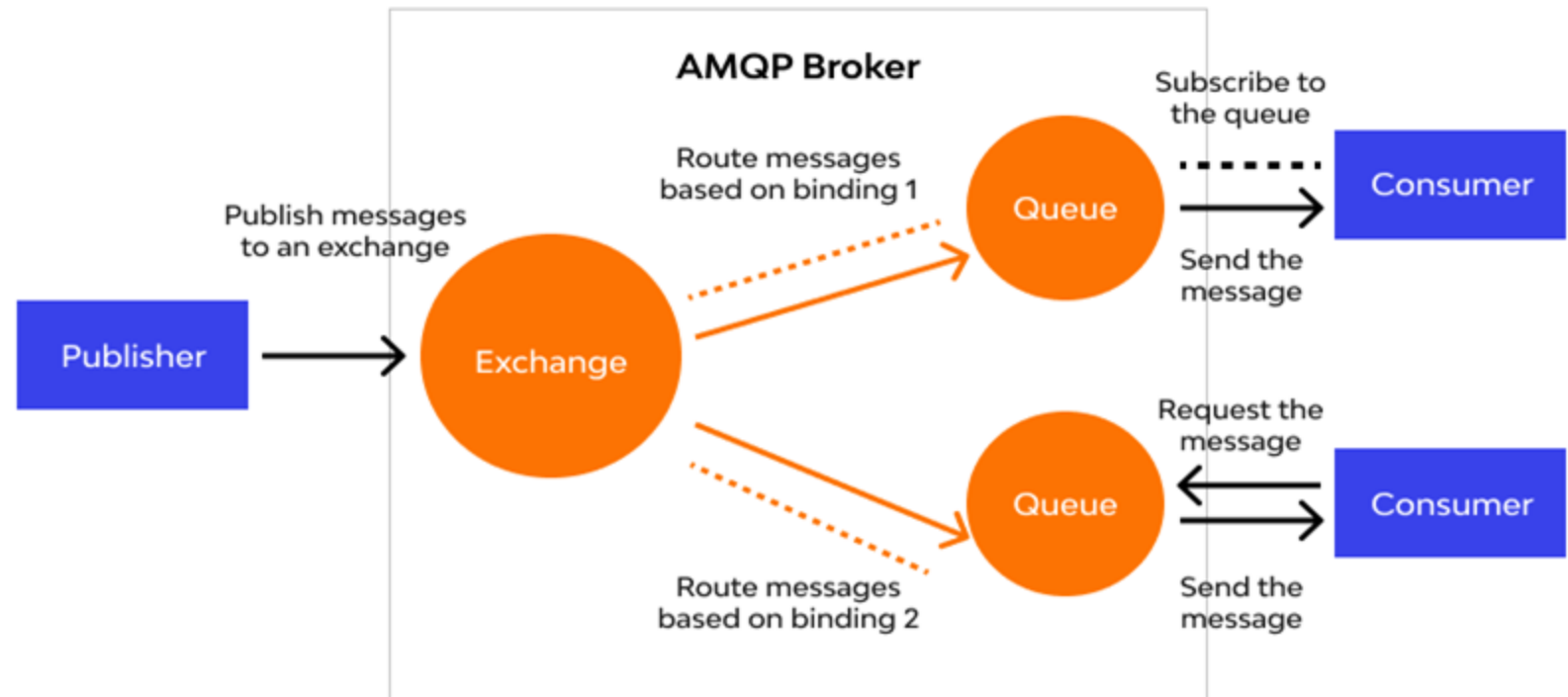


Department of Computer Science and Engineering

Daffodil International University

AMQP

The Advanced Message Queuing Protocol (AMQP) is an open standard application layer protocol used for reliable and secure message exchange between devices and applications.

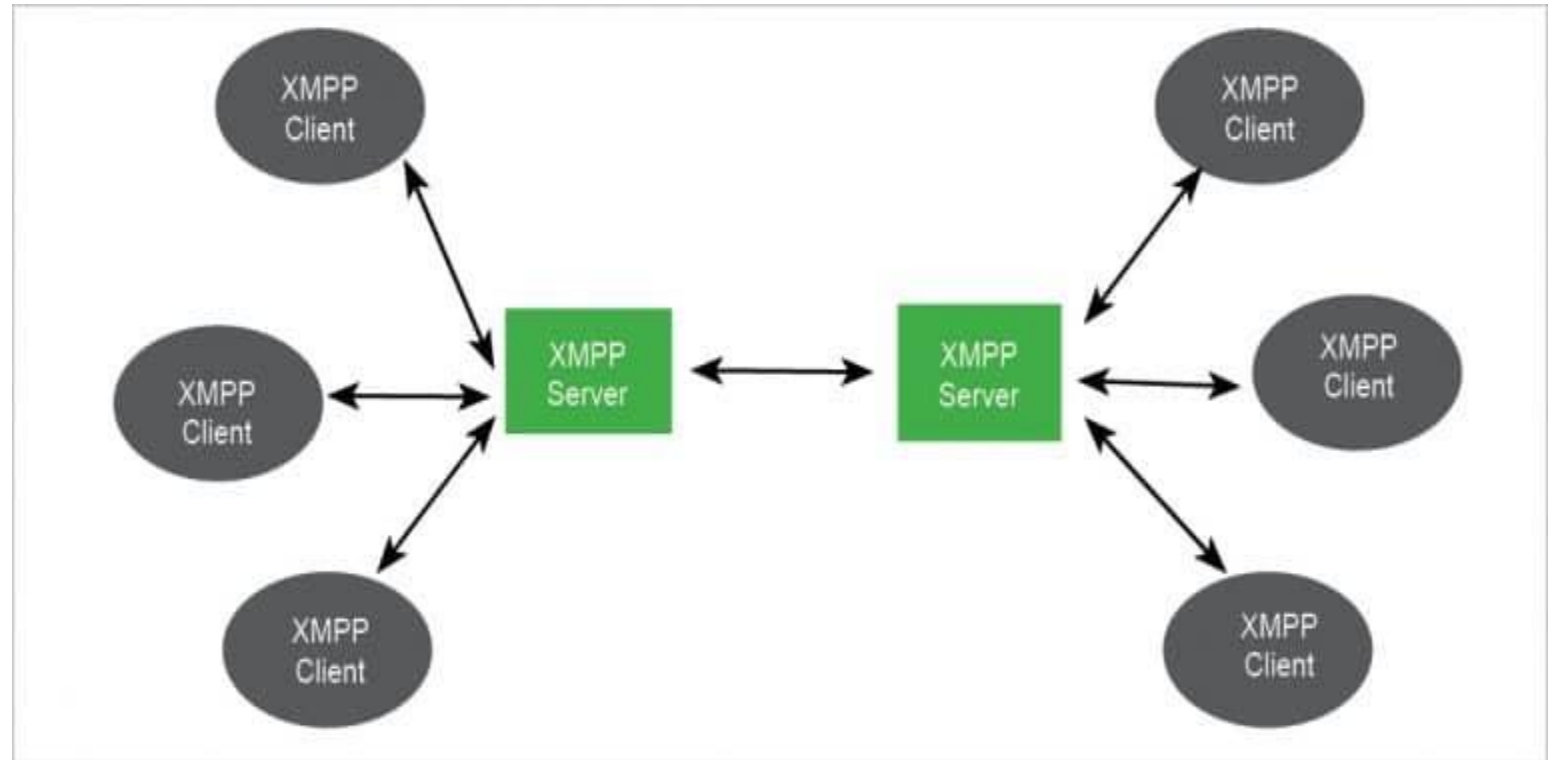


Department of Computer Science and Engineering

Daffodil International University

XMPP

XMPP, or Extensible Messaging and Presence Protocol, is an open, XML-based protocol used in IoT for real-time communication and data exchange.



Choosing the Right Protocol

☐ Factors to Consider:

- **Range:** Short-range (Wi-Fi, Bluetooth) vs. long-range (LoRaWAN, NB-IoT).
- **Power Consumption:** Low-power (Zigbee, LPWAN) vs. high-power (Wi-Fi).
- **Data Rate:** High (Wi-Fi, HTTP) vs. low (LoRaWAN, CoAP).
- **Security:** Stronger encryption (AMQP, HTTPS) for sensitive data.

☐ Selecting the right protocol depends on the IoT application's requirements.

☐ Combining multiple protocols can enhance performance and efficiency.

Challenges in IoT Data Communication

❑ Limited Bandwidth & Connectivity Issues

- High data traffic from numerous IoT devices leads to network congestion.
- Poor connectivity in remote or urban areas can severely limit performance.

❑ High Power Consumption

- Continuous data transmission leads to rapid battery drainage.
- Power-hungry IoT devices require energy-efficient solutions for longer operation.

❑ Scalability Issues

- As IoT networks grow, managing and maintaining large-scale networks becomes complex.
- Ensuring efficient communication across an increasing number of devices is a key challenge.

❑ Latency Problems

- High latency can negatively affect real-time applications such as autonomous vehicles and healthcare systems.
- Reducing delay in data transmission is crucial for IoT systems requiring fast responses.

Department of Computer Science and Engineering

Daffodil International University

Case Study: Smart Grid Communication Failure

- ❑ **Incident:** In 2015, a smart grid system in the U.S. experienced a failure where IoT-enabled smart meters stopped transmitting real-time energy data due to network congestion.
- ❑ **Impact:**
 - Customers were overcharged or undercharged due to incorrect readings.
 - Utility companies lost revenue due to billing errors.
 - Power distribution issues occurred because of delayed load balancing decisions.
- ❑ **Root Cause:** Overloaded IoT communication network, outdated protocol handling, and poor failover mechanisms.

Department of Computer Science and Engineering

Daffodil International University

Analysis of Causes and Consequences

❑ Causes of Communication Failure

- **Network Overload:** Too many IoT devices transmitting data simultaneously.
- **Lack of Data Prioritization:** No differentiation between critical and non-critical data.
- **Unoptimized Communication Protocols:** Use of HTTP instead of lightweight protocols like MQTT or CoAP.
- **Weak Error Handling:** No redundancy or fallback mechanisms.

❑ Consequences

- **Financial Loss:** Incorrect billing led to revenue loss and legal disputes.
- **Consumer Trust Issues:** Customers lost trust in smart metering reliability.
- **Operational Downtime:** Manual intervention required to correct errors, delaying energy distribution decisions.

Department of Computer Science and Engineering

Daffodil International University

Possible Solutions

☐ **Adopt Scalable Network Architectures**

- Implement edge computing to process data locally before sending it to the cloud.
- Use LoRaWAN or 5G to handle large-scale IoT deployments.

☐ **Use Efficient Communication Protocols**

- Switch from HTTP to MQTT or CoAP, which are designed for IoT.
- Implement data compression techniques to reduce payload size.

☐ **Implement Redundancy & Failover Mechanisms**

- Use multi-path routing to avoid single points of failure.
- Deploy backup cloud servers for automatic failover.

☐ **Prioritize & Optimize Data Transmission**

- Categorize data (e.g., critical vs. non-critical) and allocate bandwidth accordingly.
- Use adaptive data rate (ADR) algorithms to optimize network traffic.

Department of Computer Science and Engineering

Daffodil International University

CEP Problems

- ❑ A university research team is developing a **Smart Agriculture Monitoring System** to help farmers remotely monitor soil conditions, temperature, and humidity. The system consists of multiple sensors deployed across a large agricultural field, a central gateway that collects sensor data, and a cloud-based server for data storage and processing. Additionally, farmers should receive **real-time alerts** on their mobile applications if soil moisture levels drop too low. What messaging protocol (e.g., MQTT, CoAP, AMQP, XMPP) should the research team use for transmitting sensor data to the cloud and delivering real-time alerts to farmers?
- ❑ A hospital is developing a **Smart Healthcare Monitoring System** to track patient vitals in real time using wearable sensors. The system transmits data to a nursing station for immediate monitoring, uploads it to the cloud for remote analysis, and triggers emergency alerts if critical conditions arise, ensuring rapid medical response. Which messaging protocol (e.g., MQTT, CoAP, AMQP, XMPP) should be used to ensure reliable and low-latency communication between medical sensors, hospital servers, and cloud systems?

Department of Computer Science and Engineering

Daffodil International University

What are IoT Security?

- ❑ An IoT attack is a cyberattack that targets Internet of Things systems, which include physical devices, vehicles, buildings, and other objects embedded with software that enables them to collect or exchange data. As IoT grows, the number of cyber threats increase.
- ❑ Confidentiality, Integrity, and Availability (CIA) form a triad that fulfills the basic security needs. For security and privacy of the IoT devices, should satisfy the CIA triad. These all three components are important for the better security of the devices. So, these security doctrines apply as a whole to the IoT same as they apply on the Internet.

Department of Computer Science and Engineering

Daffodil International University

CIA

wallarm



Department of Computer Science and Engineering

Daffodil International University

CIA (Contd.)

- ❑ **Confidentiality:** Confidentiality ensures that access must be granted to the authorized user to the information and data reports. The access is subject to the extent of the need for access.
- ❑ **Integrity:** The Integrity is determined and ensured when the data is well secured and protected. The encrypted data only be modified by the authentic user during the process, transmission and storage.
- ❑ **Availability:** Availability of the data play an important role. Information security and authentication are very vital for data security. But, the availability of the data at the required time is a must. It is useless if the data is not available on time or in an emergency or critical situation.

Department of Computer Science and Engineering

Daffodil International University

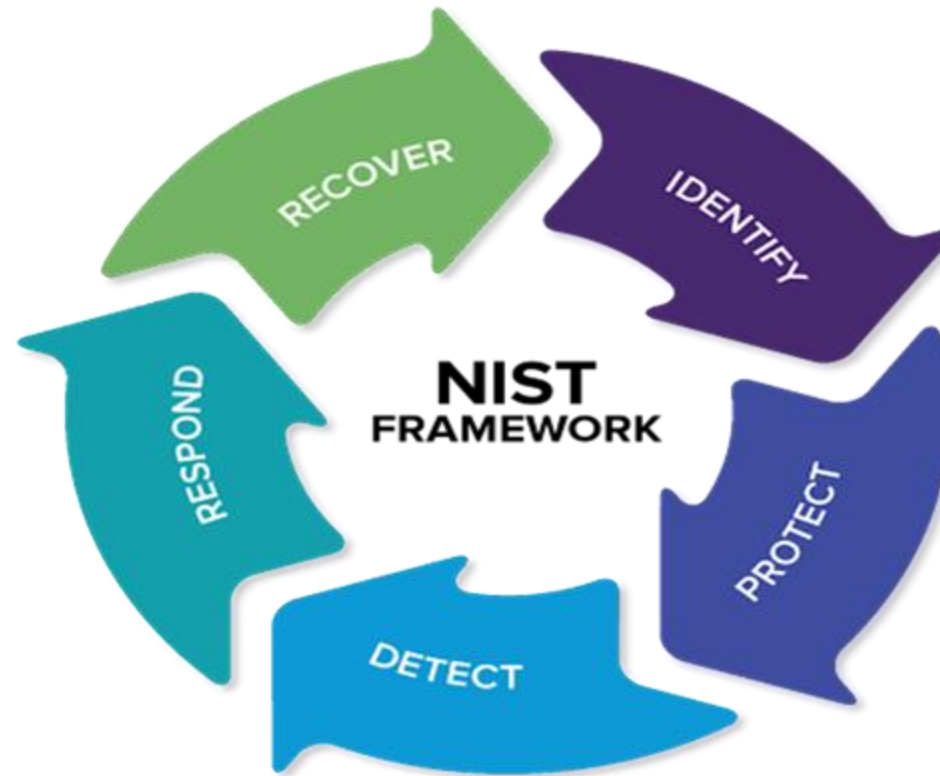
CIA (Contd.)

- ☐ For security and privacy of the IoT devices, it should satisfy the CIA triad. Components of the CIA triad are confidentiality, integrity, and availability. These all three components are important for the better security of the devices.
- ☐ It poses a significant impact to any individual or institution or organization involved in any CIA triad basic requirements is missed. As a definition for impacts provided by the NIST (the National Institute for Standards and Technology) suggesting High, Moderate, and Low potential impacts due to the loss of CIA.

Department of Computer Science and Engineering

Daffodil International University

NIST Framework



Department of Computer Science and Engineering

Daffodil International University

IoT Security



Types of IoT attacks

- ❑ IoT attacks are a cybercrime against Internet of Things devices. These devices can be vulnerable to hijacking due to weak IoT security measures, outdated firmware, and poor system design. Here are some of the most common types of IoT attacks:
 1. **Device spoofing:** A type of attack where a malicious device manipulates an authentic device's IP address, MAC address, or other identifying information and pretends to be a legitimate one.
 2. **Man-in-the-middle (MitM) attacks:** The concept of a MitM attack involves a hacker intercepting communication between two systems. The attacker impersonates the original sender to trick the other party into thinking they're receiving a legitimate message. MitM is usually carried out to extract sensitive information and disrupt services.
 3. **Distributed denial of service (DDoS) attacks.** DDoS attacks on IoT devices overload the network by flooding it with constant traffic, such as fake requests. This way, an attacker overwhelms the system, crashes it, and causes a denial of service to legitimate users.

Department of Computer Science and Engineering

Daffodil International University

Types of IoT attacks

4. **Eavesdropping.** Threat actors perform eavesdropping, also known as sniffing or spying, to intercept and listen to or monitor the communication between IoT devices.
5. **Malware attacks.** Cybercriminals install malicious software on IoT devices to gain unauthorized access to sensitive data, control the device, or spy on network activity or conversations.
6. **Zero-day attacks.** During a zero-day attack, a hacker exploits unpatched vulnerabilities in the software of IoT devices previously unknown to cybersecurity engineers. Such attacks are dangerous because there's no available fix during an attack.
7. **Password cracking.** Hackers use different methods, like brute force attacks, to decrypt system passwords and gain access to IoT devices. The weaker the default passwords and password practices, the easier it is for attackers to hijack IoT systems.
8. **Firmware manipulation.** In this type of attack, a cybercriminal modifies the firmware of an IoT device to alter its functionality and further perform malicious actions.

Department of Computer Science and Engineering

Daffodil International University

How IoT devices are attacked

- ❑ **Most IoT devices offer immense convenience and efficiency for our daily lives and routine tasks, whether at home or work. Aside from your laptop, smartphone, or tablet, which are relatively well secured, other devices like smart TVs or wearables to track your steps are more often exposed to cybersecurity threats, so it's important to consider the IoT network security. The following are the most common reasons why your IoT devices may become a sweet catch for hackers:**
 1. **Weak passwords:** One of the fundamental reasons IoT devices are a prime target for threat actors is default or easy-to-guess passwords that allow them entry into the device with little effort.
 2. **Unsecured cloud storage:** Lack of protection in cloud storage may allow hackers to manipulate or steal your confidential data easily.
 3. **Unpatched software:** Because outdated IoT software or firmware contain known vulnerabilities, it provides threat actors a loophole to exploit weaknesses.
 4. **Insecure network connections:** Public or unsecured Wi-Fi networks increase the risk of devices being attacked and ease hackers' process of gaining control over them.

Department of Computer Science and Engineering

Daffodil International University

How IoT devices are attacked

5. **Lack of encryption:** Anyone who intercepts your unencrypted traffic can read it. This applies to any traffic you choose to leave outside the secure VPN tunnel — exposing sensitive information is a major split tunneling security risk.
6. **Physical tampering:** Physical access to an IoT device may allow attackers to extract sensitive data, install malicious firmware, and compromise the security of your IoT devices.

How to protect IoT systems and devices

❑ Enterprises can use the following tools and technologies to improve their data protection protocols and security posture:

1. **Introduce IoT security during the design phase:** The IoT security risks and issues discussed, most can be overcome with better preparation, particularly during the research and development process at the start of any consumer- enterprise- or industrial-based IoT (IIoT) device development. Enabling security by default is critical, along with providing the most recent operating systems and using secure hardware.
2. **PKI and digital certificates:** PKI can secure client-server connections between multiple networked devices. Using a two-key asymmetric cryptosystem, PKI can facilitate the encryption and decryption of private messages and interactions using digital certificates. These systems help to protect the clear text information input by users into websites to complete private transactions. E-commerce wouldn't be able to operate without the security of PKI.

Department of Computer Science and Engineering

Daffodil International University

How to protect IoT systems and devices (Contd.)

3. **Network security:** Networks provide a huge opportunity for threat actors to remotely control IoT devices. Because networks involve both digital and physical components, on-premises IoT security should address both types of access points. Protecting an IoT network includes ensuring port security, disabling port forwarding and never opening ports when not needed; using antimalware, firewalls, intrusion detection systems and intrusion prevention systems; blocking unauthorized IP addresses; and ensuring systems are patched and up to date.
3. **API security:** APIs are the backbone of most sophisticated websites. They enable travel agencies, for example, to aggregate flight information from multiple airlines into one location. Unfortunately, hackers can compromise these channels of communication, making API security necessary for protecting the integrity of data being sent from IoT devices to back-end systems and ensuring only authorized devices, developers and apps communicate with APIs.

Department of Computer Science and Engineering

Daffodil International University

How to protect IoT systems and devices (Contd.)

5. **Network access control (NAC):** NAC can help identify and inventory IoT devices connecting to a network. This provides a baseline for tracking and monitoring devices.
6. **Segmentation:** IoT devices that need to connect directly to the internet should be segmented into their own networks and have restricted access to the enterprise network. Network segments should monitor for anomalous activity, taking action if an issue is detected.
7. **Security gateways:** Acting as an intermediary between IoT devices and the network, security gateways have more processing power, memory and capabilities than the IoT devices themselves, which lets them add features such as firewalls to ensure hackers can't gain access to the IoT devices they connect.
8. **Patch management and continuous software updates:** It's critical to provide a way to update devices and software either over network connections or through automation. Having a coordinated disclosure of vulnerabilities is also important for updating devices as soon as possible. Consider end-of-life strategies as well.

Department of Computer Science and Engineering

Daffodil International University

How to protect IoT systems and devices (Contd.)

9. **Training:** IoT and operational system security are new to many existing security teams. Security staff must keep up to date with new or unknown systems, learn new architectures and programming languages, and be ready for new security challenges. Cybersecurity teams should receive regular cybersecurity training to keep up with modern threats and security measures.
10. **Team integration:** Along with training, integrating disparate and regularly siloed teams can be useful. For example, having programming developers work with security specialists can help ensure the proper controls are added to devices during the development phase.
11. **Consumer education:** Consumers must be made aware of the dangers of IoT systems and provided steps to stay secure, such as updating default credentials and applying software updates. Consumers can also play a role in requiring device manufacturers to create secure devices and refusing to use those that don't meet high-security standards.

Department of Computer Science and Engineering

Daffodil International University

How to protect IoT systems and devices (Contd.)

12. **Enforcement and automation of zero-trust policies:** The zero-trust model dictates that all users whether inside or outside the organization's network must be verified, authorized and continually evaluated for security configuration and posture before being given access to applications and data. Automating zero-trust policies and enforcing them across the board can help mitigate security threats against IoT devices.
13. **Multifactor authentication (MFA):** MFA adds an extra layer of security by requiring more than one form of identification when requesting access to a device or network. By enforcing MFA policies, both enterprises and home users can improve the security of IoT devices.
14. **Machine learning (ML):** ML technology can be used to secure IoT devices by automating the management and scanning of devices throughout the entire network. Since every device connected to the network is scanned, it stops assaults automatically before IT teams are alerted. That's what happened in 2018 when Microsoft Windows Defender software stopped a Trojan malware attack in 30 minutes.

Department of Computer Science and Engineering

Daffodil International University

Case Study: Major IoT Security Breach – Mirai Botnet Attack

❑ What Happened?

- In 2016, the Mirai botnet infected hundreds of thousands of IoT devices (CCTV cameras, routers, DVRs).
- The botnet launched a massive Distributed Denial of Service (DDoS) attack, bringing down major websites like Twitter, Netflix, Reddit, and GitHub.
- Attackers exploited default usernames and passwords on IoT devices to take control of them.

❑ Key Facts:

- The attack targeted **Dyn, a major DNS provider**, causing global internet disruptions.
- Devices were turned into "**zombie**" bots to flood servers with malicious traffic.
- One of the **largest DDoS attacks in history**, peaking at **1.2 terabits per second**.

Department of Computer Science and Engineering

Daffodil International University

Case Study: Major IoT Security Breach – Mirai Botnet Attack

❑ Scanning & Infection

- Mirai scanned the internet for vulnerable IoT devices with default credentials.
- Once a device was infected, it reported back to the command-and-control (C2) server.

❑ Botnet Formation

- The infected IoT devices were hijacked and turned into bots.
- These bots were then used to send massive traffic to target servers.

❑ DDoS Attack Execution

- The botnet launched DDoS attacks against major services, overloading servers and causing widespread internet outages.

❑ Impact Analysis

- **Global Internet Disruptions** – Major platforms went offline.
- **Business Losses** – Companies like Twitter, Amazon, and PayPal **suffered downtime**.
- **Security Crisis** – Exposed the **weak security of IoT devices**, leading to regulatory discussions.

Department of Computer Science and Engineering

Daffodil International University

Case Study: Major IoT Security Breach – Mirai Botnet Attack

☐ **How to Protect IoT Devices from Similar Attacks**

- Strong Authentication & Password Policies
- Change default usernames & passwords on IoT devices.
- Implement Multi-Factor Authentication (MFA) for device access.

☐ **Network Segmentation & Traffic Filtering**

- Isolate IoT devices on separate networks from critical systems.
- Use firewalls & Intrusion Detection Systems (IDS) to detect abnormal traffic.

☐ **Automatic Firmware Updates & Patching**

- Ensure IoT devices receive regular security updates to fix vulnerabilities.
- Disable remote access & unused services to reduce attack vectors.

☐ **DDoS Mitigation Strategies**

- Implement rate limiting & traffic filtering on network infrastructure.
- Use cloud-based DDoS protection services to absorb attacks.

Department of Computer Science and Engineering

Daffodil International University

CEP Problems

- ❑ A manufacturing company's **Industrial IoT (IIoT) system** controls robotic arms, conveyor belts, and sensors for real-time automation. Recently, operators noticed unauthorized robotic movements and production delays. Investigation revealed a possible cyber attack, where an attacker intercepted communication between IIoT devices and the control system, potentially disrupting operations. Identify a **possible attack** that could have compromised the IIoT system. What security mechanisms can be applied to **secure communication, authenticate devices, and detect anomalies** in the IIoT environment?
- ❑ A family's **Smart Home Automation System** controls locks, cameras, lights, and temperature via a mobile app. Recently, they noticed unauthorized lock activity and camera disruptions. Investigation revealed a cyberattack compromising the system's security. Identify a **potential attack** that could have compromised the smart home system. How does this attack affect the **Confidentiality, Integrity, and Availability (CIA)** of the system? What security measures can be implemented to **detect, prevent, and mitigate** this type of attack in an IoT-based smart home environment?

Introduction to STM 32

- Course Title: Embedded Systems & IoT
- Course Code: CSE233

Dept. of CSE, DIU

Agenda

- **Overview of STM32**
- **STM32 Development Environment**
- **Setting up the First STM32 Project**
- **STM32 Microcontroller Architecture**

Overview of STM32



STM32 is a family of 32-bit microcontrollers from STMicroelectronics, based on ARM Cortex-M cores.

It includes various series like

- STM32F0 (**entry-level**),
- STM32F4 (**high-performance**),
- STM32L (**low-power**), and others.

STM32 offers a wide range of peripherals, power management features, and processing power, making it suitable for embedded applications.

Overview of STM32



STM32F103C8T6 Full Form & Meaning:
“Blue Pill” or STM32F103C8T6 name has a meaning behind it.

STM – stands for the manufacturer’s name
STMicroelectronics

32 – stands for 32-bit ARM architecture

F103 – stands to indicate that the architecture ARM
Cortex M3

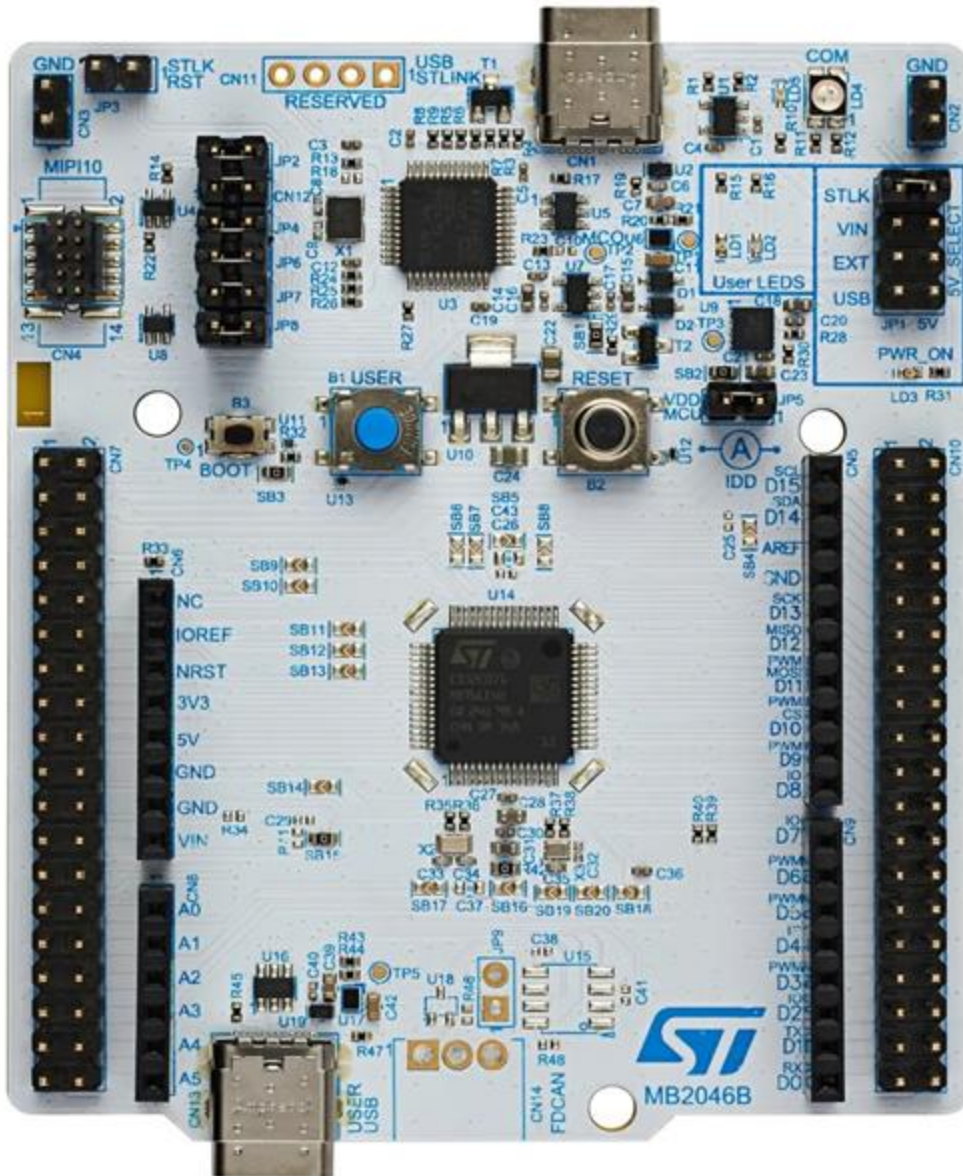
C – 48-pin

8 – 64KB Flash memory

T – package type is LQFP

6 – operating temperature -40°C to +85°C

STM32 Nucleo-64



- Supports Arduino and ST morpho connectivity
- ARDUINO® Uno V3 expansion connector
- 32.768 kHz crystal oscillator
- 1 user and 1 reset push-buttons
- 24 MHz or 48 MHz HSE
- USB Type-C®, Micro-B, or Mini-B connector for the ST-LINK
- CAN FD header
- Memories: 256kB Flash, 48kB RAM

Why use stm32 over Arduino?

	Uno	STM32 Blue Pill
Pricing	\$20-23	\$3-5
I/O Pins	14	37
PWM Pins	6	15
Analog In Pins	6	10
Flash Memory	32 kB	64 kB
SRAM	2 kB	20 kB
Clock speed	16 MHz	72 MHz
Voltage Level	5V	3.3 V
USB Connectivity	Standard A/B USB	Micro-USB
Hardware Serial Ports	1	3
I2C Support	Yes (1x)	Yes (2x)

STM32 vs ESP32

Point	ESP32	STM32
1. Core Architecture	Tensilica Xtensa dual-core 32-bit	ARM Cortex-M series (M0, M3, M4, M7, etc.)
2. Clock Speed	Up to 240 MHz	Varies (typically 32–216 MHz depending on model)
3. RAM	~520 KB SRAM	Varies (4 KB to 512 KB SRAM depending on model)
4. Flash Memory	Typically 4 MB external (via SPI), up to 16 MB	Varies (16 KB to 2 MB internal Flash)
5. Wireless Connectivity	Built-in Wi-Fi (802.11 b/g/n) and Bluetooth (Classic + BLE)	No built-in Wi-Fi/Bluetooth (requires external modules)

STM32 vs ESP32

Point	ESP32	STM32
6. ADC/DAC	12-bit ADC, 2 x 8-bit DAC	Up to 16-bit ADC, some with DAC (typically 12-bit)
7. GPIO Pins	~34 GPIOs with multiple functions	Varies widely, typically 20–168 GPIOs
8. Power Consumption	Higher idle power, but supports deep sleep (~10 uA)	Generally lower power, excellent ultra-low power modes (nanoamp range in stop mode)
9. Development Environment	ESP-IDF (official), Arduino, MicroPython	STM32CubeIDE, Keil, IAR, Arduino (for some models), MicroPython
10. Cost and Availability	Low-cost, widely available for IoT/Wi-Fi projects	Varies by model, generally affordable but higher cost for advanced models

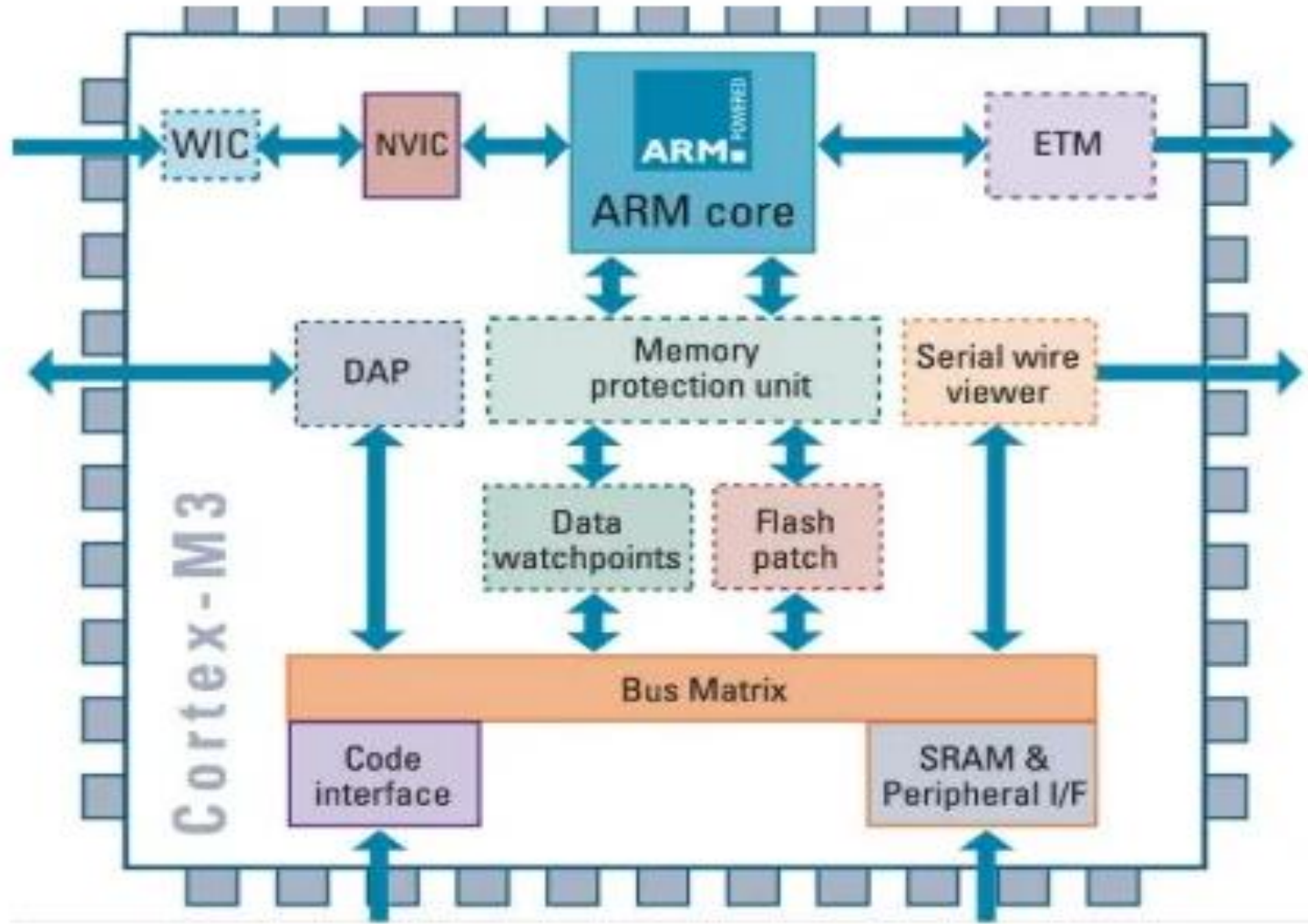
Top IDEs for STM32 Development:

- Keil MDK (Microcontroller Development Kit)
- STM32CubeIDE
- PlatformIO with VSCode
- Atollic TrueSTUDIO
- IAR Embedded Workbench
- Eclipse IDE for STM32 Development

STM32CubeIDE

STM32CubeIDE is an **official IDE** developed by STMicroelectronics, specifically tailored for STM32 microcontrollers. It offers seamless integration with STM32CubeMX, a graphical tool for hardware configuration, which speeds up the initialization process. The IDE is based on Eclipse and provides an extensive range of plugins to enhance productivity. Moreover, STM32CubeIDE comes with advanced debugging features like runtime variable monitoring and data visualization.

ARM Architecture



ARM stands for **Advanced Risk Machine**.

It is one of the most popular architectures used in devices like cameras, mobile phones and embedded system devices.

These are known for their low-power-consumption vs better performance abilities.

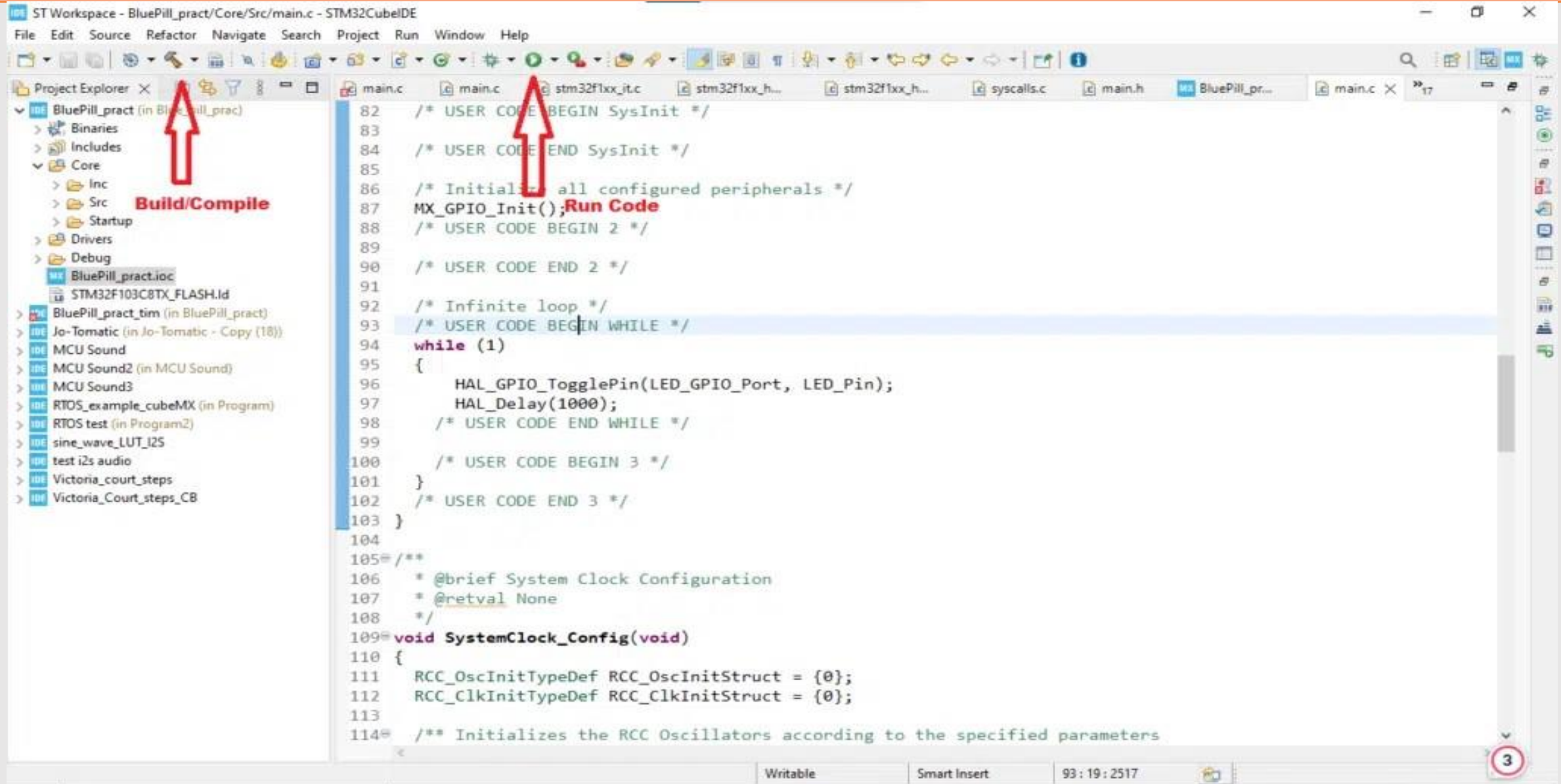
STM32 project to blink an LED



```
while (1)
{
    // Toggle the LED
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);

    // Wait for 500 ms
    HAL_Delay(500);
}
```

STM32 project to blink an LED



Q1

Daffodil Smart City aims to improve the safety, security, and environmental quality of its community areas, including gates, parks, walkways, swimming pools, and parking spaces. Residents often use these spaces early in the morning and during the evening, requiring well-lit, safe, and environmentally comfortable conditions. Additionally, ensuring security during low-activity periods and after hours remains a challenge for the management team. **Design** a cost-effective, scalable IoT-based smart system using the STM32 platform to monitor air quality, temperature, light levels, and motion, while also automatically controlling streetlights and security alarms as needed.

Complex Engineering Problems (CEP) with STM32 and ESP32



Problem Overview

Daffodil Smart City needs to enhance the safety, security, and environmental quality of its community areas (gates, parks, walkways, swimming pools, parking spaces). These areas are frequently used during early mornings and evenings, necessitating well-lit, safe, and comfortable conditions. Additionally, monitoring security during off-hours or low-activity periods poses a challenge for city management. The goal is to design a cost-effective, scalable IoT system based on STM32 microcontrollers to automatically monitor environmental parameters (air quality, temperature, light) and detect motion, controlling streetlights and security alarms accordingly.

System Requirements

The system must:

- Monitor **air quality**, **temperature**, **humidity**, **light intensity**, and **motion**.
- Automatically control **street lights** based on motion and light conditions.
- Activate **security alarms** when suspicious motion is detected during restricted times.
- Send real-time data to a central **dashboard** via wireless connectivity.
- Be **low-cost**, **energy-efficient**, and **scalable**.

Solution of the CEP with STM32 and ESP32

Components Required

Category	Component
Microcontroller	STM32F103C8T6
Wi-Fi Module	ESP32
Air Quality Sensor	MQ135
Temperature Sensor	DHT22
Light Sensor	LDR
Motion Sensor	PIR Sensor
Actuator	Relay Module
Alarm Device	Buzzer or Siren
Lighting Device	LED Streetlight
Power Supply	Solar Panel + Battery
Supporting Parts	Resistors, Capacitors, Voltage Regulator, Cables

System Design

Functional Description

✓ STM32F103C8T6:

- Acts as the main controller. It reads sensor data from the LDR, DHT22, MQ135, and PIR sensors. It makes real-time decisions such as turning ON lights or triggering alarms based on predefined thresholds and timing rules.

✓ ESP32:

- Handles wireless communication. It receives processed data from STM32 via UART and uploads it to a cloud server (e.g., Firebase or ThingsBoard). It can also receive commands for remote control if needed.

✓ MQ135:

- Monitors air quality by detecting the concentration of gases like CO₂, NH₃, benzene, etc., helping assess environmental health.

System Design

✓ DHT22:

- Monitors ambient temperature and humidity levels to assess comfort in public spaces.

✓ LDR:

- Measures ambient light intensity. STM32 uses this data to decide if it's dark enough to turn on streetlights.

✓ PIR Sensor:

- Detects motion of people. Used to trigger lights and alarms, particularly during inactive hours.

✓ Relay Module:

- Connected to STM32 to control power to the streetlight and buzzer (alarm). Acts like a switch based on logic decisions from STM32.

System Design

✓ Streetlight & Buzzer:

- Turned ON or OFF depending on light/motion conditions or intrusion detection.

✓ Solar Panel + Battery:

- Provides an eco-friendly and off-grid power source suitable for outdoor installations.

Software Design

Functional Flow

1. Initialization:

- Initialize STM32 peripherals (GPIO, ADC, UART, timers).
- Initialize sensors (air quality, temperature, light, motion).
- Configure relay control pins.

2. Sensor Data Acquisition:

- Read air quality sensor data.
- Read temperature sensor data.
- Read ambient light sensor value.

Software Design

1. Decision Making:

- If ambient light is below threshold *and* motion is detected → turn ON streetlights.
- If no motion for a defined timeout → turn OFF streetlights.
- If motion detected during off-hours (e.g., midnight to 6 am) → activate security alarm.
- Continuously monitor air quality and temperature to log or send alerts if outside safe ranges.

2. Output Control:

- Activate/deactivate relays for streetlights and security alarms accordingly.

3. Optional IoT Communication:

- Transmit sensor data and alerts to remote server or user app via Wi-Fi/Bluetooth.

Conclusion

- The designed STM32-based IoT system provides a smart, automated solution for enhancing safety, security, and environmental monitoring in the Daffodil Smart City. By integrating air quality, temperature, light, and motion sensors, the system dynamically controls street lighting and security alarms based on real-time conditions and usage patterns. The modular and scalable design allows future expansion with IoT communication capabilities and additional sensors or actuators, supporting sustainable smart city development at a cost-effective scale.